

INFORMATIQUE • 1ÈRE TI

MODULE 1 : ALGORITHMIQUE

LEÇON N°1

ÉCRITURE D'ALGORITHMES
SÉQUENTIELS ET STRUCTURES
DE CONTRÔLE

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon introduit les fondements de l'algorithmique : variables, constantes, instructions de base et structures de contrôle (alternatives et itératives). Elle donne à l'élève les outils pour écrire des algorithmes séquentiels corrects et structurés, compétence clé du module Algorithmique et du Bac TI.

- À la fin de cette leçon, je sais définir les notions de variable, de constante, d'instruction et d'algorithme séquentiel
- À la fin de cette leçon, je sais déclarer et utiliser correctement des variables et des constantes avec leurs types
- À la fin de cette leçon, je sais utiliser les instructions de base (lecture, écriture, affectation) pour écrire un algorithme séquentiel
- À la fin de cette leçon, je sais citer des exemples de structures de contrôle
- À la fin de cette leçon, je sais utiliser les structures alternatives Si...Alors et Si...Alors...Sinon
- À la fin de cette leçon, je sais identifier les instructions d'initialisation, d'incrémenta-tion et de décrémentation

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|---|---|
| 1. Notions fondamentales : variable, constante, instruction, algorithme | Répéter |
| 2. Instructions de base : lecture, écriture, affectation | 6. Synthèse : écrire un algorithme complet et structuré |
| 3. Structures de contrôle : panorama et structures alternatives | 7. Activité d'intégration |
| 4. Initialisation, incrémentation, décrémentation | 8. Méthodes et exercices résolus |
| 5. Structures itératives : Pour, Tant que, | 9. Exercices |
| | 10. Corrigés des exercices |
| | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir les notions de variable, de constante, d'instruction et d'algorithme séquentiel
- À la fin de cette leçon, je sais déclarer et utiliser correctement des variables et des constantes avec leurs types
- À la fin de cette leçon, je sais utiliser les instructions de base (lecture, écriture, affectation) pour écrire un algorithme séquentiel
- À la fin de cette leçon, je sais citer des exemples de structures de contrôle
- À la fin de cette leçon, je sais utiliser les structures alternatives Si...Alors et Si...Alors...Sinon
- À la fin de cette leçon, je sais identifier les instructions d'initialisation, d'incrémentation et de décrémentation
- À la fin de cette leçon, je sais identifier une boucle et sa condition d'arrêt
- À la fin de cette leçon, je sais utiliser les structures itératives Pour...Faire, Tant que...Faire et Répéter...Jusqu'à

PRÉREQUIS

- Notion d'algorithme vue en classe de Seconde (suite d'actions pour résoudre un problème)

- Opérations arithmétiques élémentaires et opérateurs (division entière, reste)
- Expressions logiques simples (comparaisons : $=$, $<$, $>$, $\leq$, $\geq$, $\neq$)
- Notion de problème et de solution (famille de situations du module)
- Lecture et interprétation d'un énoncé mathématique simple

1 NOTIONS FONDAMENTALES : VARIABLE, CONSTANTE, INSTRUCTION, ALGORITHME

Imagine Aminatou, gérante d'un kiosque de crédit téléphonique au marché Mokolo à Yaoundé. Chaque soir, sa routine est immuable : elle ouvre son cahier, note le solde initial, puis pour chaque client, elle vérifie si le solde suffit, calcule la monnaie, met à jour le total des ventes et incrémente son compteur de clients. Elle ne le sait pas encore, mais elle exécute un **algorithme** : une suite d'actions précises, ordonnées, avec des choix (si solde suffisant) et des répétitions (pour chaque client). Comment transcrire cette logique humaine en un langage qu'une machine peut exécuter sans ambiguïté ? C'est tout l'objet de cette première section : poser les briques élémentaires — variables, constantes, instructions — pour construire des algorithmes **séquentiels**, c'est-à-dire sans branchement ni boucle, juste une exécution ligne par ligne, du début à la fin.

1.1 L'algorithme : une recette rigoureuse pour résoudre un problème

DÉFINITION Algorithme

Un algorithme est une suite finie et ordonnée d'instructions non ambiguës, permettant de résoudre un problème ou d'accomplir une tâche, en partant de données d'entrée pour produire un résultat de sortie.

À RETENIR Caractéristiques fondamentales

Tout algorithme respecte cinq propriétés essentielles :

1. **Finitude** : il se termine après un nombre fini d'étapes (pas de boucle infinie).
2. **Précision (non-ambiguïté)** : chaque instruction a un sens unique, compréhensible sans interprétation.
3. **Ordre** : les instructions s'exécutent les unes après les autres (séquentiellement) sauf si une structure de contrôle en décide autrement.
4. **Entrées** : zéro, une ou plusieurs valeurs initiales (données du problème).
5. **Sorties** : au moins un résultat produit (affiché, stocké, transmis).

LESAVAIS-TU ? Al-Khwarizmi, le père de l'algorithmique

Le mot « algorithme » dérive du nom du mathématicien perse Muhammad ibn Musa al-Khwarizmi (vers 780–850), qui travailla à la Maison de la Sagesse de Bagdad. Son ouvrage *Al-Kitab al-mukhtasar fi hisab al-jabr wal-muqabala* (« Le livre abrégé sur le calcul par la

restauration et la comparaison ») a donné le mot « algèbre », et son nom, latinisé en Algorismi, a désigné l'art de calculer par des règles systématiques. Aujourd'hui, chaque fois que tu écris un `Pour i de 1 à 10 Faire, tu honores une tradition vieille de douze siècles !`

1.2 Les variables : des boîtes étiquetées en mémoire

DÉFINITION Variable

Une variable est une zone de la mémoire vive (RAM) identifiée par un nom (identificateur), capable de contenir une seule valeur à la fois d'un type donné. Cette valeur peut changer au cours de l'exécution de l'algorithme.

EXEMPLE Analogie de la boîte étiquetée

Imagine une étagère dans le bureau d'Aminatou. Chaque tiroir porte une étiquette : `solde`, `nbClients`, `montantVente`. À l'intérieur, il y a un papier avec un nombre écrit au crayon à papier : on peut l'effacer et le remplacer. Le nom sur l'étiquette = l'identificateur ; le type = « nombre entier » ou « nombre à virgule » ; la valeur = le chiffre écrit sur le papier.



Avant instruction : `solde ← 15000`

Après instruction

Représentation mémoire : une variable est une case nommée, typée, dont le contenu évolue par affectation.

1.3 Les types de base : choisir le bon conteneur

PROPRIÉTÉ Types de données élémentaires en algorithmique

En algorithmique (et dans la plupart des langages), on distingue cinq types de base pour les variables simples :

Type	Domaine de valeurs / Usage	Exemple littéral
ENTIER	Nombres relatifs sans virgule (... , - 2, - 1, 0, 1, 2, ...)	42, -7, 0
REEL	Nombres à virgule (notation décimale ou scientifique)	3.14, -0.001, 2.5e3
CARACTERE	Un seul symbole (lettre, chiffre, signe)	'A', '7', '@'
CHAINE	Suite de caractères (texte)	"Yaoundé", "TI-2025"
BOOLEEN	Valeur logique : VRAI ou FAUX	VRAI, FAUX

MÉTHODE Choisir le type d'une variable à partir de l'énoncé

1. Lis l'énoncé et repère chaque donnée manipulée (entrée, résultat intermédiaire, sortie).
2. Demande-toi : « Cette donnée est-elle un nombre entier ? Un prix avec centimes ? Un simple oui/non ? Un mot ? »
3. Applique la règle :
 - Compteur, quantité discrète → ENTIER
 - Prix, mesure, moyenne, taux → REEL
 - Choix binaire, test, drapeau → BOOLEEN
 - Lettre unique, touche clavier → CARACTERE
 - Nom, adresse, message, matricule → CHAINE
4. Nomme la variable de façon explicite (ex. nbNotes, moyenne, estAdmis).

EXEMPLE Déclaration des variables pour un calcul de moyenne

Aminatou veut calculer la moyenne des notes d'une classe. Elle identifie :

- sommeNotes : accumulateur, peut contenir des décimales → REEL
- nbNotes : compteur d'élèves, nombre entier → ENTIER
- note : note courante lue au clavier, souvent sur 20 avec demi-points → REEL
- TVA : taux fixe 19,25% pour une facture éventuelle → Constante REEL

1.4 Les constantes : des valeurs figées pour la clarté et la maintenance

DÉFINITION Constante

Une constante est un identificateur associé à une valeur fixe, définie une seule fois au début de l'algorithme et qui ne peut jamais être modifiée pendant l'exécution. Elle améliore la lisibilité et évite les « nombres magiques » dispersés dans le code.

PROPRIÉTÉ Syntaxe de déclaration des constantes

```
CONSTANTES  
  NOM_CONSTANTE = valeur ;  
  AUTRE_NOM = autre_valeur ;  
FIN_CONSTANTES
```

- Par convention, les noms de constantes s'écrivent en MAJUSCULES (souvent avec underscores _).
- La valeur doit être un littéral connu à l'écriture (pas une variable, pas un calcul).

ATTENTION Piège : confondre variable et constante

- Erreur : Écrire `TVA = 19.25` dans la section `VARIABLES` puis modifier `TVA` plus bas. Le compilateur/interpréteur ne l'interdira pas forcément, mais c'est une faute de conception.
- Bonne pratique : Tout paramètre métier susceptible de changer (taux de TVA, seuil de découvert, nombre max d'élèves) doit être une constante. Le jour où le taux passe à 18%, tu changes une seule ligne.

1.5 Les instructions de base : lire, écrire, affecter

DÉFINITION Instruction

Une instruction est une action élémentaire, exécutable par la machine, qui modifie l'état de la mémoire (affectation), communique avec l'extérieur (lecture/écriture) ou contrôle le flux (structures alternatives/itératives — vues plus loin). Un algorithme séquentiel n'est qu'une liste d'instructions exécutées dans l'ordre écrit, de la première à la dernière.

PROPRIÉTÉ Trois instructions fondamentales du séquentiel

1. **Affectation** : `variable ← expression`
Calcule la valeur de `expression` (constante, variable, calcul) et la stocke dans `variable`. L'ancien contenu est écrasé.
2. **Lecture (Entrée)** : `Lire(variable)` ou `Saisir(variable)`
Attend que l'utilisateur tape une valeur au clavier (ou lire un fichier) et la stocke dans `variable` (doit être compatible avec le type).
3. **Écriture (Sortie)** : `Ecrire(expression)` ou `Afficher(expression)`
Affiche la valeur de `expression` à l'écran (console). Peut enchaîner plusieurs expressions séparées par des virgules.

EXEMPLE Séquence complète : calcul du prix TTC

```
ALGORITHME CalculTTC
VARIABLES
    prixHT, prixTTC : REEL
    quantite : ENTIER
CONSTANTES
    TAUX_TVA = 0.1925 // 19,25 %
DEBUT
    Ecrire("Entrez le prix HT unitaire : ")
    Lire(prixHT)
    Ecrire("Entrez la quantité : ")
    Lire(quantite)
    prixTTC ← prixHT * (1 + TAUX_TVA) * quantite
    Ecrire("Total TTC = ", prixTTC, " FCFA")
FIN
```

1.6 Règles de nommage des identificateurs (variables et constantes)**PROPRIÉTÉ** Règles syntaxiques pour les noms (identificateurs)

1. **Caractères autorisés** : lettres (A-Z, a-z, accentuées déconseillées), chiffres (0-9), underscore _.
2. **Premier caractère** : obligatoirement une lettre (pas de chiffre, pas de _ en tête).
3. **Sensible à la casse** : `solde`, `Solde` et `SOLDE` sont trois identificateurs distincts.
4. **Mots réservés interdits** : `SI`, `SINON`, `POUR`, `TANTQUE`, `ENTIER`, `REEL`, `DEBUT`, `FIN`, etc.
5. **Pas d'espace** : `prix unitaire` est illégal → utiliser `prixUnitaire` (camelCase) ou

```
prix_unitaire(snake_case).
```

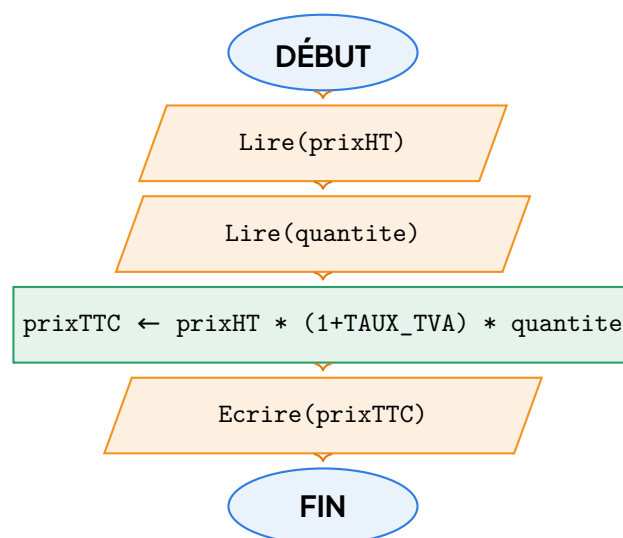
À RETENIR Conventions de style recommandées (bonnes pratiques professionnelles)

- Variables / fonctions : camelCase (ex. nbClients, calculerMoyenne).
- Constantes : MAJUSCULES_SNAKE_CASE (ex. TAUX_TVA, MAX_ELEVES).
- Noms significatifs : x, tmp, a1 sont à proscrire hors boucles très courtes. Préfère sommeNotes, estValide.
- Booléens : préfixe est, a, peut (ex. estMajeur, aSoldeSuffisant).

1.7 L'algorithme séquentiel : structure globale et organigramme

DÉFINITION Algorithme séquentiel

Un algorithme séquentiel est un algorithme dont les instructions s'exécutent strictement dans l'ordre d'écriture, de la première à la dernière, sans aucune structure de choix (SI) ni de répétition (POUR, TANTQUE). C'est la forme la plus simple : une « recette » linéaire.



Organigramme (algorithme) d'un algorithme séquentiel : flux unique, vertical, sans embranchement.

MÉTHODE Écrire un algorithme séquentiel complet — démarche en 5 étapes

1. Analyser le problème : identifier les entrées (données fournies), les sorties (résultats attendus), les formules ou règles de gestion.
2. Choisir les variables et constantes : nom, type, rôle (entrée, intermédiaire, sortie). Les déclarer dans les sections VARIABLES et CONSTANTES.
3. Écrire le corps (DEBUT ... FIN) dans l'ordre logique :
 - Instructions de lecture pour chaque entrée.
 - Instructions de calcul/affectation pour chaque résultat intermédiaire.
 - Instructions d'écriture pour chaque sortie.
4. Tester mentalement avec un jeu de données simple (ex. prixHT=1000, quantite=2 → prixTTC=2385).
5. Commenter les lignes non triviales (formule métier, astuce).

EXERCICE RÉSOLU 1 Calcul de la moyenne de trois notes

Énoncé : Écrire un algorithme qui demande trois notes sur 20, calcule leur moyenne et l'affiche avec un message.

Solution détaillée :

1. Analyse : Entrées = 3 notes (réelles). Sortie = moyenne (réelle). Formule = $(n1+n2+n3)/3$.
2. Déclarations : note1, note2, note3, moyenne : REEL. Pas de constante nécessaire.
3. Algorithme :

```
ALGORITHME MoyenneTroisNotes
VARIABLES
    note1, note2, note3, moyenne : REEL
DEBUT
    Ecrire("Entrez la note 1 : ")
    Lire(note1)
    Ecrire("Entrez la note 2 : ")
    Lire(note2)
    Ecrire("Entrez la note 3 : ")
    Lire(note3)
    moyenne <- (note1 + note2 + note3) / 3
    Ecrire("La moyenne est : ", moyenne)
FIN
```

4. Test : notes 12, 14, 16 → somme 42 → moyenne 14. Correct.

ATTENTION Erreurs fréquentes sur les algorithmes séquentiels

- **Variable non déclarée** : utiliser `total` sans l'avoir mis dans `VARIABLES`. → Erreur de compilation / exécution.
- **Variable non initialisée avant lecture** : faire `somme <- somme + note` sans avoir mis `somme <- 0` avant. → Résultat aléatoire (valeur « poubelle » en mémoire).
- **Inversion lecture/écriture** : écrire `Ecrire("Note : "); Lire(note)` au lieu de `Ecrire("Note : "); Lire(note)` (l'ordre n'a pas d'impact ici, mais `Lire` avant `Ecrire` le prompt déroute l'utilisateur).
- **Type incompatible** : lire un entier dans une variable `REEL` (souvent toléré) mais l'inverse (lire 12.5 dans un `ENTIER`) provoque une erreur ou une troncature silencieuse selon le langage.
- **Oublier le symbole d'affectation** : écrire `moyenne = (n1+n2+n3)/3` (test d'égalité) au lieu de `moyenne <- (n1+n2+n3)/3`. En algorithmique, `<-` (ou `:=`) est l'unique opérateur d'affectation.

EXPÉRIENCE TP guidé : Mon premier algorithme séquentiel — Calcul de facture

Objectif : Écrire, tester sur papier et corriger un algorithme complet qui calcule le montant TTC d'une facture avec plusieurs articles.

Matériel : Papier, stylo, calculatrice (ou environnement d'exécution algorithmique type Algobox, Python en mode script).

Énoncé : Un client achète 3 articles. Pour chaque article, on connaît le prix HT unitaire et la quantité. La TVA est de 19,25%. L'algorithme doit afficher le total HT, le montant de la TVA et le total TTC.

Manipulation pas à pas :

1. **Déclarer les variables** : `prixHT1, qte1, prixHT2, qte2, prixHT3, qte3` : REEL (ou ENTIER pour quantités), `totalHT, montantTVA, totalTTC` : REEL. **Constante** `TVA = 0.1925`.
2. Lire les 6 valeurs (3 prix, 3 quantités) avec des prompts clairs.
3. Calculer : `totalHT <- prixHT1*qte1 + prixHT2*qte2 + prixHT3*qte3`.
4. Calculer : `montantTVA <- totalHT * TVA`.
5. Calculer : `totalTTC <- totalHT + montantTVA` (ou `totalHT * (1+TVA)`).
6. Afficher les trois résultats avec étiquettes et unité « FCFA ».

Observations attendues :

- Si tu saisis des quantités entières, le total HT peut être entier, mais `montantTVA` devient réel.

- Vérifie l'arrondi : en gestion, on arrondit souvent à l'entier supérieur (Franc CFA). Cela nécessite une fonction `ArrondiSup` (vue plus tard).

Interprétation : Tu viens d'écrire ton premier algorithme de gestion commerciale réel, structuré, typé et commenté. C'est la base de tout système de facturation, du kiosque de Mokolo aux ERP des grandes entreprises.

EXERCICE 1 Entraînement : Déclarations et séquentiel

1. Pour chacun des énoncés ci-dessous, donne le type le plus approprié (ENTIER, REEL, CHAÎNE, BOOLEEN, CARACTERE) et un nom de variable significatif :
 - L'âge d'un élève.
 - Le prix du carburant à la pompe (ex. 745 FCFA/L).
 - Le matricule d'un étudiant (ex. « TI2025-0142 »).
 - La réponse à la question « Êtes-vous boursier ? ».
 - La touche pressée au clavier pour valider.
2. Écris l'algorithme séquentiel complet qui :
 - Demande le rayon d'un cercle.
 - Calcule son périmètre ($2 \times \pi \times R$) et sa surface ($\pi \times R^2$).
 - Affiche les deux résultats.
 - Utilise une constante `PI = 3.14159`.
3. Repère l'erreur dans l'algorithme suivant et corrige-la :

```
ALGORITHME Erreur
VARIABLES
  a, b : ENTIER
DEBUT
  Ecrire("Donnez a : ")
  Lire(a)
  Ecrire("Donnez b : ")
  Lire(b)
  a + b <- somme // Ligne suspecte
  Ecrire("Somme = ", somme)
FIN
```

CORRIGÉ Exercice 1

1. Types et noms suggérés :

- Âge → age : ENTIER
- Prix carburant → prixCarburant : REEL (même si affiché entier, calculs futurs)
- Matricule → matricule : CHAÎNE (contient lettres, chiffres, tiret)
- Boursier → estBoursier : BOOLEEN
- Touche → touche : CARACTERE

2. Algorithme cercle :

```

ALGORITHME PerimetreSurfaceCercle
VARIABLES
    rayon, perimetre, surface : REEL
CONSTANTES
    PI = 3.14159
DEBUT
    Ecrire("Entrez le rayon du cercle : ")
    Lire(rayon)
    perimetre ← 2 * PI * rayon
    surface ← PI * rayon * rayon
    Ecrire("Périmètre = ", perimetre)
    Ecrire("Surface = ", surface)
FIN
  
```

3. Erreur : `a + b ← somme` tente d'affecter une valeur à une expression (côté gauche). L'affectation ne s'écrit que `variable ← expression`. Correction : `somme ← a + b` (il faut déclarer `somme : ENTIER` au préalable).

2 INSTRUCTIONS DE BASE : LECTURE, ÉCRITURE, AFFECTATION

Dans la section précédente, nous avons posé les fondations : nous savons désormais ce qu'est une **variable** (une boîte étiquetée en mémoire), une **constante** (une boîte scellée) et un **algorithme** (une recette précise). Il nous reste à apprendre comment faire vivre ces variables : comment y mettre une valeur, comment en sortir une valeur pour l'afficher, et comment transformer une valeur en une autre. C'est le rôle des trois instructions fondamentales que tout algorithme séquentiel manipule en permanence : **Lire**, **Écrire** et l'**affectation** ($\leftarrow$).

2.1 L'instruction de lecture : Lire

DÉFINITION Instruction Lire — Entrée de données

L'instruction `Lire` permet de transférer une valeur saisie par l'utilisateur (au clavier) dans une variable déjà déclarée. Elle réalise le lien entre le monde extérieur (l'humain) et la mé-

moire de l'ordinateur.

PROPRIÉTÉ Syntaxe et effet de Lire

- **Syntaxe** : Lire(nom_variable)
- **Effet** : Le programme marque une pause, attend que l'utilisateur tape une valeur au clavier et valide par Entrée, puis stocke cette valeur dans la variable indiquée. L'ancienne valeur de la variable (si elle en avait une) est écrasée.
- **Compatibilité** : La valeur saisie doit être du même **type** que la variable (entier, réel, chaîne de caractères, booléen). Si l'utilisateur tape abc dans une variable de type Entier, le comportement est imprévisible (erreur d'exécution ou valeur par défaut selon l'interpréteur).

EXEMPLE Lecture simple — Contexte : caisse d'un magasin à Douala

On veut saisir le prix unitaire d'un article et la quantité achetée.

```
Algorithme Saisie_Achat
Variables
    prix_unitaire : Réel
    quantite      : Entier
Début
    Écrire("Entrez le prix unitaire (FCFA) : ")
    Lire(prix_unitaire)

    Écrire("Entrez la quantité : ")
    Lire(quantite)
Fin
```

Remarque : Le message affiché juste avant Lire (appelé **invite de saisie** ou **prompt**) est indispensable pour guider l'utilisateur. Sans lui, l'écran reste muet et l'utilisateur ne sait pas ce qu'on attend de lui.

2.2 L'instruction d'écriture : Écrire

DÉFINITION Instruction Écrire — Sortie de résultats

L'instruction Écrire permet d'afficher à l'écran (sortie standard) des messages textuels, des valeurs de variables, ou des résultats d'expressions. C'est le seul moyen pour l'algorithme de communiquer ses résultats vers l'extérieur.

PROPRIÉTÉ Syntaxe et variantes de Écrire

- Syntaxe générale : `Écrire( liste d'arguments séparés par des virgules )`
- Arguments acceptés :
 - Chaînes de caractères littérales entre guillemets : `"Montant TTC : "`
 - Noms de variables : `montant_ttc`
 - Expressions arithmétiques ou logiques : `prix * quantite, age >= 18`
- Séparateur : La virgule , dans la liste des arguments provoque généralement un espace à l'affichage (comportement standard en pseudo-code).
- Retour à la ligne : Par défaut, `Écrire` termine par un retour à la ligne. (Certains dialectes proposent `ÉcrireSansRetour` ou `Afficher sans saut de ligne`).

EXEMPLE Écriture composée — Affichage d'une facture simplifiée

Algorithme Afficher_Facture

Variables

article : Chaîne
 prix_ht : Réel
 tva : Réel
 prix_ttc : Réel

Début

article <- "Cahier 200 pages"
 prix_ht <- 500
 tva <- 0.1925
 prix_ttc <- prix_ht * (1 + tva)

Écrire("=== TICKET DE CAISSE ===")

Écrire("Article : ", article)

Écrire("Prix HT : ", prix_ht, " FCFA")

Écrire("TVA (19,25 \%) : ", prix_ht * tva, " FCFA")

Écrire("Prix TTC : ", prix_ttc, " FCFA")

Écrire("=====")

Fin

Sortie attendue :

```

=== TICKET DE CAISSE ===
Article : Cahier 200 pages
Prix HT : 500 FCFA
TVA (19,25%) : 96.25 FCFA
Prix TTC : 596.25 FCFA
=====
  
```

2.3 L'instruction d'affectation : $\leftarrow$ — Cœur de la transformation

C'est l'instruction la plus fréquente, la plus puissante, et celle qui cause le plus de malentendus chez les débutants.

DÉFINITION Instruction d'affectation — Sens d'exécution

L'affectation, notée $\leftarrow$ (flèche vers la gauche), évalue l'expression située à droite de la flèche, puis stocke le résultat obtenu dans la variable située à gauche.

Variable $\leftarrow$ Expression

On lit : « Variable prend la valeur de Expression » ou « Variable reçoit Expression ».

Source (Droite)

Calcul (Expression)

Destination (Gauche)

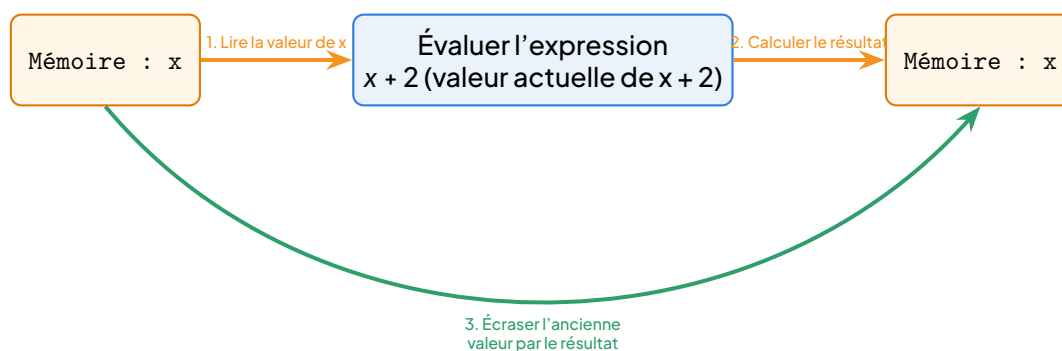


Schéma d'exécution de l'affectation $x \leftarrow x + 2$:
la flèche $\leftarrow$ impose un sens unique Droite $\rightarrow$ Gauche.

PROPRIÉTÉ Règles strictes de l'affectation

1. Ordre d'évaluation : TOUJOURS la partie droite (expression) est calculée avant toute modification de la variable de gauche. C'est ce qui permet $x \leftarrow x + 1$.
2. Compatibilité de type : Le type du résultat de l'expression doit être compatible avec le type de la variable cible (ex : on ne peut pas affecter une chaîne dans un Entier sans conversion explicite).
3. Unicité de la cible : La partie gauche doit être une et une seule variable. Les écritures $x + y \leftarrow z$ ou $5 \leftarrow x$ sont interdites (erreur de syntaxe).
4. Opérateur distinct : $\leftarrow$ n'est pas le signe égal mathématique $=$. Voir encadré Attention ci-dessous.

ATTENTION Piège majeur : Confusion = (maths) vs $\leftarrow$ (algo)

En mathématiques, $x = x + 1$ est une équation impossible (aucun nombre n'est égal à lui-même plus 1). En algorithmique, $x \leftarrow x + 1$ est une instruction d'incrémentement parfaitement valide : « Prends la valeur actuelle de x , ajoute 1, remets le résultat dans x ».

- Si x vaut 5 avant l'instruction, il vaut 6 après.
- JAMAIS n'écrivez $x = x + 1$ dans un algorithme (sauf dans une condition `Si`). Utilisez $\leftarrow$ (ou `:=` selon les notations, mais $\leftarrow$ est la norme MINESEC).

2.4 Initialisation, incrémentement et décrémentation

Ces trois opérations sont des cas particuliers d'affectation qu'il faut savoir identifier et écrire correctement.

DÉFINITION Initialisation

Donner une valeur de départ à une variable avant de l'utiliser dans un calcul ou une structure itérative. C'est une affectation simple, souvent faite au début de l'algorithme ou juste avant une boucle.

```
somme  $\leftarrow$  0    // Initialisation d'un accumulateur
compteur  $\leftarrow$  -1 // Initialisation d'un compteur
produit  $\leftarrow$  1   // Initialisation pour un produit (élément neutre)
```

DÉFINITION Incrémentement

Augmenter la valeur d'une variable d'un pas constant (généralement 1).

```
i  $\leftarrow$  i + 1    // Incrémentement de 1 (standard)
pas  $\leftarrow$  pas + 2 // Incrémentement de 2
```

DÉFINITION Décrémentation

Diminuer la valeur d'une variable d'un pas constant (généralement 1).

```
i  $\leftarrow$  i - 1    // Décrémentation de 1
stock  $\leftarrow$  stock - qte_vendue // Décrémentation d'une quantité variable
```

À RETENIR Repérage dans un algorithme

- Initialisation : Affectation avant une boucle ou un traitement (ex : `somme ← 0`).
- Incrémentation/Décrémentation : Affectation à l'intérieur d'une boucle où la variable cible figure aussi dans l'expression (ex : `i ← i + 1`).
- Condition d'arrêt : Expression logique qui devient fausse pour sortir de la boucle (ex : `i > 10`).

2.5 Évaluation d'expressions et types compatibles

Avant d'affecter, l'expression de droite est évaluée selon les règles de priorité opératoires standards (parenthèses, puissances, $\times/\div$, $+/ -$, comparaisons, `et/ou/non`).

EXEMPLE Expressions complexes et priorités

```
Variables
a, b, c, res : Entier
Début
  a ← -10
  b ← -3
  c ← -2

  // Priorités : * et / avant + et -
  res ← -a + b * c      // res ← -10 + (3 * 2) = 16
  Écrire("res 1 : ", res)

  res ← (a + b) * c     // res ← -(10 + 3) * 2 = 26
  Écrire("res 2 : ", res)

  // Division entière vs réelle (selon types)
  // Si res est Entier : 10 / 3 = 3 (partie entière)
  // Si res est Réel : 10 / 3 = 3.333...
  res ← a / b
  Écrire("res 3 : ", res)

  // Modulo (reste de la division entière)
  res ← a % b           // 10 modulo 3 = 1
  Écrire("res 4 : ", res)
Fin
```

2.6 La trace d'exécution (tableau des valeurs) : « Jouer l'ordinateur »

Pour vérifier un algorithme, trouver un bug ou comprendre un code existant, la technique universelle est la **trace d'exécution** (ou **tableau des valeurs**). On simule pas à pas l'exécution en notant l'état de chaque variable après chaque instruction.

MÉTHODE Construire une table de trace d'exécution

1. Colonnes : Une colonne par variable manipulée + une colonne `Instruction` exécutée (ou numéro de ligne) + éventuellement une colonne `Sortie` (Écrire).
2. Ligne 0 (État initial) : Noter les valeurs initiales (souvent « ? » si non initialisées, ou la valeur après déclaration/affectation initiale).
3. Parcours séquentiel : Pour chaque instruction de l'algorithme (dans l'ordre) :
 - Si `Lire(v)` : noter la valeur simulée saisie dans la colonne `v`.
 - Si `v ← expr` : calculer `expr` avec les valeurs actuelles des variables, noter le résultat dans la colonne `v`. Les autres colonnes restent inchangées (recopier la valeur d'au-dessus ou laisser vide).
 - Si `Écrire(...)` : noter ce qui s'affiche dans la colonne `Sortie`.
4. Ligne finale : État de la mémoire à la fin du programme.

Étape / Instruction	a	b	temp	Sortie (Écrire)
Début / Init (a=5, b=3)	5	3	?	–
temp ← a	5	3	5	–
a ← b	3	3	5	–
b ← temp	3	5	5	–
Écrire("a=", a, " b=", b)	3	5	5	a=3 b=5
Fin	3	5	5	–

Trace complète de l'échange de deux variables $a = 5, b = 3$ via une variable intermédiaire `temp`. Observer la conservation des valeurs.

2.7 Structure canonique d'un algorithme séquentiel complet

Tout algorithme présenté à l'examen ou en devoir doit respecter cette structure rigide (norme MINESEC / pseudo-code standard) :

PROPRIÉTÉ Squelette obligatoire d'un algorithme

```

Algorithme Nom_Significatif_PascalCase
// Optionnel : commentaire d'en-tête (but, auteur, date)
Variables

```

```
// Déclarations : nom_var : Type [<- valeur_initiale]
// Indentation : 4 espaces (ou 1 tab)
Début
  // Corps de l'algorithme : instructions séquentielles
  // Une instruction par ligne idéalement
  // Commentaires avec //
Fin
```

- **Algorithme** : mot-clé de début, suivi d'un nom sans espace (ex : CalculSurfaceChamp).
- **Variables** : section de déclaration. Chaque ligne : `identificateur : Type`. L'affectation initiale `<-` est autorisée mais l'instruction `Lire` ne l'est pas ici.
- **Début / Fin** : bornent le bloc d'exécution. `Fin` est le point d'arrêt.

2.8 Exemples d'algorithmes séquentiels complets

EXEMPLE Exemple 1 : Surface d'un champ rectangulaire (agriculture, région du Nord)

Un paysan veut connaître la surface de son champ en hectares pour estimer le rendement.
1 hectare = 10 000 m².

```
Algorithme Surface_Champ_Hectares
Variables
  longueur, largeur : Réel    // en mètres
  surface_m2, surface_ha : Réel
Début
  Écrire(" --- Calcul Surface Champ ---")
  Écrire("Longueur (m) : ")
  Lire(longueur)
  Écrire("Largeur (m) : ")
  Lire(largeur)

  surface_m2 <- longueur * largeur
  surface_ha <- surface_m2 / 10000

  Écrire("Surface : ", surface_m2, " m²")
  Écrire("Soit : ", surface_ha, " hectares")
Fin
```

EXEMPLE Exemple 2 : Conversion FCFA ↔ Euro (taux fixe BEAC)

Taux officiel arrondi : 1 Euro = 655,957 FCFA (souvent arrondi à 656 FCFA dans les exercices scolaires).

```
Algorithme Conversion_Monnaie
```

```
Constante
    TAUX_EURO_FCFA <- 655.957 // Réel
Variables
    montant_fcfa, montant_euro : Réel
    choix : Entier
Début
    Écrire("1. FCFA -> Euro")
    Écrire("2. Euro -> FCFA")
    Écrire("Votre choix (1 ou 2) : ")
    Lire(choix)

    Si choix = 1 Alors
        Écrire("Montant en FCFA : ")
        Lire(montant_fcfa)
        montant_euro <- montant_fcfa / TAUX_EURO_FCFA
        Écrire(montant_fcfa, " FCFA = ", montant_euro, " EUR")
    Sinon
        Si choix = 2 Alors
            Écrire("Montant en Euro : ")
            Lire(montant_euro)
            montant_fcfa <- montant_euro * TAUX_EURO_FCFA
            Écrire(montant_euro, " EUR = ", montant_fcfa, " FCFA")
        Sinon
            Écrire("Choix invalide.")
        Fin Si
    Fin Si
Fin
```

Note : Nous utilisons déjà `Si` ici par anticipation ; la structure alternative sera détaillée en Section 3. L'essentiel ici est la manipulation des variables et des affectations.

EXERCICE RÉSOLU 2 Calcul TTC avec TVA camerounaise (19,25 %)

Énoncé : Écrire un algorithme qui lit le prix HT d'un article, calcule le montant de la TVA (19,25 %) et le prix TTC, puis affiche les trois montants formatés. Données : Prix HT = 50 000 FCFA. TVA = 19,25 %. Analyse :

- **Entrée :** `prix_ht` (Réel).
- **Constante :** `TAUX_TVA = 0,1925`.
- **Calculs :** `mt_tva ← prix_ht * TAUX_TVA`; `prix_ttc ← prix_ht + mt_tva` (ou `prix_ht * (1 + TAUX_TVA)`).
- **Sorties :** `prix_ht`, `mt_tva`, `prix_ttc`.

SOLUTION

Algorithme :

```
Algorithme Calcul_TTC_Cameroun
```

```

Constante
    TAUX_TVA <- 0.1925
Variables
    prix_ht, mt_tva, prix_ttc : Réel
Début
    Écrire("=== CALCUL PRIX TTC (TVA 19,25\%) ===")
    Écrire("Entrez le prix HT (FCFA) : ")
    Lire(prix_ht)

    mt_tva <- prix_ht * TAUX_TVA
    prix_ttc <- prix_ht + mt_tva

    Écrire("-----")
    Écrire("Prix HT : ", prix_ht, " FCFA")
    Écrire("TVA (19,25\%) : ", mt_tva, " FCFA")
    Écrire("Prix TTC : ", prix_ttc, " FCFA")
    Écrire("-----")
Fin

```

Trace d'exécution pour prix_ht = 50000 :

Instruction	prix_ht	mt_tva	prix_ttc
Lire(prix_ht) [saisie 50000]	50000	?	?
mt_tva ← prix_ht * 0.1925	50000	9625	?
prix_ttc ← prix_ht + mt_tva	50000	9625	59625
Écrire(...)	50000	9625	59625

Sortie console :

```

=== CALCUL PRIX TTC (TVA 19,25%) ===
Entrez le prix HT (FCFA) : 50000
-----
Prix HT : 50000 FCFA
TVA (19,25%) : 9625 FCFA
Prix TTC : 59625 FCFA
-----

```

2.9 Cas classique : Échange de deux variables (Permutation)

C'est l'exercice « passage obligé » pour tester la maîtrise de l'affectation séquentielle et de la variable intermédiaire.

ATTENTION Erreur fatale : L'échange direct sans variable temporaire

```

// TENTATIVE INCORRECTE
a <- 5

```

```
b <- 3
a <- b // a devient 3. La valeur 5 est PERDUE à jamais.
b <- a // b devient 3 (valeur actuelle de a).
// Résultat : a=3, b=3. ÉCHEC.
```

Règle d'or : Pour échanger deux variables, il faut impérativement une troisième variable (souvent appelée `temp`, `tmp`, aux ou `intermediaire`) pour stocker l'une des valeurs le temps de copier l'autre.

EXEMPLE Algorithme complet : Permutation de deux entiers

```
Algorithme Permuter_Deux_Valeurs
Variables
  a, b, temp : Entier
Début
  Écrire("Valeur de a : ")
  Lire(a)
  Écrire("Valeur de b : ")
  Lire(b)

  Écrire("Avant échange : a = ", a, ", b = ", b)

  // La chorégraphie à 3 temps
  temp <- a // 1. Sauvegarde de a
  a <- b    // 2. a reçoit b (a est libre)
  b <- temp // 3. b reçoit l'ancien a (sauvé dans temp)

  Écrire("Après échange : a = ", a, ", b = ", b)
Fin
```

EXPÉRIENCE TP Guidé : Trace manuelle et implémentation C

Objectif : Maîtriser la trace d'exécution et transcrire en langage C. **Durée :** 30 min. **Matériel :** Papier/crayon + compilateur C (Code ::Blocks, Dev-C++, ou gcc en ligne de commande sous Linux/Termux).

Étape 1 — Trace papier (5 min). Pour l'algorithme `Permuter_Deux_Valeurs` ci-dessus, avec les entrées `a = 12` et `b = 7`, compléter le tableau de trace ci-dessous (ne pas regarder la correction avant d'avoir fini).

Instruction	a	b	temp
Lire(a) [12]			
Lire(b) [7]			
temp ← a			
a ← b			
b ← temp			

Étape 2 — Code C (15 min). Transcrire l'algorithme en C. Rappels : `scanf` pour lire, `printf` pour écrire, & obligatoire devant la variable dans `scanf`.

```
#include <stdio.h>

int main() {
    int a, b, temp;

    printf("Valeur de a : ");
    scanf("%d", &a);
    printf("Valeur de b : ");
    scanf("%d", &b);

    printf("Avant echange : a = %d, b = %d\n", a, b);

    temp = a;
    a = b;
    b = temp;

    printf("Après echange : a = %d, b = %d\n", a, b);
    return 0;
}
```

Étape 3 — Test et validation (10 min). Compiler (`gcc permut.c -o permut`), exécuter (`./permut`), tester avec 12 et 7, puis avec des négatifs (-5 et 20). Vérifier que la trace papier correspond à l'exécution réelle. Observation attendue : La variable `temp` est invisible à l'écran mais indispensable en mémoire. Sans elle, l'échange est impossible en séquentiel.

À RETENIR Bilan de la section — Check-list compétences

- Lire(var) : Pause programme → Saisie clavier → Stockage dans var. Toujours précédé d'un Écrire guide.
- Écrire(arg1, arg2, ...) : Affiche arguments séparés par des espaces, retour à la ligne final. Accepte texte, variables, expressions.
- Affectation (var ← expr) : Droite vers Gauche. Évaluer expr → Stocker dans var. Types compatibles obligatoires.

- Initialisation : Affectation avant utilisation (ex : $\text{somme} \leftarrow 0$).
- Incrémentation : $\text{var} \leftarrow \text{var} + \text{pas}$ (souvent +1).
- Décrémentation : $\text{var} \leftarrow \text{var} - \text{pas}$ (souvent -1).
- Trace d'exécution : Tableau colonnes = variables, lignes = instructions. Outil de débogage #1.
- Structure Algo : Algorithme Nom / Variables / Début / Corps / Fin.
- Échange : Nécessite temp . $\text{temp} \leftarrow a$; $a \leftarrow b$; $b \leftarrow \text{temp}$.
- Pièges : $=$ vs $\leftarrow$; oubli temp ; oubli invite Écrire avant Lire ; division entière inattendue.

EXERCICE 2 Entraînement — Niveau 1 : Prise en main

1. Écrire un algorithme qui lit trois entiers a , b , c et affiche leur somme et leur moyenne (réelle).
2. Écrire un algorithme qui lit le rayon R d'un cercle (réel) et affiche son périmètre ($2\pi R$) et sa surface (πR^2). Utiliser une constante $PI \leftarrow 3.14159$.
3. Soit les affectations successives : $x \leftarrow 10$; $y \leftarrow 5$; $x \leftarrow x + y$; $y \leftarrow x - y$; $x \leftarrow x - y$. Faire la trace complète et donner les valeurs finales de x et y . (Astuce : c'est un échange sans variable temporaire, fonctionne-t-il ?).

EXERCICE 3 Entraînement — Niveau 2 : Application contextuelle

1. Gestion de stock (Boutique « Au Bon Prix », Yaoundé). Un article a un stock initial S . On vend Q unités. Écrire l'algorithme qui lit S et Q , calcule le nouveau stock N , et affiche « Rupture de stock » si $N \leq 0$ (utiliser Si vu en anticipation), sinon affiche N .
2. Calcul de note moyenne (Lycée). Un élève a 3 notes : N_1 (coeff 2), N_2 (coeff 3), N_3 (coeff 5). Écrire l'algorithme qui lit les 3 notes, calcule la moyenne pondérée et l'affiche avec 2 décimales.

CORRIGÉ Exercice Niveau 1

1. Somme et moyenne

Algorithme Somme_Moyenne_3
Variables
a, b, c : Entier

```

    somme : Entier
    moyenne : Réel
Début
    Écrire("Entrez a : "); Lire(a)
    Écrire("Entrez b : "); Lire(b)
    Écrire("Entrez c : "); Lire(c)
    somme ← a + b + c
    moyenne ← somme / 3.0 // Division réelle forcée par 3.0
    Écrire("Somme = ", somme)
    Écrire("Moyenne = ", moyenne)
Fin

```

2. Cercle

```

Algorithme Perimetre_Surface_Cercle
Constante
    PI ← 3.14159
Variables
    R, perimetre, surface : Réel
Début
    Écrire("Rayon R : "); Lire(R)
    perimetre ← 2 * PI * R
    surface ← PI * R * R
    Écrire("Périmètre = ", perimetre)
    Écrire("Surface = ", surface)
Fin

```

3. Échange sans variable temporaire (Trace)

Instruction	x	y
Init	10	5
$x \leftarrow x + y$	15	5
$y \leftarrow x - y$	15	10
$x \leftarrow x - y$	5	10

Résultat : $x=5$, $y=10$. Cela fonctionne! C'est l'algorithme d'échange par addition/soustraction (ou XOR en binaire). Attention toutefois : risque de débordement (overflow) si $x+y$ dépasse la capacité du type (ex : entiers 16 bits). La méthode avec `temp` reste la plus sûre et la plus lisible.

LESAVAIS-TU ? Le saviez-vous ? — L'affectation en matériel

Au niveau processeur (CPU), l'affectation $x \leftarrow x + 1$ se traduit par une séquence de micro-opérations : FETCH (aller chercher x en RAM/cache dans un registre), ALU (Addition dans l'Unité Arithmétique et Logique), STORE (réécrire le registre vers l'adresse mémoire de x). C'est pour cela que $x \leftarrow x + 1$ n'est pas atomique en multiprocesseur : deux cœurs peuvent lire la même valeur, l'incrémenter, et l'écrire, perdant une incrémentation (pro-

blème de race condition). En Terminale Réseau/Système, nous verrons les verrous (`mutex`) et les instructions atomiques (`LOCK INC` en assembleur x86) pour résoudre cela.

3 STRUCTURES DE CONTRÔLE : PANORAMA ET STRUCTURES ALTERNATIVES

Nous savons désormais lire une valeur (`Lire`), afficher un résultat (`Ecrire`) et stocker une information dans une variable grâce à l'affectation (`←`). Avec ces seules instructions, nos algorithmes s'exécutent **linéairement**, instruction après instruction, du début à la fin. Or, la plupart des problèmes réels exigent de **choisir** entre plusieurs actions selon une situation (un élève a-t-il la moyenne ? un colis est-il trop lourd ?) ou de **répéter** une tâche un nombre connu ou inconnu de fois (saisir 30 notes, attendre une réponse correcte). C'est le rôle des **structures de contrôle** : elles rompent la séquentialité pour donner de l'intelligence à l'algorithme.

3.1 Qu'est-ce qu'une structure de contrôle ?

DÉFINITION Structure de contrôle

Une **structure de contrôle** est une construction algorithmique qui modifie l'ordre séquentiel d'exécution des instructions. Elle permet de diriger le flot du programme vers l'une ou l'autre séquence d'instructions (choix) ou de réexécuter plusieurs fois un même bloc (répétition).

Le programme officiel en identifie trois familles fondamentales :

PROPRIÉTÉ Les trois familles de structures de contrôle

1. Structure séquentielle (par défaut) : les instructions s'enchaînent les unes après les autres sans branchement. C'est ce que nous avons pratiqué jusqu'ici.
2. Structure alternative (choix conditionnel) : selon qu'une condition est vraie ou fausse, on exécute tel bloc ou tel autre bloc. Mots-clés : `Si`, `Alors`, `Sinon`, `FinSi`, `Selon`, `Cas`.
3. Structure itérative (boucle) : un bloc d'instructions est répété tant qu'une condition reste vraie, ou un nombre déterminé de fois. Mots-clés : `Pour`, `Faire`, `Tant que`, `Répéter`, `Jusqu'à`.

À RETENIR

Tout algorithme, aussi complexe soit-il, se construit en combinant ces trois structures de base (séquence, alternative, itération). C'est le théorème de structure (Böhm-Jacopini) : pas besoin de `Goto` pour programmer n'importe quel calcul.

3.2 La condition : cœur de la décision

Toute alternative et toute boucle reposent sur une **condition** : une expression qui s'évalue en une valeur booléenne (VRAI ou FAUX).

DÉFINITION Condition et opérateurs

Une **condition** est une expression logique dont la valeur est VRAI ou FAUX. Elle se construit avec :

- Opérateurs de comparaison (renvoient un booléen) : `=` (égal), `≠` (différent), `<` (strictement inférieur), `>` (strictement supérieur), `≤` (inférieur ou égal), `≥` (supérieur ou égal).
- Opérateurs logiques (combinent des booléens) : `ET` (conjonction, vrai si les deux opérandes sont vrais), `OU` (disjonction, vrai si au moins un opérande est vrai), `NON` (négation, inverse la valeur).

La priorité (de la plus forte à la plus faible) est : `NON`, opérateurs de comparaison, `ET`, `OU`. Utilisez des parenthèses pour lever toute ambiguïté.

EXEMPLE Exemples de conditions

Soient `age` (entier), `moyenne` (réel), `sexe` (caractère).

- `age ≥ 18` – Vrai si la personne est majeure.
- `moyenne ≥ 10 ET moyenne < 12` – Vrai pour une mention « Passable ».
- `NON (sexe = 'M' OU sexe = 'F')` – Vrai si le sexe saisi n'est ni 'M' ni 'F' (saisie invalide).

3.3 Structure alternative simple : « Si ... Alors ... FinSi »

Lorsque l'on ne doit exécuter un bloc d'instructions que si une condition est remplie, et ne rien faire sinon, on utilise la forme simple.

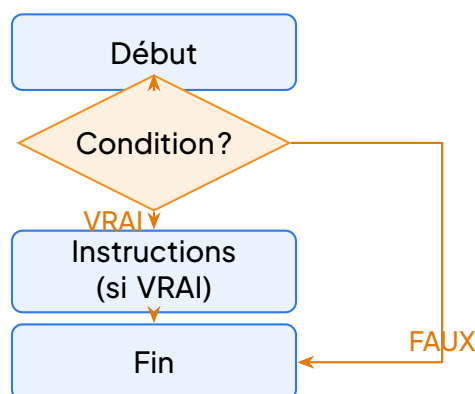
PROPRIÉTÉ Syntaxe : Alternative simple

```

Si <condition> Alors
    <instruction(s) si VRAI>
FinSi

```

Sémantique : On évalue <condition>. Si elle est **VRAI**, on exécute le bloc d'instructions ; si elle est **FAUX**, on saute directement après **FinSi**. Dans les deux cas, l'exécution continue normalement.



Organigramme de la structure alternative simple Si ... Alors ... FinSi. Le losange représente le test ; la branche de droite (FAUX) contourne le bloc d'instructions.

EXEMPLE Exemple : Autorisation d'accès à un cybercafé (Yaoundé)

Un cybercafé de la rue de l'Hôpital n'autorise l'accès qu'aux personnes majeures (≥ 18 ans).

```

Algorithme AccesCyber
Variable age : Entier
Début
    Ecrire("Entrez votre âge : ")
    Lire(age)
    Si age >= 18 Alors
        Ecrire("Accès autorisé. Bienvenue !")
    FinSi
    Ecrire("Fin du programme.")
Fin

```

Si l'utilisateur tape 16, le message « Accès autorisé » n'apparaît pas ; on passe directement à « Fin du programme. »

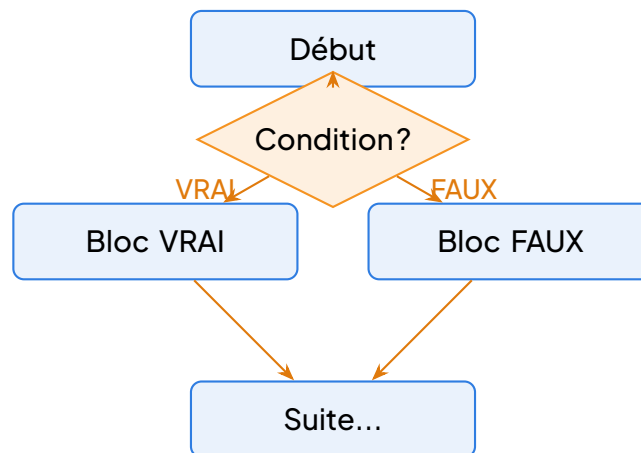
3.4 Structure alternative complète : « Si ... Alors ... Sinon ... FinSi »

Souvent, il faut traiter les deux cas : action A si vrai, action B si faux. C'est l'alternative complète.

PROPRIÉTÉ Syntaxe : Alternative complète

```
Si <condition> Alors
    <instructions si VRAI>
Sinon
    <instructions si FAUX>
FinSi
```

Sémantique : Exactement l'une des deux branches est exécutée (mutuellement exclusives). Après FinSi, le flot se réunit.



Organigramme de l'alternative complète Si ... Alors ...
Sinon ... FinSi. Les deux branches convergent vers la suite.

EXEMPLE Exemple : Tarification transport Yaoundé–Douala

La société « Voyage Confort » applique le tarif suivant sur l'axe Yaoundé–Douala (environ 270 km) :

- Moins de 12 ans : 2500 FCFA (tarif enfant)
- 12 ans et plus : 5000 FCFA (tarif adulte)

```
Algorithme TarifTransport
Variable age : Entier, tarif : Entier
Début
```

```
    Ecrire("Âge du voyageur : ")
    Lire(age)
    Si age < 12 Alors
```

```

    tarif <- 2500
    Ecrire("Tarif enfant :", tarif, " FCFA")
Sinon
    tarif <- 5000
    Ecrire("Tarif adulte :", tarif, " FCFA")
FinSi
Fin

```

Testez mentalement : $\text{age} = 10 \rightarrow$ **branche Alors** (2500 FCFA); $\text{age} = 12 \rightarrow$ **branche Sinon** (5000 FCFA). La frontière est nette.

3.5 Alternatives imbriquées et cas multiples

Lorsque les cas sont plus de deux, on **imbrique** des alternatives : le **Sinon** contient un nouveau **Si**. Pour la lisibilité, on aligne souvent les **Si** successifs (style « **Sinon Si** »).

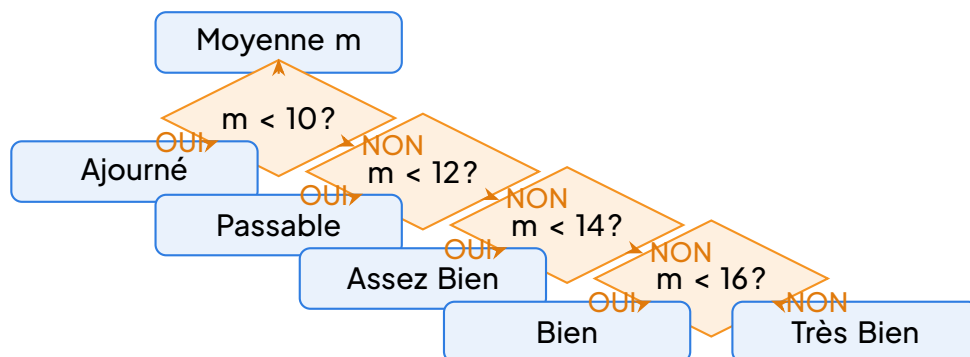
PROPRIÉTÉ Alternatives imbriquées (style « **Sinon Si** »)

```

Si <cond1> Alors
    <bloc1>
Sinon Si <cond2> Alors
    <bloc2>
Sinon Si <cond3> Alors
    <bloc3>
...
Sinon
    <bloc par défaut>
FinSi

```

Chaque condition est testée dans l'ordre ; dès qu'une est **VRAI**, son bloc s'exécute et on sort de toute la structure (on saute au **FinSi** final).



Arbre de décision pour la mention selon la moyenne (alternatives imbriquées).
Chaque losange teste une borne ; on descend à gauche si vrai, à droite si faux.

MÉTHODE Traduire un énoncé à cas multiples en alternatives imbriquées

1. Identifiez la variable discriminante (ex. : la moyenne m).
2. Ordonnez les bornes du plus petit au plus grand (ou inversement, mais restez cohérent).
3. Écrivez le premier **Si** sur la première borne (ex. **Si** $m < 10$ **Alors** « Ajourné »).
4. Pour chaque borne intermédiaire, écrivez **Sinon Si** <borne_suivante> **Alors** <mention>.
5. Terminez par un **Sinon** sans condition pour le cas restant (ex. **Sinon** « Très Bien »).
6. Fermez par un unique **FinSi**.
7. Relisez : chaque valeur possible de la variable doit tomber dans exactement un cas.

ATTENTION Pièges classiques avec les alternatives

- Oublier **FinSi** : chaque **Si** (simple ou complet) doit être fermé par son **FinSi**. Un **FinSi** manquant fait « avaler » la suite du programme par la branche **Alors**, ou provoque une erreur de syntaxe au Probatoire.
- Traiter deux fois le même cas : avec des **Si** indépendants (non imbriqués), une valeur peut satisfaire plusieurs conditions.

```
// MAUVAIS : deux Si indépendants
Si m >= 10 Alors Ecrire("Admis") FinSi
Si m >= 12 Alors Ecrire("Mention") FinSi
// Pour m=14, les DEUX messages s'affichent.
```

Correction : imbriquer ou utiliser **Sinon Si**.

- Confondre = (affectation) et = (comparaison) : en pseudo-code, l'affectation est $\leftarrow$; le test d'égalité s'écrit $=$. En C/JavaScript, c'est $==$ ou $===$; $=$ seul est une affectation (piège majeur!).
- Ordre des bornes inversé : si vous testez **Si** $m < 16$ **Alors** « Bien » **avant** **Si** $m < 14$ **Alors** « Assez Bien », une moyenne de 13 déclenchera « Bien » (car $13 < 16$ est vrai) au lieu de « Assez Bien ». Testez toujours de la borne la plus restrictive à la plus large.

EXERCICE RÉSOLU 3 Mention pour une moyenne saisie

Énoncé : Écrire un algorithme qui lit une moyenne m (réel, entre 0 et 20) et affiche la men-

tion correspondante selon le barème officiel camerounais :

Intervalle	Mention
[0, 10[	Ajourné
[10, 12[	Passable
[12, 14[	Assez Bien
[14, 16[	Bien
[16, 20]	Très Bien

Tester avec $m = 13,5$.

SOLUTION

Solution détaillée :

1. Analyse : Variable m (Réel). Structure alternative imbriquée sur 5 cas. Bornes ordonnées : 10, 12, 14, 16.
2. Algorithme :

```

Algorithme MentionEleve
Variable m : Reel
Début
    Ecrire("Entrez la moyenne (0 à 20) : ")
    Lire(m)
    Si m < 10 Alors
        Ecrire("Ajourné")
    Sinon Si m < 12 Alors
        Ecrire("Passable")
    Sinon Si m < 14 Alors
        Ecrire("Assez Bien")
    Sinon Si m < 16 Alors
        Ecrire("Bien")
    Sinon
        Ecrire("Très Bien")
    FinSi
Fin
  
```

3. Test avec $m = 13,5$:

$m < 10$? $13,5 < 10$ est FAUX → on passe au Sinon Si
 $m < 12$? $13,5 < 12$ est FAUX → on passe au Sinon Si
 $m < 14$? $13,5 < 14$ est VRAI → on exécute `Ecrire("Assez Bien")`
 On sort de la structure (FinSi).

4. Résultat affiché : Assez Bien.

LESAVAIS-TU ? L'organigramme, allié précieux au Probatoire TI

Au Probatoire Technologies de l'Information, l'épreuve pratique d'algorithmique note souvent l'organigramme autant que le pseudo-code. Un organigramme propre, complet, avec des losanges pour les tests, des flèches orientées, et des connexions correctes peut vous valoir la moitié des points même si votre écriture textuelle comporte une maladresse syntaxique. Prenez l'habitude, dès maintenant, de tracer l'organigramme avant d'écrire le code : c'est votre plan de construction, votre « architecture logicielle » en miniature. Les correcteurs apprécient la rigueur visuelle (alignement, espace, lisibilité).

3.6 Complément : l'instruction « Selon ... Cas » (choix multiple)

Bien que le programme officiel ne l'impose pas explicitement en Première, l'instruction `Selon` (ou `Cas`) est **très utile** pour les cas multiples sur une **variable discrète** (entier, caractère, énuméré). Elle évite l'imbrication fastidieuse et se traduit directement en `switch` en C/JavaScript.

PROPRIÉTÉ Syntaxe : Selon ... Cas

```
Selon <variable> Faire
  Cas <valeur1> : <instructions1>
  Cas <valeur2> : <instructions2>
  ...
  Default : <instructions_par_defaut>
FinSelon
```

Règles : <variable> doit être de type ordinal (entier, caractère). Les <valeur> sont des constantes (pas d'intervalles). Le `Default` (optionnel) capture toutes les valeurs non listées. Pas de « `break` » nécessaire : on exécute le cas correspondant et on sort directement de `FinSelon`.

EXEMPLE Selon ... Cas : jour de la semaine

```
Algorithme NomJour
Variable j : Entier // 1=Lundi ... 7=Dimanche
Début
  Ecrire("Numéro du jour (1-7) : ")
  Lire(j)
  Selon j Faire
    Cas 1 : Ecrire("Lundi")
    Cas 2 : Ecrire("Mardi")
    Cas 3 : Ecrire("Mercredi")
    Cas 4 : Ecrire("Jeudi")
    Cas 5 : Ecrire("Vendredi")
    Cas 6 : Ecrire("Samedi")
    Cas 7 : Ecrire("Dimanche")
```

```

    Defaut : Ecrire("Numéro invalide")
FinSelon
Fin

```

À RETENIR Résumé des alternatives

- Si ... Alors ... FinSi : 0 ou 1 action (cas unique).
- Si ... Alors ... Sinon ... FinSi : 1 action parmi 2 (choix binaire).
- Si ... Sinon Si ... Sinon ... FinSi : 1 action parmi n (intervalles, conditions complexes).
- Selon ... Cas ... FinSelon : 1 action parmi n (valeurs discrètes exactes, plus lisible).

Dans tous les cas : un seul bloc s'exécute, puis on continue après le FinSi / FinSelon.

EXERCICE 4 Entraînement : structures alternatives

1. Écrire un algorithme qui lit trois entiers a , b , c et affiche le plus grand (utiliser des alternatives imbriquées).
2. Un parking de la mairie de Douala facture : 1ère heure 500 FCFA, heures suivantes 300 FCFA. Écrire l'algorithme qui lit la durée de stationnement (entier, en heures) et affiche le montant à payer.
3. Traduire l'algorithme MentionEleve (ci-dessus) en version Selon ... Cas en expliquant pourquoi c'est impossible directement et comment contourner la limitation.

CORRIGÉ Exercice n°1 – Plus grand de trois entiers

```

Algorithme MaxDeTrois
Variables a, b, c, max : Entier
Début
    Ecrire("Entrez a, b, c : ")
    Lire(a, b, c)
    Si a >= b Alors
        Si a >= c Alors
            max <- a
        Sinon
            max <- c
        FinSi
    Sinon
        Si b >= c Alors
            max <- b

```

```

        Sinon
            max <- c
        FinSi
    FinSi
    Ecrire("Le maximum est : ", max)
Fin

```

Note : On teste $a \geq b$ puis, selon le résultat, on compare le vainqueur à c . Structure en « tournoi ».

CORRIGÉ Exercice n°2 – Parking Douala

```

Algorithme FacturationParking
Variables duree, montant : Entier
Début
    Ecrire("Durée de stationnement (heures) : ")
    Lire(duree)
    Si duree <= 1 Alors
        montant <- 500
    Sinon
        montant <- 500 + (duree - 1) * 300
    FinSi
    Ecrire("Montant à payer : ", montant, " FCFA")
Fin

```

CORRIGÉ Exercice n°3 – Selon ... Cas pour les mentions

Selon ne gère que des valeurs discrètes exactes, pas des intervalles continus comme $[10, 12[$. On ne peut pas écrire `Cas 10..11.99`.

Contournement : transformer la moyenne en un code discret (ex. `tronc(m/2)`) ou une fonction `tranche(m)` puis faire un `Selon` sur ce code. Mais l'alternative imbriquée reste la méthode naturelle et attendue au Probatoire pour des intervalles.

4 INITIALISATION, INCRÉMENTATION, DÉCRÉMENTATION

Dans la section précédente, nous avons découvert les structures alternatives (`Si ... Sinon`) qui permettent de prendre des décisions. Pour qu'une structure itérative (boucle) fonctionne correctement, il faut pouvoir compter les passages, accumuler des résultats et contrôler l'arrêt. C'est précisément le rôle de l'initialisation, de l'incrémentement et de la décrémentement. Sans ces mécanismes, une boucle ne sait ni par où commencer, ni comment avancer, ni quand

s'arrêter.

4.1 Initialisation : donner une valeur de départ

Avant d'utiliser une variable dans un calcul ou un comptage, il est **impératif** de lui attribuer une valeur initiale. Une variable non initialisée contient une valeur indéterminée (souvent appelée « valeur poubelle »), ce qui rend le résultat du programme imprévisible.

DÉFINITION Initialisation

L'**initialisation** est l'opération qui consiste à affecter une première valeur à une variable avant sa première utilisation. Elle se fait généralement juste après la déclaration ou au début de l'algorithme, en dehors de toute boucle.

PROPRIÉTÉ Valeurs d'initialisation usuelles

Le choix de la valeur initiale dépend de l'opération à effectuer sur la variable :

- Somme / Accumulation : initialiser à 0 (élément neutre de l'addition).
- Produit : initialiser à 1 (élément neutre de la multiplication).
- Compteur : initialiser à 0 (si on compte le nombre de passages) ou 1 (si on énumère des éléments à partir du premier).
- Recherche de maximum : initialiser à la première valeur du jeu de données (lue avant la boucle) ; à défaut, à une valeur très petite (conceptuellement $-\infty$).
- Recherche de minimum : initialiser à la première valeur du jeu de données (lue avant la boucle) ; à défaut, à une valeur très grande (conceptuellement $+\infty$).
- Concaténation de chaînes : initialiser à la chaîne vide "".

MÉTHODE Choisir la bonne valeur d'initialisation

1. Identifier le rôle de la variable : compteur, accumulateur (somme/produit), indicateur (max/min), drapeau (booléen).
2. Déterminer l'élément neutre de l'opération associée : 0 pour +, 1 pour ×, "" pour la concaténation.
3. Si la variable doit recevoir une valeur issue des données (ex. maximum, minimum), lire la première donnée avant la boucle et l'affecter à la variable.
4. Placer l'initialisation avant la structure itérative qui utilise la variable.

ATTENTION Piège classique : initialiser à l'intérieur de la boucle

```
// ERREUR : initialisation DANS la boucle
Pour i de 1 à 5 faire
    somme ← 0      // ← ERREUR : somme remise à 0 à chaque tour !
    somme ← somme + i
FinPour
Écrire(somme)    // Affiche 5 au lieu de 15
```

La variable `somme` est réinitialisée à chaque itération, effaçant le travail des tours précédents. Correction : placer `somme ← 0` avant `Pour`.

4.2 Incrémentation : faire avancer un compteur

L'incrémentation est l'opération qui augmente la valeur d'une variable d'un pas constant (généralement 1). Elle est le « moteur » des boucles à compteur.

DÉFINITION Incrémentation

L'**incrémentation** d'une variable `i` de pas `p` (entier naturel non nul) est l'affectation :

$$i \leftarrow i + p$$

Lorsque $p = 1$, on parle simplement d'incrémentation (ou incrémentation unitaire). Dans de nombreux langages (C, JavaScript), on utilise l'opérateur raccourci `i++` ou `++i`.

PROPRIÉTÉ Rôle de l'incrémentation dans une boucle

Dans une boucle `Pour` ou `Tant que`, l'incrémentation :

- fait progresser le compteur d'un pas vers la condition d'arrêt ;
- garantit (si le pas est bien choisi) que la condition de fin deviendra vraie après un nombre fini d'itérations ;
- permet d'indexer des tableaux (`T[i]`) ou de numéroter des étapes.

EXEMPLE Incrémentation de pas variable

On veut afficher les entiers pairs de 2 à 20. Le compteur `i` commence à 2 et s'incrémente de 2 à chaque tour.

```
i ← 2
Tant que i ≤ 20 faire
```

```
Écrire(i)
i ← i + 2      // Incrémentation de pas 2
FinTantQue
// Affiche : 2 4 6 8 10 12 14 16 18 20
```

4.3 Décrémentation : compter à rebours

La décrémentation diminue la valeur d'une variable. Elle est utile pour les boucles à rebours, les compte à rebours, ou le parcours d'un tableau de la fin vers le début.

DÉFINITION Décrémentation

La **décrémentation** d'une variable i de pas p est l'affectation :

$$i \leftarrow i - p$$

Pour un pas unitaire, on écrit $i \leftarrow i - 1$ (ou $i--$ en C/JavaScript).

EXEMPLE Boucle décomptée : affichage d'un compte à rebours

```
i ← 10
Répéter
  Écrire(i, "...")
  i ← i - 1      // Décrémentation
Jusqu'à i = 0
Écrire("Décollage !")
```

La condition d'arrêt $i = 0$ est testée après l'exécution du corps (structure Répéter ... Jusqu'à), donc la valeur 0 n'est pas affichée.

4.4 Accumulateur : construire un résultat pas à pas

Un accumulateur est une variable qui « mémorise » un résultat partiel et le met à jour à chaque itération. Contrairement au compteur qui ne fait que compter les passages, l'accumulateur agrège des valeurs (somme, produit, concaténation, etc.).

DÉFINITION Accumulateur

Un **accumulateur** est une variable initialisée à l'élément neutre de l'opération visée, puis

mise à jour à chaque itération par une affectation de la forme :

$$\text{accu} \leftarrow \text{accu} \ \text{valeur_courante}$$

où $\oplus$ est l'opérateur d'accumulation (+, ×, concaténation, max, min, etc.).

PROPRIÉTÉ**Différence fondamentale : compteur vs accumulateur**

Caractéristique	Compteur	Accumulateur
Valeur initiale	0 (ou 1)	Élément neutre de l'opération (0 pour +, 1 pour ×)
Mise à jour	$\text{cpt} \leftarrow \text{cpt} + 1$ (pas constant)	$\text{accu} \leftarrow \text{accu} \ \text{donnée}$ (dépend des données)
Sémantique	« Combien de fois ? »	« Combien / Quel résultat au total ? »
Exemple	Nombre de notes saisies	Somme des notes, moyenne, maximum

LESAVAIS-TU ?**Le premier « bug » de l'histoire**

En 1947, sur le Harvard Mark II, une mite coincée dans un relais a causé une erreur de calcul. L'opératrice Grace Hopper a collé l'insecte dans le journal de bord avec la mention « First actual case of bug being found ». Aujourd'hui encore, « déboguer » (déboguer) signifie traquer les erreurs logiques — comme une initialisation oubliée ou une incrémentation mal placée.

4.5 Illustration : évolution parallèle d'un compteur et d'un accumulateur

Le tableau ci-dessous montre l'évolution pas à pas d'un compteur `cpt` et d'un accumulateur `somme` lors du calcul de la somme des 5 premiers entiers (1 + 2 + 3 + 4 + 5).

Itération	Valeur lue (i)	Opération	Compteur	Accumulateur somme
Init.	-	cpt ← 0 ; somme ← 0	0	0
1		somme ← somme + 1 ; cpt ← cpt + 1	1	1
2		somme ← somme + 2 ; cpt ← cpt + 1	2	3
3		somme ← somme + 3 ; cpt ← cpt + 1	3	6
4		somme ← somme + 4 ; cpt ← cpt + 1	4	10
5		somme ← somme + 5 ; cpt ← cpt + 1	5	15

Évolution du compteur et de l'accumulateur sur 5 itérations — le compteur augmente de 1 à chaque tour, l'accumulateur ajoute la valeur courante.

Le schéma ci-dessous représente la même évolution sous forme de « boîtes mémoire » qui se mettent à jour en parallèle.

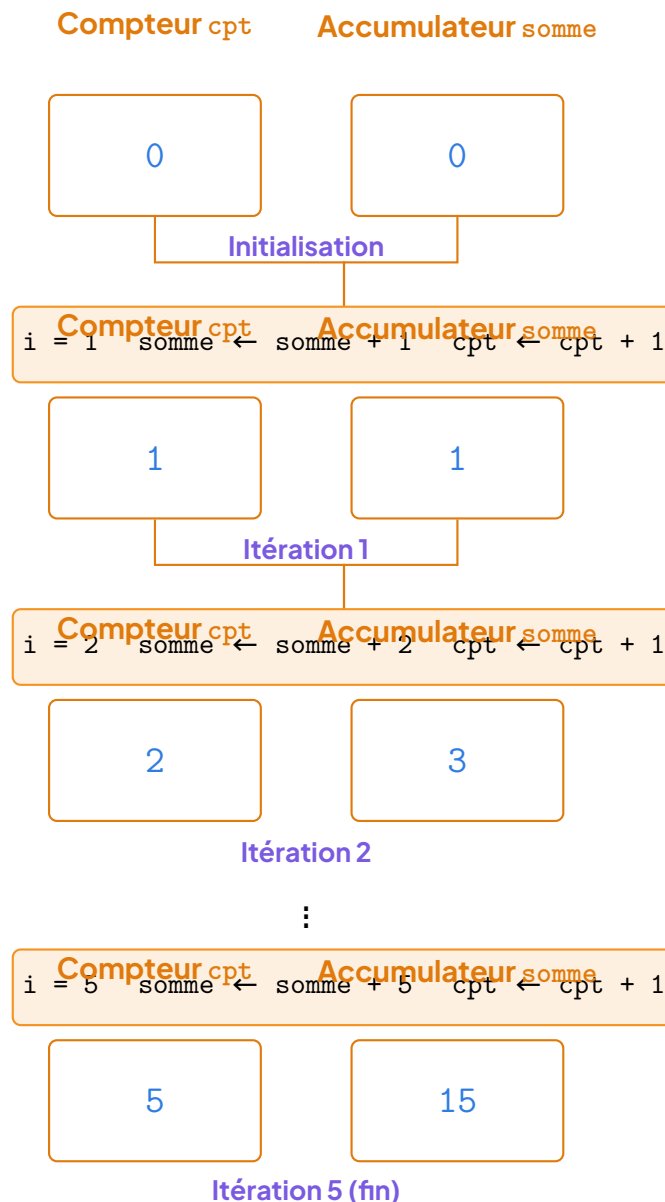


Schéma d'évolution parallèle : le compteur et l'accumulateur sont deux variables distinctes mises à jour à chaque itération.

4.6 Exemple résolu complet : les ventes journalières d'Aminatou

EXERCICE RÉSOLU 4 Calcul du chiffre d'affaires hebdomadaire

Énoncé. Aminatou tient une boutique de vêtements au marché Mokolo à Yaoundé. Elle note ses ventes quotidiennes (en FCFA) du lundi au vendredi : 500, 1 000, 250, 750, 1 200.

Elle veut un algorithme qui calcule et affiche le total de la semaine ainsi que le nombre de jours ouvrés.

SOLUTION

Analyse.

- Données : 5 valeurs numériques (ventes journalières).
- Résultats attendus : `total` (somme) et `nbJours` (compteur).
- Traitement : parcours séquentiel des 5 valeurs, accumulation dans `total`, comptage dans `nbJours`.

Algorithme (pseudo-code standard).

```
Algorithme VentesHebdo
Variables
    total, nbJours, venteDuJour : Entier
    i : Entier
Début
    // 1. INITIALISATION (hors boucle)
    total ← 0          // Élément neutre de l'addition
    nbJours ← 0        // Compteur démarre à 0

    // 2. BOUCLE DE TRAITEMENT (5 itérations fixes → Pour)
    Pour i de 1 à 5 faire
        // Lecture de la vente du jour (simulée ici par des valeurs fixes)
        // Note : en situation réelle, on utiliserait Lire(venteDuJour)
        Si i = 1 alors venteDuJour ← 500
        Sinon Si i = 2 alors venteDuJour ← 1000
        Sinon Si i = 3 alors venteDuJour ← 250
        Sinon Si i = 4 alors venteDuJour ← 750
        Sinon venteDuJour ← 1200
        FinSi

        // ACCUMULATION
        total ← total + venteDuJour

        // INCRÉMENTATION DU COMPTEUR
        nbJours ← nbJours + 1
    FinPour

    // 3. AFFICHAGE DES RÉSULTATS
    Écrire("Chiffre d'affaires de la semaine : ", total, " FCFA")
    Écrire("Nombre de jours ouvrés : ", nbJours)
Fin
```

Trace d'exécution (tableau de valeurs).

Itération i	venteDuJour	Opération sur total	total	nbJours
Init.	-	$\text{total} \leftarrow 0$; $\text{nbJours} \leftarrow 0$	0	0
1	500	$\text{total} \leftarrow 0 + 500$	500	1
2	1000	$\text{total} \leftarrow 500 + 1000$	1500	2
3	250	$\text{total} \leftarrow 1500 + 250$	1750	3
4	750	$\text{total} \leftarrow 1750 + 750$	2500	4
5	1200	$\text{total} \leftarrow 2500 + 1200$	3700	5

Résultat affiché :

Chiffre d'affaires de la semaine : 3700 FCFA

Nombre de jours ouvrés : 5

Remarques pédagogiques.

- L'initialisation de `total` à 0 et de `nbJours` à 0 se fait une seule fois, avant la boucle.
- L'accumulation `total ← total + venteDuJour` utilise la valeur courante de `total` (résultat partiel) et la nouvelle donnée `venteDuJour`.
- L'incrémentation `nbJours ← nbJours + 1` est indépendante de la valeur des ventes ; elle compte simplement les passages.
- Si on avait initialisé `total` à l'intérieur de la boucle, le résultat final aurait été 1200 (dernière valeur seulement).

4.7 Erreurs fréquentes et bonnes pratiques

ATTENTION Les 5 erreurs à ne plus commettre

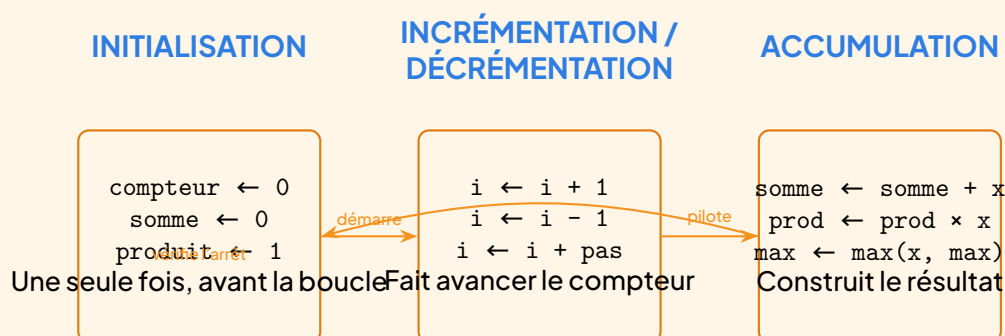
1. Initialiser à l'intérieur de la boucle : la variable repart à zéro à chaque tour → résultat faux.
2. Initialiser un produit à 0 : $0 \times \text{quoi que ce soit} = 0$. Toujours initialiser un produit à 1.
3. Oublier l'incrémentation (ou la décrémentation) dans une boucle Tant que ou Répéter → boucle infinie (le programme « plante »).
4. Confondre compteur et accumulateur : incrémenter l'accumulateur de 1 au lieu d'y ajouter la donnée, ou ajouter la donnée au compteur.
5. Mauvais pas d'incrémentation : par exemple $i \leftarrow i + 2$ dans une boucle Pour i de 1 à 10 qui attend un pas de 1 (conflit avec l'incrémentation implicite du Pour).

MÉTHODE Check-list avant de lancer une boucle

1. Identifié : quelle variable compte ? quelle variable accumule ?
2. Initialisé : compteur à 0 (ou 1), accumulateur à son neutre (0 pour +, 1 pour ×), avant la boucle.
3. Condition d'arrêt : claire, atteignable, liée à la variable qui évolue.
4. Mise à jour : incrémentation/décrémentation du compteur et accumulation de la donnée à l'intérieur de la boucle, dans le bon ordre (souvent : lire → accumuler → incrémenter).
5. Testé : faire une trace manuelle sur 2–3 itérations (papier/crayon) avant de coder.

4.8 Synthèse : le trio inséparable

Toute boucle bien construite repose sur trois piliers :



Maîtriser ces trois mécanismes, c'est s'assurer que chaque boucle que vous écrirez terminera correctement et produira le résultat attendu. Dans la prochaine section, nous les mettrons en œuvre concrètement dans les trois structures itératives : **Pour**, **Tant que** et **Répéter ... Jusqu'à**.

5 STRUCTURES ITÉRATIVES : POUR, TANT QUE, RÉPÉTER

Dans la section précédente, nous avons vu comment **initialiser**, **incrémenter** et **décrémenter** une variable. Ces trois opérations sont les briques élémentaires qui permettent de contrôler la répétition d'un bloc d'instructions. Sans elles, une boucle tournerait indéfiniment — c'est le fameux bug de boucle infinie que tout programmeur redoute. Nous allons maintenant assembler ces briques pour construire les trois structures itératives officielles du programme : **Pour**, **Tant que** et **Répéter ... Jusqu'à**.

5.1 Qu'est-ce qu'une structure itérative ?

DÉFINITION Structure itérative (ou boucle)

Une structure itérative est une structure de contrôle qui permet d'exécuter plusieurs fois (de zéro à une infinité de fois) une même séquence d'instructions, appelée corps de la boucle, tant qu'une condition d'arrêt n'est pas réalisée.

À RETENIR Les trois piliers d'une boucle bien formée

Toute boucle correcte repose sur trois étapes indispensables, dans l'ordre :

1. Initialisation : donner une valeur de départ à la variable de contrôle (souvent un compteur).
2. Test de la condition d'arrêt : évaluer une expression booléenne qui détermine si l'on continue ou si l'on sort de la boucle.
3. Modification (incrément/décément) : faire évoluer la variable de contrôle pour que la condition d'arrêt devienne finalement vraie.

L'oubli de l'un de ces piliers est la cause n°1 des boucles infinies.

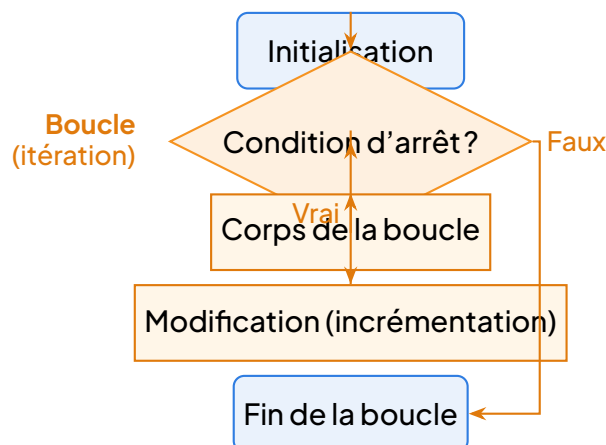


Schéma général d'une structure itérative : le cycle Initialisation → Test → Corps → Modification → Test...

5.2 La boucle Pour : nombre d'itérations connu à l'avance

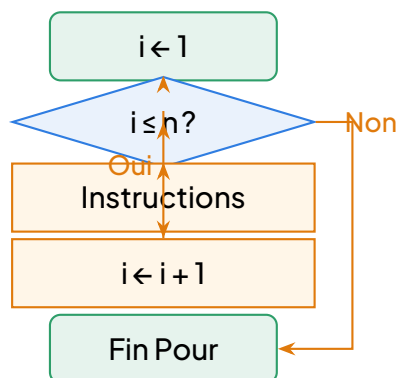
C'est la boucle la plus structurée et la plus sûre lorsque l'on sait combien de fois on doit répéter le traitement (parcours d'un tableau de taille fixe, affichage d'une table de multiplication, calcul d'une somme sur n termes).

PROPRIÉTÉ Syntaxe de la boucle Pour

```
Pour variable ← valeur_initiale À valeur_finale [Pas pas] Faire
    // Corps de la boucle (instructions)
FinPour
```

- **variable** : variable de contrôle (souvent i, j, k). Elle est locale à la boucle (ne doit pas être modifiée dans le corps).
- **valeur_initiale, valeur_finale** : entiers (ou expressions entières). La boucle s'exécute pour **variable** prenant les valeurs $\text{valeur_initiale}, \text{valeur_initiale} + \text{pas}, \dots$, jusqu'à **valeur_finale** incluse.
- **Pas** (optionnel, défaut = 1) : incrément (positif) ou décrément (négatif).

Règle d'or : Le nombre d'itérations est connu avant l'entrée dans la boucle : $N = \lfloor \frac{\text{finale} - \text{initiale}}{\text{pas}} \rfloor + 1$ (si $\text{pas} > 0$ et $\text{initiale} \leq \text{finale}$).

Test AVANT (0 à n fois)

Organigramme de la boucle Pour : le test est effectué avant la première itération. Si $1 > n$, le corps n'est jamais exécuté.

EXEMPLE Table de multiplication (exemple classique)

Écrire un algorithme qui affiche la table de multiplication d'un entier n saisi par l'utilisateur (de 1 à 10).

```
Algorithme TableMultiplication
Variable n, i : Entier
Début
    Écrire("Entrez un entier : ")
    Lire(n)
    Écrire("Table de ", n, " :")
```

```
Pour i ← 1 À 10 Faire
    Écrire(i, " x ", n, " = ", i * n)
FinPour
Fin
```

Analyse : Ici, le nombre d'itérations est fixe et connu (10). La variable *i* prend successivement les valeurs 1, 2, ..., 10. On n'a pas le droit de modifier *i* à l'intérieur de la boucle.

5.3 La boucle Tant que : test avant chaque itération (0 à n passages)

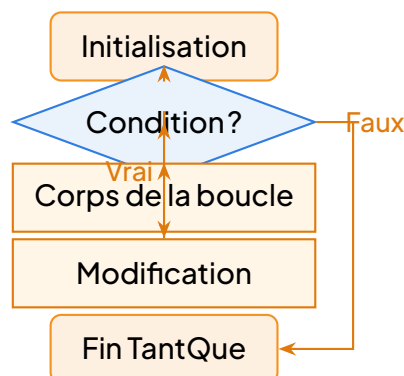
Utilisée lorsque le nombre d'itérations n'est **pas connu à l'avance**, mais dépend d'une condition qui devient fausse au cours de l'exécution (saisie utilisateur valide, recherche dans un fichier, convergence numérique).

PROPRIÉTÉ Syntaxe de la boucle Tant que

```
// Initialisation de la condition (hors boucle)
Tant que condition Faire
    // Corps de la boucle
// Modification de la condition (IMPÉRATIF)
FinTantQue
```

La *condition* est une expression booléenne. Tant qu'elle est VRAI, le corps s'exécute. Dès qu'elle devient FAUX, on sort de la boucle. Point crucial : Le test est fait AVANT la première exécution. Si la condition est fausse dès le départ, le corps ne s'exécute jamais (0 itération).

Test AVANT (0 à n fois)



Organigramme de la boucle Tant que : structure identique au Pour mais la gestion du compteur est manuelle.

ATTENTION Piège mortel : la boucle infinie par oubli d'incrémentation

C'est l'erreur n°1 en algorithmique et en C/JavaScript.

```
// ≡ INCORRECT : Boucle infinie garantie !  
i ← 1  
Tant que i ≤ 10 Faire  
    Écrire(i)  
    // Oubli de : i ← i + 1  
FinTantQue
```

La variable `i` vaut toujours 1, la condition `i ≤ 10` reste toujours vraie. Le programme « plante » (gel de l'interface, CPU à 100%). Vérifiez systématiquement que la variable testée est modifiée dans le corps.

EXEMPLE Saisie contrôlée d'une note (0 à 20) — Cas d'école du Tant que

On veut obliger l'utilisateur à entrer une note valide. On ne sait pas combien de fois il va se tromper.

```
Algorithme SaisieNoteControlee  
Variable note : Réel  
Début  
    // 1. Initialisation : on force l'entrée dans la boucle  
    note ← -1  
    // 2. Tant que la note est invalide, on redemande  
    Tant que note < 0 OU note > 20 Faire  
        Écrire("Entrez une note entre 0 et 20 : ")  
        Lire(note)  
        Si note < 0 OU note > 20 Alors  
            Écrire("Erreur : note hors limites. Recommencez.")  
        FinSi  
    FinTantQue  
    Écrire("Note acceptée : ", note)  
Fin
```

Pourquoi `Tant que`? Parce qu'on ne connaît pas le nombre d'essais. La condition d'arrêt (`note valide`) dépend de l'utilisateur.

5.4 La boucle Répéter ... Jusqu'à : test après chaque itération (au moins 1 passage)

C'est la variante « do-while » : le corps s'exécute **au moins une fois** avant le premier test. Idéale pour les menus interactifs ou la saisie contrôlée où l'on veut afficher la demande au moins une fois sans initialiser artificiellement la variable.

PROPRIÉTÉ Syntaxe de la boucle Répéter ... Jusqu'à

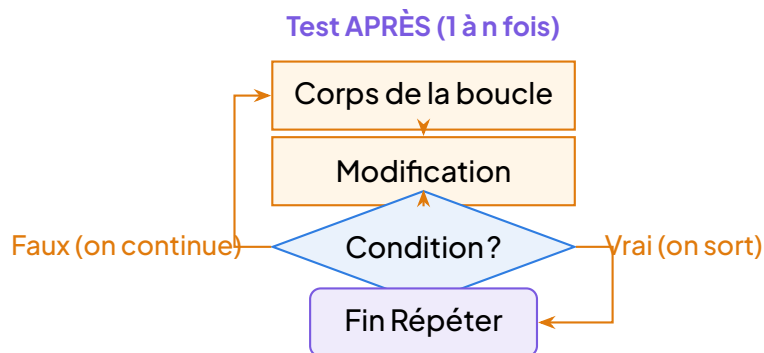
```

Répéter
  // Corps de la boucle (exécuté au moins 1 fois)
  // Modification de la condition
Jusqu'à condition

```

La **condition** est une expression booléenne. La boucle continue **TANT QUE** la condition est **FAUSSE**. Elle s'arrête quand la condition devient **VRAIE**. Attention : La logique est inversée par rapport à **Tant que** :

- Tant que (VRAI) → on continue.
- Jusqu'à (VRAI) → on arrête.



Organigramme de la boucle Répéter ... Jusqu'à : le corps est exécuté une première fois sans condition.

EXEMPLE Saisie contrôlée avec Répéter (plus naturel)

Même problème que précédemment, mais plus élégant : pas d'initialisation bidon.

```

Algorithme SaisieNoteRepeter
Variable note : Réel
Début
  Répéter
    Écrire("Entrez une note entre 0 et 20 : ")
    Lire(note)
    Si note < 0 OU note > 20 Alors
      Écrire("Erreur : note hors limites.")
    FinSi
  Jusqu'à note ≥ 0 ET note ≤ 20
  Écrire("Note acceptée : ", note)
Fin

```

Analyse : La condition d'arrêt est $\text{note} \geq 0 \text{ ET } \text{note} \leq 20$ (note valide). Tant qu'elle est fausse (note invalide), on répète. Le corps s'exécute au moins une fois : l'utilisateur est forcé de saisir quelque chose.

5.5 Comparaison et choix de la boucle

MÉTHODE Comment choisir la bonne boucle ?

Poser-vous ces trois questions dans l'ordre :

1. Le nombre d'itérations est-il connu avant d'entrer dans la boucle? (ex : traiter 10 lignes, afficher 1..n) → Pour.
2. Peut-on avoir zéro itération légitime? (ex : recherche qui échoue immédiatement, fichier vide) → Tant que.
3. Le corps doit-il s'exécuter au moins une fois? (ex : menu affiché avant choix, saisie utilisateur) → Répéter ... Jusqu'à.

Critère	Pour	Tant que	Répéter...Jusqu'à
Position du test			
Exécution minimale	Avant (entrée)	Avant (entrée)	
Continue si...	0 fois	0 fois	Après (sortie)
Arrêt quand...	Compteur ≤ borne	Condition VRAIE	1 fois
Cas d'usage type	Compteur > borne	Condition FAUSSE	Condition FAUSSE
Gestion variable	Parcours tableau, calcul sur n connu	Condition VRAIE	Condition VRAIE
	Automatique (interdite de la modifier)	Saisie contrôlée, recherche, fichier	Menu interactif, saisie
		Manuelle (init + incr dans corps)	Manuelle (incr dans corps)

Tableau comparatif des trois structures itératives. La couleur met en évidence les différences clés : position du test, minimum d'exécutions, logique de la condition.

5.6 Équivalence : toute boucle Pour peut s'écrire en Tant que

PROPRIÉTÉ Équivalence Pour ↔ Tant que (admise)

La boucle Pour est une structure syntaxique de commodité (sucre syntaxique). Tout algorithme utilisant Pour peut être réécrit exactement avec Tant que en explicitant les trois piliers.

```
// Version POUR (structure compacte)
Pour i ← 1 À n Faire
    Instructions
FinPour

// Version TANT QUE (structure explicite)
i ← 1           // 1. Initialisation
Tant que i ≤ n Faire // 2. Test
    Instructions
    i ← i + 1     // 3. Modification (incréméntation)
FinTantQue
```

L'inverse n'est pas toujours vrai : un **Tant que** dont la condition ne dépend pas d'un simple compteur linéaire ne peut pas se transformer en **Pour** standard.

EXERCICE RÉSOLU 5 Démonstration guidée : réécrire une boucle Pour en Tant que

Énoncé : Réécrire l'algorithme suivant en remplaçant la boucle **Pour** par une boucle **Tant que** strictement équivalente.

```
Algorithme SommePour
Variable n, i, somme : Entier
Début
    Écrire("Entrez n : "); Lire(n)
    somme ← 0
    Pour i ← 1 À n Faire
        somme ← somme + i
    FinPour
    Écrire("Somme = ", somme)
Fin
```

SOLUTION

Solution détaillée :

1. Identifier les 3 piliers du **Pour** :

- Initialisation : $i \leftarrow 1$ (début de l'intervalle).
- Condition de continuation : $i \leq n$ (tant qu'on n'a pas dépassé la borne finale).
- Modification : $i \leftarrow i + 1$ (pas par défaut = 1).

2. Construire le **Tant que** : Placer l'initialisation avant la boucle, la condition dans l'entête **Tant que**, la modification à la fin du corps.

3. Code résultant :

```
Algorithme SommeTantQue
Variable n, i, somme : Entier
Début
    Écrire("Entrez n : "); Lire(n)
```

```

    somme ← 0
    i ← 1           // 1. Initialisation EXPLICITE
    Tant que i ≤ n Faire // 2. Test EXPLICITE (condition de continuation)
        somme ← somme + i
        i ← i + 1     // 3. Modification EXPLICITE (incrémentation)
    FinTantQue
    Écrire("Somme = ", somme)
Fin

```

Vérification : Pour $n=3$, trace identique :

- $i=1$: test 1 3 (Vrai) → somme=1, $i=2$
- $i=2$: test 2 3 (Vrai) → somme=3, $i=3$
- $i=3$: test 3 3 (Vrai) → somme=6, $i=4$
- $i=4$: test 4 3 (Faux) → Sortie. Somme = 6. ☐

5.7 Exemple complet : somme des n premiers entiers (trace détaillée)

EXEMPLE Calcul de $S = 1 + 2 + \dots + 10$ avec les 3 boucles

Objectif : Calculer la somme des 10 premiers entiers ($S_{10} = 55$) et tracer les 3 premières itérations.

Version 1 : Pour (recommandée ici)

```

    somme ← 0
    Pour i ← 1 À 10 Faire
        somme ← somme + i
    FinPour
    // somme = 55

```

Version 2 : Tant que

```

    somme ← 0
    i ← 1
    Tant que i ≤ 10 Faire
        somme ← somme + i
        i ← i + 1
    FinTantQue

```

Version 3 : Répéter ... Jusqu'à

```

    somme ← 0
    i ← 1
    Répéter
        somme ← somme + i
        i ← i + 1
    Jusqu'à i > 10 // Attention : condition INVERSEE (arrêt quand VRAI)

```

Trace des 3 premières itérations (identique pour les 3 versions) :

Itération	i (début)	somme (avant)	Opération	État après itération
1	1	0	somme $\leftarrow$ 0 + 1	i=2, somme=1
2	2	1	somme $\leftarrow$ 1 + 2	i=3, somme=3
3	3	3	somme $\leftarrow$ 3 + 3	i=4, somme=6

Résultat final (après 10 itérations) : somme = 55. Note : La formule directe est $S_n = \frac{n(n+1)}{2} = \frac{10 \times 11}{2} = 55$. La boucle sert à comprendre la mécanique itérative.

5.8 Synthèse : écrire un algorithme itératif structuré

À RETENIR Check-list pour valider votre boucle

Avant de rendre votre copie ou de compiler votre code C/JS, cochez ces 5 points :

1. Initialisation : La variable de contrôle a-t-elle une valeur de départ avant le premier test ?
2. Condition : Est-elle correctement formulée (opérateurs `<`, `>`, `=`, et connecteurs `ET`, `OU`, `NON`) ?
3. Modification : La variable de contrôle est-elle obligatoirement modifiée dans le corps (pour `Tant que/Répéter`) ou gérée automatiquement (Pour) ?
4. Sens de la condition : `Tant que` continue si VRAI ; `Jusqu'à` continue si FAUX.
5. Terminaison garantie : Y a-t-il une preuve (même informelle) que la condition de sortie deviendra un jour vraie ?

LESAVAIS-TU ? La terminaison : propriété fondamentale de l'algorithme

Dans la définition formelle d'un algorithme (Knuth, 1968), la finitude (ou terminaison) est une propriété exigée : « Un algorithme doit toujours se terminer après un nombre fini d'étapes. » Une boucle infinie (`Tant que Vrai Faire ... FinTantQue` sans `break` ni modification de condition) n'est pas un algorithme, c'est un processus sans fin. En maintenance informatique, un programme « gelé » dans une boucle infinie consomme 100% du CPU, empêche le multitâche et force un redémarrage matériel (appui long sur le bouton `Power` ou `Ctrl+Alt+Suppr` pour tuer le processus). Au Cameroun, dans les cybercafés de Yaoundé ou Douala, c'est une cause fréquente de plantage de poste client mal codé en C ou JavaScript.

EXERCICE 5 Entraînement : Choisir et écrire

1. Écrire un algorithme qui affiche les entiers pairs de 2 à 20 inclus. (Quelle boucle?)
2. Écrire un algorithme qui demande un mot de passe jusqu'à ce que l'utilisateur tape « OnBuch2025 ». (Quelle boucle?)
3. Écrire un algorithme qui calcule 10! (factorielle) avec une boucle **Pour**, puis le réécrire avec **Tant que**.
4. Challenge : Écrire un algorithme qui lit des entiers positifs saisis au clavier et s'arrête dès qu'un nombre négatif est entré. Il affiche la somme des positifs saisis. (Attention : la condition d'arrêt porte sur le nombre lu, pas sur un compteur).

CORRIGÉ Exercice 1

1. Boucle **Pour** (nombre connu, pas de 2).

```
Pour i ← 2 À 20 Pas 2 Faire
    Écrire(i)
FinPour
```

2. Boucle **Répéter** ... Jusqu'à (au moins 1 saisie, condition d'arrêt = succès).

```
Variable mdp : Chaîne
Répéter
    Écrire("Mot de passe : "); Lire(mdp)
Jusqu'à mdp = "OnBuch2025"
Écrire("Accès autorisé.")
```

3. Factorielle 10! = 3 628 800.

```
// Version Pour
fact ← 1
Pour i ← 1 À 10 Faire
    fact ← fact * i
FinPour

// Version Tant que
fact ← 1; i ← 1
Tant que i ≤ 10 Faire
    fact ← fact * i
    i ← i + 1
FinTantQue
```

4. Boucle **Tant que** (nombre d'itérations inconnu, 0 possible si 1er nombre négatif).

```
Variable x, somme : Entier
somme ← 0
```

```
Écrire("Entrez un entier (négatif pour stopper) : ")
Lire(x)
Tant que x ≥ 0 Faire
    somme ← somme + x
    Écrire("Entrez un entier (négatif pour stopper) : ")
    Lire(x)
FinTantQue
Écrire("Somme = ", somme)
```

Note : La lecture est dupliquée (avant et dans la boucle) pour respecter la structure *Tant que* (test avant). Avec *Répéter*, on éviterait la duplication.

6 SYNTHÈSE : ÉCRIRE UN ALGORITHME COMPLET ET STRUCTURÉ

Dans les sections précédentes, tu as appris à utiliser séparément les trois structures itératives : Pour ... Faire, Tant que ... Faire et Répéter ... Jusqu'à. Tu sais aussi écrire des alternatives (Si ... Sinon) et des algorithmes séquentiels. Mais dans la réalité — et surtout au Probatoire — un problème ne se résout presque jamais avec une seule structure : il faut **combiner** séquence, alternative et boucle dans un même algorithme. Cette dernière section te donne la **méthodologie** complète pour passer d'un énoncé en français à un algorithme correct, lisible et vérifiable. Nous l'appliquerons à un fil rouge concret : la gestion des ventes du kiosque d'Ami-natou.

6.1 Méthodologie générale de résolution d'un problème

Face à un énoncé, le réflexe du débutant est de se précipiter sur le pseudo-code. C'est la meilleure façon de produire un algorithme faux. Le professionnel, lui, suit toujours la même démarche en six étapes, de l'analyse jusqu'à la vérification.

MÉTHODE Les 6 étapes de résolution algorithmique d'un problème

1. Analyser l'énoncé. Lire le problème plusieurs fois et le reformuler avec ses propres mots. Souligner les données fournies et le résultat attendu.
2. Identifier les entrées et les sorties. Les **entrées** sont les valeurs que l'utilisateur doit fournir (elles donneront lieu à des Lire). Les **sorties** sont les résultats à afficher (elles donneront lieu à des Écrire).
3. Choisir les variables, les constantes et leurs types. Dresser la liste de tous les objets nécessaires (compteurs, accumulateurs, valeurs saisies) en précisant pour chacun un nom significatif et un type (Entier, Réel, Chaîne, Booléen).
4. Choisir les structures de contrôle. Se poser les bonnes questions : y a-t-il une répéti-

tion? Le nombre de tours est-il connu à l'avance (Pour) ou dépend-il d'une condition (Tant que, Répéter)? Y a-t-il une décision à prendre (Si ... Sinon)?

5. Rédiger l'algorithme en pseudo-code, en respectant l'indentation et l'ordre : déclarations, initialisations, traitement, affichage.
6. Tracer l'algorithme sur un petit jeu de données pour vérifier qu'il produit bien les résultats attendus (voir la trace d'exécution plus bas).



La méthodologie de résolution algorithmique en 6 étapes (diagramme en escalier) : on ne rédige qu'après avoir analysé, et on ne conclut qu'après avoir tracé.

À RETENIR Le réflexe à acquérir

Un algorithme se conçoit avant de se rédiger. Les étapes 1 à 4 se font au brouillon ; seules les étapes 5 et 6 apparaissent sur la copie. Un algorithme bien analysé se rédige en quelques minutes ; un algorithme rédigé sans analyse se corrige pendant des heures.

6.2 Combiner les structures : une alternative à l'intérieur d'une boucle

La combinaison la plus fréquente — et la plus demandée à l'examen — consiste à placer un **test** à l'intérieur d'une **boucle**. À chaque tour de boucle, on examine un cas et on décide quoi en faire. C'est le schéma du comptage de cas : on parcourt des données une par une, et selon qu'elles vérifient ou non une condition, on met à jour des compteurs ou des accumulateurs différents.

EXEMPLE Schéma général du comptage de cas

Compter, parmi n notes saisies, combien sont au moins égales à 10 (admis) et combien sont inférieures à 10 (ajournés) :

```
nbAdmis <- 0
nbAjournes <- 0
Pour i Allant de 1 à n Faire
  Lire(note)
  Si note >= 10 Alors
    nbAdmis <- nbAdmis + 1
  Sinon
    nbAjournes <- nbAjournes + 1
  FinSi
FinPour
Ecrire("Admis : ", nbAdmis, " – Ajournés : ", nbAjournes)
```

On reconnaît le squelette : initialisation des compteurs avant la boucle, test à l'intérieur, incrémentation du bon compteur dans chaque branche, affichage après la boucle.

6.3 Fil rouge : la gestion des ventes du kiosque d'Aminatou

EXEMPLE Énoncé du problème

Aminatou tient un kiosque de beignets et de bouillons près du lycée. Chaque matin, elle sert ses clients l'un après l'autre. Pour chaque client, elle saisit le montant de l'achat (en FCFA) et la somme remise par le client. Si la somme remise est suffisante, elle rend la monnaie et valide la vente ; sinon, elle refuse la vente. À la fin de la matinée (elle ne connaît pas à l'avance le nombre de clients : elle répond « oui » ou « non » à la question « Y a-t-il encore un client ? »), elle veut afficher le nombre de ventes validées, le nombre de ventes refusées et le total encaissé.

Appliquons la méthodologie.

Étape 1 — Analyse. Répétition d'un même traitement (servir un client) un nombre de fois inconnu à l'avance ; à chaque client, une décision (solde suffisant ou non) ; à la fin, trois résultats cumulés.

Étape 2 — Entrées / sorties. Entrées : le montant de l'achat, la somme remise, la réponse « encore un client ? ». Sorties : nombre de ventes validées, nombre de ventes refusées, total encaissé.

Étape 3 — Variables et types.

Nom	Type	Rôle
montantAchat	Entier	Montant de l'achat du client courant (entrée)
sommeRemise	Entier	Somme remise par le client courant (entrée)
reponse	Chaîne	Réponse « oui »/« non » à « encore un client ? »
nbValidees	Entier	Compteur des ventes validées
nbRefusees	Entier	Compteur des ventes refusées
totalEncaisse	Entier	Accumulateur des montants encaissés

Étape 4 — Choix des structures. Le nombre de clients est inconnu à l'avance, mais chaque client est servi au moins une fois avant de poser la question : la boucle Répéter ... Jusqu'à est donc la plus naturelle (le corps s'exécute au moins une fois, et la condition d'arrêt est testée en fin de boucle). À l'intérieur, un Si ... Sinon décide si la vente est validée.

Étape 5 — Rédaction.

Algorithme KiosqueAminatou

Variables

montantAchat, sommeRemise : Entier
nbValidees, nbRefusees, totalEncaisse : Entier
reponse : Chaîne

Debut

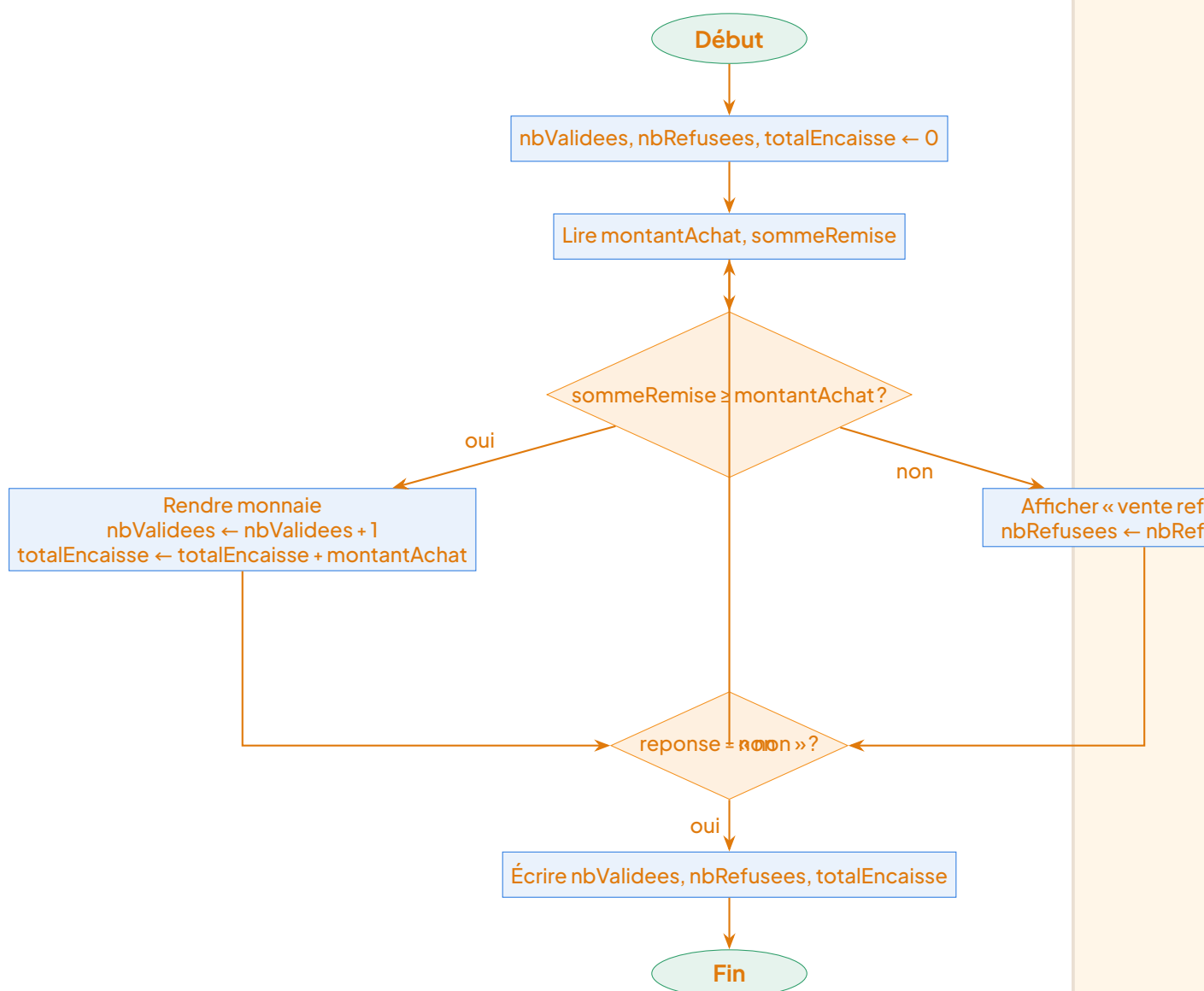
```
// Initialisation des compteurs et de l'accumulateur
nbValidees <- 0
nbRefusees <- 0
totalEncaisse <- 0
Repete
  Ecrire("Montant de l'achat (FCFA) : ")
  Lire(montantAchat)
  Ecrire("Somme remise par le client (FCFA) : ")
  Lire(sommeRemise)
  Si sommeRemise >= montantAchat Alors
    // Vente validée : on rend la monnaie et on cumule
    Ecrire("Monnaie à rendre : ", sommeRemise - montantAchat, " FCFA")
    nbValidees <- nbValidees + 1
    totalEncaisse <- totalEncaisse + montantAchat
  Sinon
    // Solde insuffisant : vente refusée
    Ecrire("Somme insuffisante : vente refusée.")
    nbRefusees <- nbRefusees + 1
  FinSi
  Ecrire("Y a-t-il encore un client ? (oui/non) : ")
  Lire(reponse)
Jusqu'à reponse = "non"
// Affichage des résultats de la matinée
Ecrire("Ventes validées : ", nbValidees)
Ecrire("Ventes refusées : ", nbRefusees)
Ecrire("Total encaissé : ", totalEncaisse, " FCFA")
```

Fin

ATTENTION Erreur fréquente : mélanger les rôles des variables

Ne confonds jamais un **compteur** et un **accumulateur**. Un compteur s'incrémente d'une

valeur fixe (`nbValidees <- nbValidees + 1`) : il compte des occurrences. Un accumulateur s'incrémente d'une valeur variable (`totalEncaisse <- totalEncaisse + montantAchat`) : il cumule des quantités. Écrire `totalEncaisse <- totalEncaisse + 1` compterait les ventes au lieu de cumuler les montants ; écrire `nbValidees <- nbValidees + montantAchat` donnerait un « nombre de clients » absurde. Autre piège lié : oublier l'initialisation à zéro avant la boucle — la variable part alors d'une valeur indéterminée et tous les résultats sont faux.



Organigramme complet de l'algorithme du kiosque : une boucle Répéter ... Jusqu'à contenant une alternative (test du solde). La flèche « non » du losange final ramène au début du corps de boucle.

6.4 Vérification par trace d'exécution

DÉFINITION Trace d'exécution

La **trace d'exécution** (ou table de trace) d'un algorithme est un tableau qui consigne, ligne par ligne, les valeurs successives de chaque variable lorsqu'on « joue » mentalement l'algorithme sur un jeu de données précis. C'est l'outil de vérification par excellence : si la trace ne donne pas les résultats attendus, l'algorithme contient une erreur.

EXEMPLE Trace complète du kiosque sur 3 clients

Jeu de données : client 1 achète pour 500 FCFA et remet 1000 FCFA ; client 2 achète pour 700 FCFA et remet 500 FCFA ; client 3 achète pour 200 FCFA et remet 200 FCFA ; puis réponse « non ».

Tour	montantAchat	sommeRemise	Test / action	nbVal.	nbRef.	total
—	—	—	Initialisation	0	0	0
1	500	1000	$1000 \geq 500$: monnaie 500, cumul	1	0	500
2	700	500	$500 < 700$: vente refusée	1	1	500
3	200	200	$200 \geq 200$: monnaie 0, cumul	2	1	700

La réponse « non » arrête la boucle. Affichage final : Ventes validées : 2 — Ventes refusées : 1 — Total encaissé : 700 FCFA. Vérification manuelle : $500 + 200 = 700$ FCFA encaissés sur 2 ventes. L'algorithme est correct sur ce jeu de données.

EXERCICE RÉSOLU 6 Adapter le fil rouge

Énoncé. Aminatou veut maintenant, en plus, connaître le montant de la plus grosse vente validée de la matinée (0 s'il n'y a eu aucune vente validée). Modifie l'algorithme en conséquence.

SOLUTION

Solution détaillée. On ajoute une variable `maxVente` de type Entier, initialisée à 0 (valeur neutre : toute vente validée est strictement positive, donc le premier maximum rencontré la remplacera). La mise à jour du maximum se fait uniquement dans la branche « vente validée », par un second test imbriqué :

Algorithme KiosqueAminatouMax

Variables

montantAchat, sommeRemise : Entier
nbValidees, nbRefusees, totalEncaisse, maxVente : Entier
reponse : Chaîne

Debut

nbValidees ← 0
nbRefusees ← 0
totalEncaisse ← 0
maxVente ← 0

Repete

Ecrire("Montant de l'achat (FCFA) : ")
Lire(montantAchat)
Ecrire("Somme remise par le client (FCFA) : ")
Lire(sommeRemise)
Si sommeRemise ≥ montantAchat Alors
Ecrire("Monnaie à rendre : ", sommeRemise - montantAchat, " FCFA")
nbValidees ← nbValidees + 1
totalEncaisse ← totalEncaisse + montantAchat
// Mise à jour du maximum (test imbriqué)
Si montantAchat > maxVente Alors
maxVente ← montantAchat
FinSi

Sinon

Ecrire("Somme insuffisante : vente refusée.")
nbRefusees ← nbRefusees + 1

FinSi

Ecrire("Y a-t-il encore un client ? (oui/non) : ")
Lire(reponse)

Jusqu'à reponse = "non"

Ecrire("Ventes validées : ", nbValidees)
Ecrire("Ventes refusées : ", nbRefusees)
Ecrire("Total encaissé : ", totalEncaisse, " FCFA")
Ecrire("Plus grosse vente : ", maxVente, " FCFA")

Fin

Sur le jeu de données précédent, la trace donne : tour 1, 500 > 0 donc $\text{maxVente} \leftarrow 500$; tour 2, vente refusée, pas de mise à jour ; tour 3, 200 < 500, maxVente inchangé. Affichage final : 500 FCFA. Remarque le piège évité : si le test de maximum avait été placé hors de la branche « validée », une vente refusée de 700 FCFA aurait faussé le résultat.

6.5 Les critères d'un bon algorithme

Un algorithme ne se juge pas seulement à ce qu'il « marche ». Quatre critères distinguent un bon algorithme d'un algorithme approximatif.

PROPRIÉTÉ Critères de qualité d'un algorithme

- **Correction** : l'algorithme produit le résultat attendu pour tout jeu de données valide, y compris les cas limites (aucun client, somme remise exactement égale au montant, etc.). On le vérifie par des traces.

- **Terminaison** : l'algorithme s'arrête toujours. Toute boucle doit posséder une **condition d'arrêt** que les instructions du corps rapprochent à chaque tour (par exemple, la lecture de `reponse` à la fin du corps du `Répéter`).
- **Lisibilité** : indentation stricte (le contenu d'un `Si` ou d'une boucle est décalé), une instruction par ligne, commentaires aux endroits clés.
- **Noms significatifs** : `totalEncaisse` se comprend immédiatement ; `x` ou `t` obligent le lecteur à deviner. Au Cameroun comme ailleurs, un correcteur — ou un collègue — doit comprendre ton algorithme sans toi.

ATTENTION Boucle infinie : le piège de la condition d'arrêt

Dans le fil rouge, si l'on oublie les lignes `Ecrire("Y a-t-il encore un client ?")` et `Lire(reponse)` à l'intérieur de la boucle, la variable `reponse` ne change jamais : la condition `Jusqu'à reponse = "non"` ne sera peut-être jamais satisfaite et l'algorithme bouclera indéfiniment. Règle d'or : toute variable intervenant dans la condition d'arrêt doit être modifiée à l'intérieur de la boucle.

6.6 Complément utile au Bac : présenter son pseudo-code en examen

MÉTHODE Présenter un algorithme sur sa copie

1. Commencer par la ligne `Algorithme NomSignificatif`, puis le bloc `Variables` avec tous les noms et types, avant le `Debut`.
2. Respecter l'enchaînement : initialisations, traitement (boucles et tests indentés), affichage des résultats, `Fin`.
3. Indenter chaque niveau de structure d'un même décalage constant ; aligner verticalement les `Si/FinSi`, `Pour/FinPour`, `Tant que/FinTantQue`, `Répéter/Jusqu'à`.
4. Utiliser le symbole d'affectation $\leftarrow$ (et non le signe `=`, réservé aux comparaisons).
5. Si le temps manque, rédiger d'abord le squelette (déclarations + structures vides avec leurs `Fin...`), puis remplir : un squelette correct rapporte déjà des points.

LESAVAIS-TU ? L'algorithmique au Bac TI

Analyse les sujets de Probatoire et de Baccalauréat TI des dernières années : les questions d'algorithmique portent presque toujours sur une boucle contenant un test — exactement

le schéma du kiosque d'Aminatou. Qu'il s'agisse de compter des notes admissibles, de cumuler des montants de factures ou de valider des saisies, le correcteur attend toujours les mêmes réflexes : initialisation avant la boucle, condition d'arrêt correcte, mise à jour des compteurs dans la bonne branche, affichage après la boucle. Maîtriser ce schéma unique, c'est sécuriser une part importante de l'épreuve.

EXERCICE 6 À toi de jouer

La bibliothèque du lycée prête des romans. Pour chaque élève qui se présente, on saisit le nombre de romans qu'il souhaite emprunter ; si ce nombre dépasse 3, le prêt est refusé, sinon il est accepté. La journée se termine quand le bibliothécaire répond « non » à « un autre élève ? ». Écris un algorithme complet qui affiche le nombre de prêts acceptés, le nombre de prêts refusés et le nombre total de romans prêtés, puis vérifie-le par une trace sur 3 élèves (demandes : 2, 5, 3).

CORRIGÉ Exercice — la bibliothèque

```

Algorithme PretsBibliotheque
Variables
    nbDemandes : Entier
    nbAcceptes, nbRefuses, totalRomans : Entier
    reponse : Chaîne
Debut
    nbAcceptes <- 0
    nbRefuses <- 0
    totalRomans <- 0
    Repeter
        Ecrire("Nombre de romans demandés : ")
        Lire(nbDemandes)
        Si nbDemandes <= 3 Alors
            nbAcceptes <- nbAcceptes + 1
            totalRomans <- totalRomans + nbDemandes
        Sinon
            Ecrire("Prêt refusé : maximum 3 romans.")
            nbRefuses <- nbRefuses + 1
        FinSi
        Ecrire("Un autre élève ? (oui/non) : ")
        Lire(reponse)
    Jusqu'à reponse = "non"
    Ecrire("Prêts acceptés : ", nbAcceptes)
    Ecrire("Prêts refusés : ", nbRefuses)
    Ecrire("Total de romans prêtés : ", totalRomans)
Fin
  
```

Trace sur (2, 5, 3) : tour 1, $2 \leq 3$, accepté, $\text{totalRomans} = 2$; tour 2, $5 > 3$, refusé ; tour 3, $3 \leq 3$, accepté, $\text{totalRomans} = 2 + 3 = 5$. Affichage : 2 acceptés, 1 refusé, 5 romans prêtés.

Attention au cas limite : la demande de 3 romans est acceptée car le test est ≤ 3 et non < 3 .

Tu disposes désormais de toute la panoplie : variables et constantes, instructions de base, alternatives, boucles, et surtout la méthode en six étapes pour les assembler. Dans la prochaine leçon, tu découvriras les **tableaux**, qui permettront à Aminatou de mémoriser toutes ses ventes de la matinée au lieu de les cumuler une à une.

7 ACTIVITÉ D'INTÉGRATION

8 ÉCRITURE D'ALGORITHMES SÉQUENTIELS ET STRUCTURES DE CONTRÔLE

8.1 Objectifs de la leçon

Cette leçon d'intégration vise à mobiliser l'ensemble des savoir-faire du module Algorithmique (6 h) relatifs à l'écriture d'algorithmes séquentiels et aux structures de contrôle. À l'issue de ce travail, vous devez être capable de :

- Analyser une situation-problème réelle pour en extraire les données d'entrée, les traitements à effectuer et les résultats attendus.
- Concevoir un algorithme complet structuré : déclaration des **variables** et **constantes**, initialisation correcte, utilisation des instructions de base (**lecture**, **écriture**, **affectation**).
- Choisir et implémenter la **structure de contrôle** adaptée : structure **alternative** (Si ... Sinon) pour la gestion de la remise, structure **itérative** (Tant que, Répéter ou Pour) pour la saisie des clients.
- Gérer correctement les **conditions d'arrêt** (sentinelle 0, nombre maximal de clients) et les compteurs/accumulateurs (incrémentations, décrémentation).
- Valider l'algorithme par une **trace d'exécution** rigoureuse sur un jeu de test représentatif.
- Argumenter le choix d'une boucle plutôt qu'une autre (Tant que vs Répéter) en fonction de la sémantique du problème (test en entrée vs test en sortie).
- Représenter graphiquement l'algorithme par un **algorithme** normé.

8.2 Situation professionnelle : La caisse du marché de Mokolo

Madame **Mendouga** tient un stand de vente de tissus pagnes au marché de Mokolo à Yaoundé. Pour moderniser sa gestion et éviter les erreurs de calcul en fin de journée, elle souhaite disposer d'un algorithme simple qu'elle pourrait ultérieurement faire implémenter sur une calculatrice programmable ou un petit script sur son téléphone.

Voici les règles de gestion qu'elle applique quotidiennement :

1. Elle reçoit ses clients les uns après les autres. La journée type ne dépasse pas **20 clients** (au-delà, elle ferme le rideau pour faire l'inventaire).
2. Pour chaque client, elle saisit le **montant total des achats** (en FCFA).
3. La saisie des clients s'arrête **immédiatement** si elle entre la valeur **0** (zéro), qui sert de **valeur sentinelle** pour signaler la fin de la journée avant la limite des 20 clients.
4. **Politique de fidélité** : Si le montant des achats d'un client dépasse **10 000 FCFA**, une **remise de 5 %** est accordée sur le montant total de ce client.

5. L'algorithme doit afficher, pour chaque client servi, le **montant net à payer** (après remise le cas échéant).
6. En fin de journée (après la sortie de boucle), l'algorithme affiche le **bilan journalier** :
 - Le **nombre total de clients servis** (hors la saisie 0).
 - La **recette totale** encaissée (somme des montants nets payés par tous les clients).

Données numériques pour le test (jeu d'essai à 4 clients) :

Client n°	Montant saisi (FCFA)
1	12 500
2	8 000
3	25 000
4	0 (fin de saisie)

Avec ce jeu de test, le client 1 bénéficie de la remise ($12\,500 > 10\,000$), le client 2 non, le client 3 oui. La saisie s'arrête au client 4 (valeur 0). Le nombre de clients servis est 3.

8.3 Consignes de l'activité

Travaillez en groupes de 3 ou 4 élèves. Chaque groupe rendra une feuille unique contenant les réponses aux questions ci-dessous.

1. **Analyse et modélisation** : Identifiez toutes les **variables** nécessaires (nom, type, rôle : entrée, sortie, travail, compteur, accumulateur). Identifiez la ou les **constantes** utiles (seuil de remise, taux de remise, nombre max de clients, valeur sentinelle). Proposez un **dictionnaire des variables** clair.
2. **Choix de la structure itérative** : Proposez deux versions de l'algorithme :
 - Version A : utilisant une boucle `Tant que ... Faire`.
 - Version B : utilisant une boucle `Répéter ... Jusqu'à`.

Pour chaque version, écrivez la condition d'arrêt exacte. Justifiez en une phrase pourquoi l'une ou l'autre est plus naturelle ici.

3. **Rédaction de l'algorithme complet (Version choisie)** : Rédigez l'algorithme final en pseudo-code normé (style Algorithme, Variables, Constantes, Début, Fin) en choisissant la version de boucle que vous jugez la plus adaptée. Soignez l'indentation et les commentaires.
4. **Représentation graphique** : Tracez l'algorithme correspondant à votre algorithme (symboles normés : ovale début/fin, parallélogramme entrée/sortie, rectangle traitement, losange décision).
5. **Trace d'exécution** : Effectuez la trace complète de votre algorithme sur le jeu de test à 4 clients fourni dans l'énoncé. Présentez la trace sous forme de tableau : N° passage | Variable(s) modifiée(s) | Valeur(s) | Affichage(s) écran.
6. **Analyse critique** : Que se passe-t-il si le premier montant saisi est 0 ? Que se passe-t-il si on saisit 21 montants non nuls ? Votre algorithme gère-t-il ces cas limites correctement ? Si non, corrigez-le.

7. **Présentation** : Un représentant du groupe présente l'algorithme au tableau. La classe compare les versions **Tant que** et **Répéter** et discute de la lisibilité de la condition d'arrêt composée (double condition : sentinelle ET limite de 20).

MÉTHODE Démarche de conception d'un algorithme de traitement de flux borné

1. Analyser le énoncé : Identifier les entrées (données fournies), les sorties (résultats attendus), les constantes (règles de gestion fixes) et les variables de travail (compteurs, accumulateurs, résultats intermédiaires).
2. Dresser le dictionnaire des variables : Nom explicite, type (Entier, Réel, Chaîne, Booléen), rôle (Entrée, Sortie, Travail, Compteur, Accumulateur), initialisation.
3. Choisir la structure de contrôle itérative :
 - Pour : nombre d'itérations connu à l'avance (ex. traiter exactement 20 clients).
 - Tant que : condition testée avant le corps (0 à n itérations), idéale si la première itération peut ne pas avoir lieu (ex. fichier vide).
 - Répéter : condition testée après le corps (1 à n itérations), idéale si une première saisie/traitement est toujours nécessaire (ex. menu interactif, saisie jusqu'à sentinelle).
4. Écrire le pseudo-code : Respecter la syntaxe normée, indenter rigoureusement (2 ou 4 espaces), commenter les blocs complexes.
5. Dessiner l'algorithme : Transcrire fidèlement la logique (attention aux connecteurs de boucle).
6. Valider par la trace : Simuler l'exécution pas à pas sur un jeu de test couvrant les cas nominaux et les cas limites (borne, sentinelle immédiate, dépassement).

8.4 Ressources syntaxiques à mobiliser

À RETENIR Syntaxe de référence pour le pseudo-code (norme Probatoire)

Déclarations :

```
Algorithme Nom_Algorithme
Variables
    NomVar1, NomVar2 : Type // Types : Entier, Reel, Chaîne, Booleen
    NomVar3 : Type
Constantes
    NomConst1 = Valeur
    NomConst2 = Valeur
Debut
    // Corps de l'algorithme
Fin
```

```
Lire(Variable)           // Saisie clavier (entrée)
Ecrire(Chaine, Variable) // Affichage écran (sortie), concaténation par virgule
Variable <- Expression  // Affectation (calcul, incrémentation, accumulation)
// Exemples :
cpt <- cpt + 1           // Incrémentation
somme <- somme + val     // Accumulation
flag <- Vrai             // Booléen
```

```
Si Condition Alors
    Instruction_Si_Vrai
Sinon
    Instruction_Si_Faux
Fin Si

// Version sans Sinon :
Si Condition Alors
    Instruction_Si_Vrai
Fin Si
```

```
// Amorçage obligatoire si variable de contrôle lue dans la boucle
Lire(VarContrôle)
Tant que Condition_Continuation Faire
    Corps_de_boucle
    // Modification de VarContrôle indispensable (lecture, incrémentation...)
    Lire(VarContrôle) // Ou cpt <- cpt + 1
Fin Tant que
```

```
Repetier
    Corps_de_boucle // La saisie/modification de la variable de contrôle est ICI
Jusqu'à Condition_Arret // Condition_Arret = NON(Condition_Continuation)
```

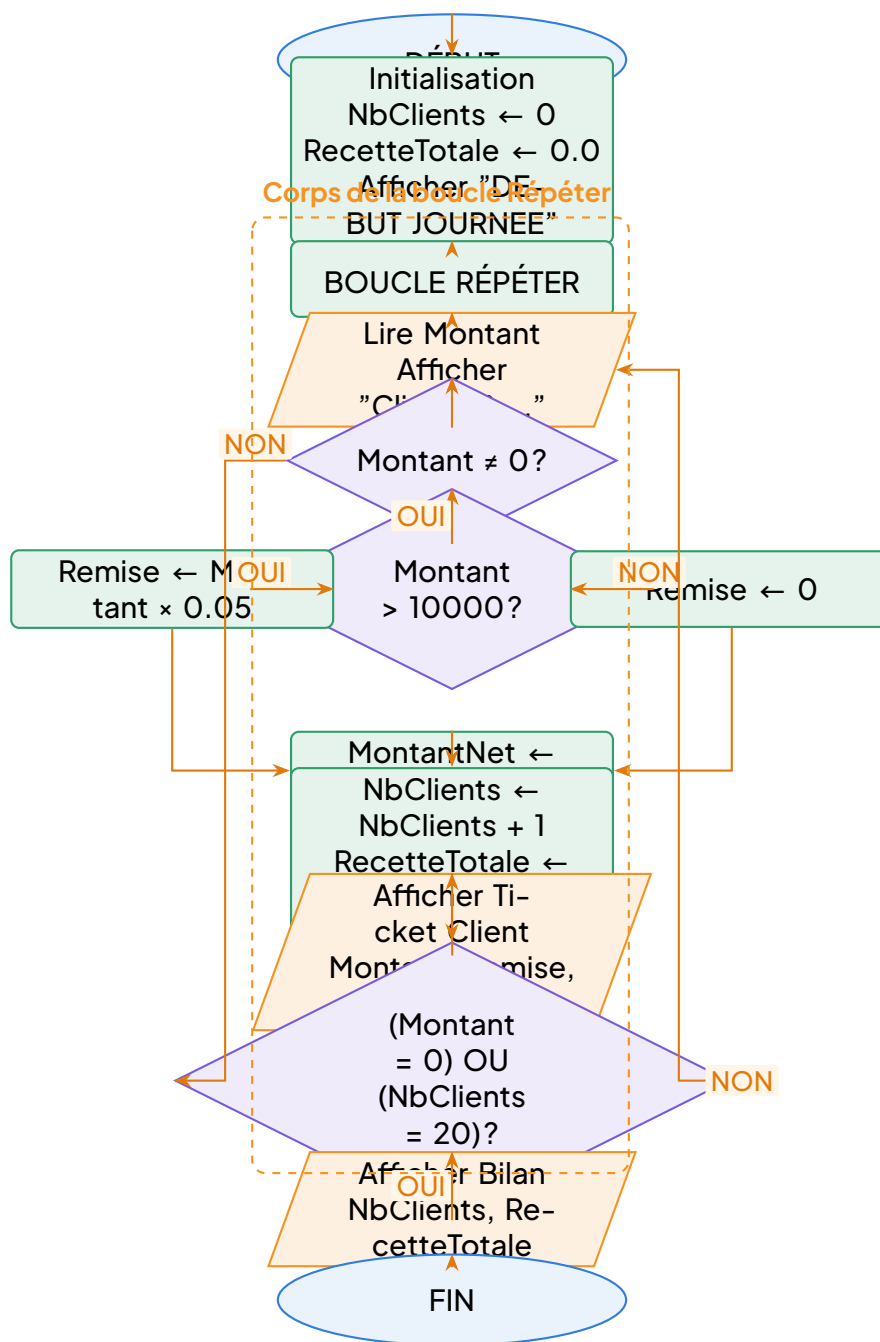
ATTENTION Pièges classiques à éviter absolument

- Oubli d'initialisation : Un compteur (NbClients) ou un accumulateur (RecetteTotale) doit être initialisé à 0 avant la boucle. C'est la cause n°1 d'erreurs aux examens.
- Sentinelle comptabilisée : Ne JAMAIS incrémenter le compteur ni accumuler la valeur sentinelle (0). Utiliser `Si Var ≠ Sentinelle Alors ... Fin Si` à l'intérieur de la boucle.
- Boucle `Tant que` sans amorçage : Si la variable testée dans `Tant que` est lue dans la

boucle, il faut une lecture avant la boucle (amorçage). Sans cela, la variable a une valeur indéterminée → comportement aléatoire ou boucle infinie.

- **Condition d'arrêt inversée :** Pour `Répéter`, la condition est celle qui arrête la boucle (ex. `Montant = 0`). Pour `Tant que`, c'est la condition qui permet de continuer (ex. `Montant ≠ 0`). `Condition_Arret = NON(Condition_Continuation)`.
- **Types incompatibles :** Affecter un `Réel` dans un `Entier` sans conversion (partie entière) ou comparer un `Entier` et une `Chaîne`.
- **Indentation fantaisiste :** L'indentation ne change pas la logique pour l'ordinateur, mais elle est obligatoire pour la lisibilité humaine et la notation à l'examen.

8.5 Algorithme de la solution (Version Répéter)



Algorithme de l'algorithme Caisse_Mokolo (Version Répéter ... Jusqu'à). Les losanges représentent les décisions, les parallélogrammes les entrées/sorties, les rectangles les traitements.

8.6 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 Corrigé détaillé de l'activité « Caisse du marché de Mokolo »

1. Analyse et dictionnaire des variables

Nom	Type	Rôle	Description / Initialisation
NbMaxClients	Entier	Constante	= 20 (limite haute journalière)
SeuilRemise	Entier	Constante	= 10000 (FCFA, déclencheur remise)
TauxRemise	Réel	Constante	= 0.05 (5 %)
Sentinelle	Entier	Constante	= 0 (valeur d'arrêt saisie)
Montant	Entier	Entrée / Travail	Montant saisi pour le client courant
Remise	Réel	Travail	Remise calculée pour le client courant
MontantNet	Réel	Sortie / Travail	Montant à payer après remise
NbClients	Entier	Compteur	Nombre de clients servis (init 0)
RecetteTotale	Réel	Accumulateur	Somme des MontantNet (init 0.0)

2. Choix de la structure itérative

- **Version A (Tant que) :** La condition de continuation est : (Montant $\neq$ Sentinelle) ET (NbClients < NbMaxClients). Le test se fait avant la première saisie. Il faut donc faire une lecture initiale (amorçage) avant la boucle, puis une seconde lecture à la fin du corps de boucle.
- **Version B (Répéter) :** La condition d'arrêt est : (Montant = Sentinelle) OU (NbClients = NbMaxClients). Le test se fait après le corps de boucle. La saisie se fait à l'intérieur de la boucle. Cela évite la redondance de l'instruction Lire(Montant) (amorçage + lecture dans la boucle).
- **Justification :** La version Répéter est plus naturelle ici car on veut au moins tenter de saisir un client (même si le premier est 0, on entre dans la boucle, on teste, on sort). Elle évite la redondance de l'instruction Lire(Montant) et garantit que Montant a une valeur connue avant le test d'arrêt.

3. Algorithme complet (Version Répéter choisie)

```

Algorithme Caisse_Mokolo
// Gestion des ventes quotidiennes avec remise fidélité
// Auteur : Groupe X - Date : ...

```

Variables

```

Montant, NbClients : Entier
Remise, MontantNet, RecetteTotale : Reel

```

Constantes

```

NbMaxClients = 20
SeuilRemise = 10000
TauxRemise = 0.05
Sentinelle = 0

```

```

Debut
// Initialisation des compteurs et accumulateurs (CRITIQUE)
NbClients <- 0
RecetteTotale <- 0.0

Ecrire("=== DEBUT DE JOURNEE - CAISSE MOKOLO ===")
Ecrire("Entrez 0 pour terminer la saisie.")

// Boucle de saisie et traitement par client
Repete
  Ecrire("Client n° ", NbClients + 1, " - Montant achats (FCFA) ? ")
  Lire(Montant)

  // Traitement seulement si ce n'est pas la sentinelle
  Si Montant <> Sentinelle Alors
    // Calcul remise
    Si Montant > SeuilRemise Alors
      Remise <- Montant * TauxRemise
    Sinon
      Remise <- 0
    Fin Si

    MontantNet <- Montant - Remise

    // Mise à jour bilan
    NbClients <- NbClients + 1 // Incrémentation compteur
    RecetteTotale <- RecetteTotale + MontantNet // Accumulation

    // Affichage ticket client
    Ecrire("-----")
    Ecrire("Client ", NbClients, " : Montant initial = ", Montant, " FCFA")
    Si Remise > 0 Alors
      Ecrire("Remise (5%) = ", Remise, " FCFA")
    Fin Si
    Ecrire(">>> A PAYER : ", MontantNet, " FCFA")
    Ecrire("-----")
  Fin Si

  // Condition d'arrêt : Sentinelle atteinte OU Quota max atteint
  Jusqu'a (Montant = Sentinelle) Ou (NbClients = NbMaxClients)

// Affichage Bilan Final
Ecrire("")
Ecrire("=== BILAN JOURNALIER ===")
Ecrire("Nombre de clients servis : ", NbClients)
Ecrire("Recette totale encaissée : ", RecetteTotale, " FCFA")
Ecrire("=====")
Fin

```

4. Trace d'exécution (Jeu de test : 12500, 8000, 25000, 0)

Passage	Instruction clé / Variable modifiée	Valeur	Affichage Écran
Initialisation			
-	NbClients, RecetteTotale	0, 0.0	"=== DEBUT...", "Entrez 0..."
Itération 1 (Client 1 : 12500)			
1	Lire(Montant)	12500	"Client n° 1 - Montant...?"
	Si Montant ≠ 0 (Vrai)	-	-
	Si 12500 > 10000 (Vrai)	-	-
	Remise ← 12500 * 0.05	625.0	-
	MontantNet ← 12500 - 625	11875.0	-
	NbClients ← 0 + 1	1	-
	RecetteTotale ← 0 + 11875	11875.0	-
	Affichage ticket	-	"Client 1 : ... A PAYER : 11875.0 FCFA"
	Test Jusqu'à (12500=0 OU 1=20)	Faux	(Boucle continue)
Itération 2 (Client 2 : 8000)			
2	Lire(Montant)	8000	"Client n° 2 - Montant...?"
	Si Montant ≠ 0 (Vrai)	-	-
	Si 8000 > 10000 (Faux)	-	-
	Remise ← 0	0.0	-
	MontantNet ← 8000 - 0	8000.0	-
	NbClients ← 1 + 1	2	-
	RecetteTotale ← 11875 + 8000	19875.0	-
	Affichage ticket	-	"Client 2 : ... A PAYER : 8000.0 FCFA"
	Test Jusqu'à (8000=0 OU 2=20)	Faux	(Boucle continue)
Itération 3 (Client 3 : 25000)			
3	Lire(Montant)	25000	"Client n° 3 - Montant...?"
	Si Montant ≠ 0 (Vrai)	-	-
	Si 25000 > 10000 (Vrai)	-	-
	Remise ← 25000 * 0.05	1250.0	-
	MontantNet ← 25000 - 1250	23750.0	-
	NbClients ← 2 + 1	3	-
	RecetteTotale ← 19875 + 23750	43625.0	-
	Affichage ticket	-	"Client 3 : ... A PAYER : 23750.0 FCFA"

Test Jusqu'à (25000=0 OU 3=20) Faux (Boucle continue)

Itération 4 (Sentinelle : 0)

4	Lire(Montant)	0	"Client n° 4 - Montant...?"
	Si Montant ≠ 0 (Faux)	-	(Bloc Si ignoré, pas d'incrément)

Vérification manuelle : Client 1 : 11875, Client 2 : 8000, Client 3 : 23750. Total = 43625 FCFA. Clients = 3. Correct.

5. Analyse des cas limites

- **Cas 1 : Premier montant = 0.** La boucle Répéter s'exécute une fois. Lire(0) → Si 0 ≠ 0 (Faux) → pas d'incréméntation (NbClients = 0), pas d'accumulation. Test Jusqu'à (0=0) Vrai → Sortie. Bilan affiche 0 clients, 0 FCFA. Comportement correct.
- **Cas 2 : 21 montants non nuls saisis.** Au 20ème client : NbClients passe de 19 à 20. Test Jusqu'à (Faux OU 20=20) → Vrai. Sortie immédiate. Le 21ème Lire n'est jamais exécuté. Comportement correct (protection quota).
- **Attention piège (Version Tant que) :** Si on utilisait Tant que sans lecture initiale (amorçage), la variable Montant serait non initialisée → comportement indéterminé (bug). Avec Répéter, la lecture est dans la boucle, donc Montant a toujours une valeur avant le test. C'est un argument fort pour Répéter ici.

6. Comparaison Tant que VS Répéter (Synthèse pour la classe)

Critère	Boucle Tant que	Boucle Répéter
Amorçage	Obligatoire : Lire(Montant) avant boucle	Inutile : Lire inside boucle
Condition	Continuation : (M≠0) ET (Nb<20)	Arrêt : (M=0) OU (Nb=20)
Logique	"Tant qu'on peut continuer, on traite"	"On traite, puis on vérifie si on s'arrête"
Risque	Oubli de la 2ème lecture dans la boucle (boucle infinie)	Exécution du corps une fois même si sentinelle immédiate (géré par Si interne)
Recommandation	Préférable si 0 itérations possible et pas de saisie dans le corps	Préférable ici car saisie est le déclencheur du traitement

8.7 Bilan et compétences validées

À RETENIR Points clés à retenir pour le Probatoire

1. Rigueur de l'initialisation : Oublier NbClients ← 0 ou RecetteTotale ← 0.0 fausse tout le bilan. Erreur n°1 aux examens.
2. Exclusion de la sentinelle : Il est impératif de ne pas comptabiliser la valeur 0. L'imbrication Si Montant ≠ Sentinelle Alors ... Fin Si à l'intérieur de la boucle est le

pattern standard pour exclure la sentinelle des calculs.

3. Choix éclairé de la boucle : La boucle Répéter ... Jusqu'à s'est avérée supérieure ici car l'action de saisie (Lire) fait partie intégrante du traitement itératif. Elle garantit que la variable de contrôle Montant a une valeur connue avant le test d'arrêt, évitant le bug classique de la variable non initialisée en Tant que.
4. Condition composée (double critère) : La gestion simultanée d'une sentinelle utilisateur (0) et d'une limite technique (20) impose l'opérateur logique OU (pour l'arrêt) / ET (pour la continuation). La table de vérité de cette condition a été validée par la trace.
5. La trace comme preuve : La trace tabulaire n'est pas un exercice scolaire inutile ; c'est l'outil de débogage par excellence avant le codage. Elle a permis de valider le calcul des remises (5 % sur 12500 = 625, sur 25000 = 1250) et l'accumulation correcte (43625 FCFA).
6. Algorithme : La représentation graphique (losange pour Si et Jusqu'à, parallélogramme pour Lire/Ecrire, rectangle pour ←) doit être fidèle au pseudo-code. Tout chemin sortant d'un losange doit être étiqueté OUI / NON (ou VRAI / FAUX).

LESAVAIS-TU ?

Le saviez-vous ? Le Cameroun numérique et l'algorithmique

Au Cameroun, la numérisation des marchés (projet e-Market du MINCOMMERCE) et la facturation électronique (DGI) reposent exactement sur ce type d'algorithmes : lecture séquentielle de lignes de facture, application de règles de gestion (TVA, remises, plafonds), accumulation de totaux, édition de reçus. Les caisses enregistreuses modernes (POS) exécutent en C ou Java embarqué la logique que vous venez d'écrire en pseudo-code. Maîtriser la Répéter sentinelle, c'est comprendre le cœur de tout système de point de vente (POS) ou de gestion de stock en entrée de gamme. En Terminale TI, vous transposerez cet algorithme en langage C (do...while, scanf/printf) et en JavaScript (boucle do...while ou while avec prompt/alert ou manipulation du DOM) pour une application web de caisse.

8.8 Exercices d'entraînement (Devoir personnel)

EXERCICE 7

Gestion de notes de classe

Un professeur souhaite saisir les notes d'un devoir (sur 20) pour sa classe de 50 élèves maximum. La saisie s'arrête sur une note négative (sentinelle) ou si les 50 notes sont entrées. L'algorithme doit afficher la moyenne de la classe, la note maximale et le nombre d'élèves ayant une note ≥ 10 (admis).

1. Proposez le dictionnaire des variables (identifiez compteurs, accumulateurs, max).

2. Écrivez l'algorithme complet avec une boucle `Tant que` (pensez à l'amorçage!).
3. Tracez l'algorithme correspondant.
4. Testez avec : 12, 8, 15, -1.

EXERCICE 8 Distributeur de billets simplifié

Un distributeur automatique de billets (DAB) ne délivre que des coupures de 10 000 et 5 000 FCFA. Le client saisit le montant souhaité (multiple de 5000, max 200 000). L'algorithme calcule et affiche le nombre minimal de billets de 10 000 et de 5 000 à rendre. (Indication : division entière et modulo).

1. Écrivez l'algorithme (pas de boucle nécessaire ici, structure séquentielle + alternative).
2. Tracez avec 35 000 FCFA.

CORRIGÉ Exercice 1 : Gestion de notes de classe

Dictionnaire :	Nom	Type	Rôle	Init
	MaxEleves	Entier	Constante	50
	Sentinelle	Entier	Constante	-1
	Note	Entier	Entrée	-
	SommeNotes	Reel	Accumulateur	0.0
	NbEleves	Entier	Compteur	0
	NbAdmis	Entier	Compteur	0
	NoteMax	Entier	Travail (Max)	-1 (ou 0)
	Moyenne	Reel	Sortie	-

Algorithme (Version `Tant que` avec amorçage) :

```

Algorithme Notes_Classe
Variables
    Note, NbEleves, NbAdmis, NoteMax, MaxEleves : Entier
    SommeNotes, Moyenne : Reel
Constantes
    MaxEleves = 50
    Sentinelle = -1
Debut
    NbEleves <- 0
    SommeNotes <- 0.0
    NbAdmis <- 0
    NoteMax <- -1
  
```

```

Ecrire("Saisie des notes (note negative pour arreter, max 50 eleves)")
Lire(Note) // AMORÇAGE INDISPENSABLE

Tant que (Note >= 0) ET (NbEleves < MaxEleves) Faire
    SommeNotes <- SommeNotes + Note
    NbEleves <- NbEleves + 1
    Si Note >= 10 Alors
        NbAdmis <- NbAdmis + 1
    Fin Si
    Si Note > NoteMax Alors
        NoteMax <- Note
    Fin Si
    Lire(Note) // LECTURE DE SUIVANT INDISPENSABLE
Fin Tant que

Si NbEleves > 0 Alors
    Moyenne <- SommeNotes / NbEleves
    Ecrire("Moyenne : ", Moyenne)
    Ecrire("Note max : ", NoteMax)
    Ecrire("Admis : ", NbAdmis, " sur ", NbEleves)
Sinon
    Ecrire("Aucune note saisie.")
Fin Si
Fin

```

Trace (12, 8, 15, -1) : NbEleves=3, Somme=35, Moyenne=11.67, Max=15, Admis=2 (12, 15).
Correct.

CORRIGÉ Exercice 2 : Distributeur de billets

```

Algorithme DAB_Billets
Variables
    Montant, Nb10k, Nb5k, Reste : Entier
Debut
    Ecrire("Montant souhaite (multiple de 5000, max 200000) : ")
    Lire(Montant)
    Si (Montant % 5000 = 0) ET (Montant <= 200000) ET (Montant > 0) Alors
        Nb10k <- Montant / 10000 // Division entière
        Reste <- Montant % 10000 // Modulo (reste)
        Nb5k <- Reste / 5000
        Ecrire("Billets 10000 : ", Nb10k)
        Ecrire("Billets 5000 : ", Nb5k)
    Sinon
        Ecrire("Montant invalide.")
    Fin Si
Fin

```

Trace (35000) : Nb10k = 35000 / 10000 = 3. Reste = 35000

9 MÉTHODES ET EXERCICES RÉSOLUS

10 ÉCRITURE D'ALGORITHMES SÉQUENTIELS ET STRUCTURES DE CONTRÔLE

10.1 Notions fondamentales : variable, constante, instruction, algorithme

DÉFINITION Variable

Une **variable** est une zone de mémoire identifiée par un **nom** (identificateur), capable de contenir une **valeur** qui peut changer au cours de l'exécution de l'algorithme. Elle est caractérisée par son **type** (Entier, Réel, Chaîne, Caractère, Booléen) qui détermine l'ensemble des valeurs admises et les opérations autorisées.

DÉFINITION Constante

Une **constante** est une zone de mémoire identifiée par un nom, dont la valeur est fixée une fois pour toutes à la déclaration et ne peut jamais être modifiée pendant l'exécution. Elle améliore la lisibilité et la maintenance (ex. : TAUX_TVA au lieu de 0,1925 dispersé dans le code).

DÉFINITION Instruction

Une **instruction** est un ordre élémentaire donné à la machine pour effectuer une action précise : lire une donnée, écrire un résultat, calculer (affectation), choisir (alternative), répéter (boucle). Un algorithme est une suite finie et ordonnée d'instructions.

DÉFINITION Algorithme séquentiel

Un **algorithme séquentiel** est un algorithme dont les instructions s'exécutent les unes après les autres, dans l'ordre d'écriture, sans structure de choix (alternative) ni de répétition (itérative). C'est la forme la plus simple : Entrées → Traitements → Sorties.

PROPRIÉTÉ Règles de nommage des identificateurs (variables, constantes, algorithmes)

- Commence par une lettre (A–Z, a–z) ; pas de chiffre en première position.
- Composé de lettres, de chiffres (0–9) et éventuellement du trait de soulignement _.
- Pas d'espace, pas d'accent, pas de caractère spécial (@, #, \$, etc.).
- La **casse** (majuscules/minuscules) est distinctive : `prixHT` ≠ `prixht`.
- Convention recommandée : `camelCase` pour les variables (`prixUnitaire`, `nbAdmis`), `MAJUSCULES_SNAKE_CASE` pour les constantes (`TAUX_TVA`, `NB_MOIS`).

PROPRIÉTÉ Types de base en algorithmique (niveau Première TI)

Type	Ensemble des valeurs	Exemples littéraux
Entier	$\mathbb{Z}$ (relatifs sans virgule)	0, -5, 42, 30
Réel	$\mathbb{R}$ (nombre à virgule flottante)	3, 14, -0,5, 12,0, 1,925E-2
Chaîne	Suite finie de caractères (texte)	"Yaoundé", 'A', "123"
Caractère	Un unique caractère	'M', '@', '7'
Booléen	{Vrai, Faux}	Vrai, Faux

Attention : En algorithmique, on écrit `Reel` (sans accent) et `Chaîne` (sans accent) comme mots-clés de type.

10.2 Méthode 1 : Déclarer et utiliser les variables et les constantes

MÉTHODE Bien déclarer ses variables et constantes

1. Repérer les données du problème. Pour chaque grandeur qui varie pendant l'exécution (un prix, un compteur, une note), prévoir une **variable**. Pour chaque grandeur fixe connue à l'avance (un taux de TVA, le nombre de mois), prévoir une **constante**.
2. Choisir un nom significatif : `prixUnitaire` et non `p`. Un nom commence par une lettre, sans espace ni accent.
3. Choisir le bon type : `Entier` (nombre sans décimale), `Réel` (nombre à virgule), `Chaîne` (texte), `Caractère` (une lettre), `Booléen` (Vrai/Faux).
4. Regrouper toutes les déclarations dans la zone `Variables` (et `Constantes`) placée entre

l'en-tête `Algorithme` et le mot `Debut`.

5. Initialiser chaque variable avant sa première utilisation (souvent à 0 pour un total ou un compteur). Pour une constante, l'initialisation est obligatoire à la déclaration.

Réflexe Bac : une variable utilisée mais non déclarée, ou déclarée avec un mauvais type (ex. `Entier` pour une moyenne 11,33), coûte systématiquement des points.

EXERCICE RÉSOLU 8 Déclarations pour la facture d'une boutique

Énoncé. Une boutique de quartier à Yaoundé vend des articles soumis à une TVA fixe de 19,25 %. On veut écrire un algorithme qui calcule le montant TTC d'un achat à partir du prix hors taxe saisi. Donner la zone de déclaration complète (constantes et variables) de cet algorithme, en justifiant le type de chaque identificateur.

SOLUTION

Solution.

1. Analyse des grandeurs. Le taux de TVA (19,25 %) est fixé par la loi : c'est une **constante**. Le prix hors taxe est saisi par l'utilisateur et le montant TTC est calculé : ce sont des **variables**.
2. Choix des types. Un prix peut comporter des centimes (ou être exprimé en FCFA avec décimales dans le calcul) : on choisit le type `Réel` pour les trois grandeurs. Le taux s'écrit $19,25/100 = 0,1925$, valeur décimale, donc `Réel` également.
3. Rédaction de la zone de déclaration.

```
Algorithme FactureTTC
Constantes
    TAUX_TVA : Réel = 0,1925
Variables
    prixHT, montantTTC : Réel
Debut
    ...
Fin
```

Justifications. `TAUX_TVA` est déclarée en `Constantes` car sa valeur ne change jamais pendant l'exécution ; le symbole `=` fixe sa valeur une fois pour toutes (définition, pas affectation). `prixHT` et `montantTTC` sont des variables de type `Réel` car elles représentent des sommes d'argent issues d'un calcul pouvant produire des décimales. Conclusion. La zone de déclaration comporte une constante réelle `TAUX_TVA` initialisée à 0,1925 et deux variables réelles `prixHT` et `montantTTC`.

10.3 Instructions de base : lecture, écriture, affectation

PROPRIÉTÉ Syntaxe des trois instructions fondamentales

- **Lecture (entrée) :** Lire(identificateur_variable)
Lit une valeur au clavier et l'affecte à la variable (qui doit exister et être de type compatible). On ne lit jamais une constante.
- **Affectation (calcul) :** variable <- expression
Évalue expression (constante, variable, opération arithmétique ou booléenne) puis range le résultat dans variable. Le sens est de droite vers la gauche. L'ancien contenu de variable est écrasé.
- **Écriture (sortie) :** Ecrire(liste_parametres)
Affiche à l'écran les éléments de la liste, séparés par des virgules. Une Chaîne entre guillemets " . . . " s'affiche telle quelle ; un identificateur sans guillemets affiche la valeur de la variable.

10.4 Méthode 2 : Utiliser les instructions de base (lecture, écriture, affectation)**MÉTHODE** Écrire un algorithme séquentiel correct

1. Identifier l'ordre des opérations : un algorithme **séquentiel** exécute les instructions dans l'ordre où elles sont écrites, sans choix ni répétition.
2. Lecture (entrée) : Lire(variable) pour recevoir une donnée de l'utilisateur. On ne lit que des variables, jamais des constantes.
3. Affectation : variable <- expression. On évalue d'abord l'expression à droite, puis on range le résultat dans la variable à gauche. Le sens est de droite vers la gauche.
4. Écriture (sortie) : Ecrire("texte", variable) pour afficher un résultat. Tout ce qui est entre guillemets est affiché tel quel ; sans guillemets, c'est la valeur de la variable qui est affichée.
5. Vérifier la trace : pour tester, dresser un tableau donnant la valeur de chaque variable après chaque ligne.

Formule à retenir : après `a <- b` puis `b <- c`, la variable `a` garde l'ancienne valeur de `b` : l'affectation copie une valeur, elle ne lie pas les variables.

EXERCICE RÉSOLU 9 Algorithme séquentiel et trace d'exécution

Énoncé. (a) Écrire un algorithme complet qui lit le prix hors taxe d'un article, calcule et affiche son prix TTC avec une TVA de 19,25 %. (b) Donner la trace d'exécution pour un prix

HT de 10000 FCFA.

SOLUTION

Solution. (a) On applique la démarche : une lecture, un calcul par affectation, une écriture.

```

Algorithme PrixTTC
Constantes
    TAUX_TVA : Reel = 0,1925
Variables
    prixHT, prixTTC : Reel
Debut
    Ecrire("Entrez le prix hors taxe : ")
    Lire(prixHT)
    prixTTC <- prixHT + prixHT * TAUX_TVA
    Ecrire("Le prix TTC est : ", prixTTC, " FCFA")
Fin
  
```

La multiplication est prioritaire sur l'addition : $\text{prixHT} * \text{TAUX_TVA}$ est calculé en premier, ce qui correspond bien au montant de la TVA. (b) Trace d'exécution pour $\text{prixHT} = 10000$:

Instruction	prixHT	prixTTC
Lire(prixHT)	10000	indéfini
$\text{prixTTC} \leftarrow \text{prixHT} + \text{prixHT} * \text{TAUX_TVA}$	10000	$10000 + 10000 * 0,1925 = 11925$
Ecrire(...)	10000	11925

Calcul complet : montant de la TVA = $10000 \times 0,1925 = 1925$ FCFA ; prix TTC = $10000 + 1925 = 11925$ FCFA. Conclusion. L'algorithme affiche : « Le prix TTC est : 11925 FCFA ».

10.5 Structures de contrôle : panorama et structures alternatives

DÉFINITION Structure de contrôle

Une **structure de contrôle** est une instruction composée qui modifie le flux d'exécution séquentiel par défaut. On distingue deux familles :

- Structures alternatives (choix) : Si ... Alors ... [Sinon ...] FinSi — exécution conditionnelle.
- Structures itératives (répétition) : Pour ... Faire ... FinPour, TantQue ... Faire ... FinTantQue, Repeter ... Jusqu'a — exécution répétée.

10.6 Méthode 3 : Utiliser les structures alternatives (Si, Si ... Sinon)

MÉTHODE Choisir et rédiger une structure alternative

1. Repérer le besoin de choix : dès que l'énoncé contient « si ... alors ... », « selon que ... », « dans le cas où ... », il faut une **structure alternative**.
2. Formuler la condition sous forme d'expression booléenne (comparaisons =, <, >, <=, >=, <>, combinées par ET, OU, NON).
3. Choisir la forme adaptée :
 - Si condition Alors ... FinSi : un seul cas à traiter, rien à faire sinon ;
 - Si condition Alors ... Sinon ... FinSi : deux cas exclusifs ;
 - Si ... SinonSi ... Sinon ... FinSi : plusieurs cas (ex. mention selon la note).
4. Indenter les instructions à l'intérieur de chaque branche et ne jamais oublier le FinSi.
5. Vérifier que les cas se recouvrent sans se chevaucher (ex. `note >= 10` puis `Sinon` couvre tout $\mathbb{R}$ sans trou).

EXERCICE RÉSOLU 10 Décision d'admission au Probatoire

Énoncé. Écrire un algorithme qui lit la moyenne générale d'un candidat au Probatoire TI (sur 20) et affiche « Admis » si la moyenne est supérieure ou égale à 10, « Refusé » sinon. On affichera de plus « avec mention Bien » si la moyenne est supérieure ou égale à 14.

SOLUTION

Solution. Il y a deux décisions : la décision principale (admis/refusé) à deux cas exclusifs, et une décision secondaire (mention) qui ne concerne que les admis : on la place dans la branche Alors.

```
Algorithme DecisionProbatoire
Variables
    moyenne : Reel
Debut
    Ecrire("Entrez la moyenne du candidat : ")
    Lire(moyenne)
    Si moyenne >= 10 Alors
        Ecrire("Admis")
        Si moyenne >= 14 Alors
            Ecrire("avec mention Bien")
        FinSi
    Sinon
        Ecrire("Refuse")
    FinSi
Fin
```

Vérification par cas. Pour `moyenne = 15,5` : la condition `15,5 ≥ 10` est vraie, on affiche « Ad-

mis » ; puis $15,5 \geq 14$ est vraie, on affiche « avec mention Bien ». Pour $\text{moyenne} = 12$: « Admis » seulement, car $12 \geq 14$ est fausse et il n'y a pas de branche *Si non* sur la mention. Pour $\text{moyenne} = 8$: la première condition est fausse, la branche *Si non* affiche « Refusé ». Conclusion. L'algorithme traite correctement les trois situations : refusé, admis simple, admis avec mention, grâce à une alternative imbriquée dans la branche *Alors*.

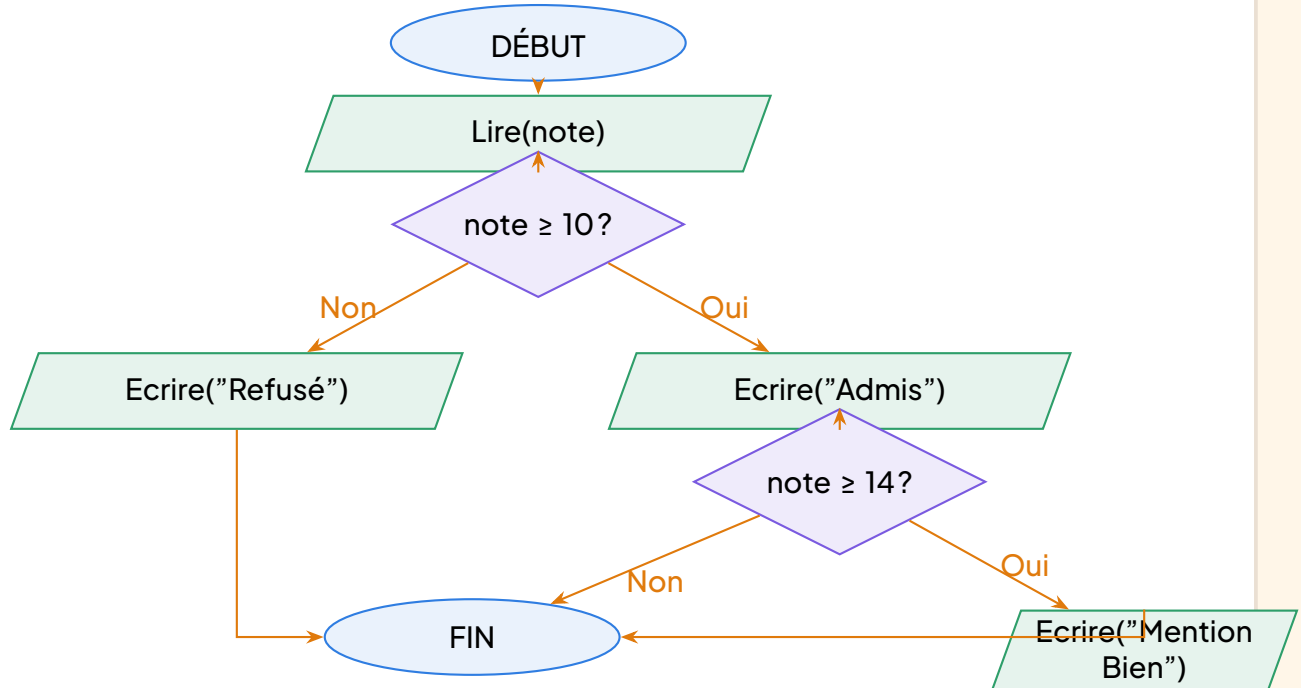
10.7 Algorithmes : représentation graphique des algorithmes

DÉFINITION Algorithme (Diagramme de flux)

Un **algorithme** (ou flowchart) est une représentation graphique normalisée d'un algorithme. Il utilise des symboles géométriques normalisés (ISO 5807) reliés par des flèches orientées indiquant le sens d'exécution.

PROPRIÉTÉ Symboles principaux de l'algorithme (niveau Première TI)

Symbole	Forme géométrique	Rôle / Instruction correspondante
Début / Fin	Ovale (ou rectangle à bords arrondis)	Point d'entrée et de sortie unique de l'algorithme.
Traitement / Affectation	Rectangle	Calcul, affectation ($x \leftarrow y + 1$), initialisation.
Entrée / Sortie	Parallélogramme	Lire(x) (entrée), Ecrire(...) (sortie).
Test / Décision	Losange (losange)	Condition booléenne (Si $x > 0$). Deux sorties : Oui (Vrai) et Non (Faux).
Connexion / Jonction	Cercle (petit)	Liaison entre parties distantes (évite les flèches croisées).
Flux	Flèche pleine	Sens d'exécution (haut $\rightarrow$ bas, gauche $\rightarrow$ droite).



Algorithme de l'algorithme DecisionProbatoire (alternative imbriquée).

ATTENTION Pièges classiques de l'algorithme

1. Oublier une sortie sur un losange : tout test a deux issues (Oui/Non ou Vrai/Faux). Les deux doivent être raccordées.
2. Sens des flèches : toujours du haut vers le bas (ou gauche vers droite). Pas de flèche remontante hors structure itérative.
3. Confondre rectangle et losange : le losange ne contient que la condition ; les actions (calculs, affichages) vont dans des rectangles ou parallélogrammes.
4. Plusieurs "Début" ou "Fin" : un algorithme principal n'a qu'un seul point d'entrée et un seul point de sortie.

10.8 Initialisation, incrémentation et décrémentation

DÉFINITION Initialisation

L'**initialisation** consiste à donner une valeur de départ à une variable avant sa première utilisation effective (souvent avant une boucle). Exemples : `somme <- 0`, `produit <- 1`, `compteur <- 0`, `max <- -INFINI`.

DÉFINITION Incrémentation / Décrémentation

L'**incrémentation** augmente une variable d'un pas (souvent 1) : `var <- var + pas`. La **décrémentation** la diminue : `var <- var - pas`. Ce ne sont pas des équations mathématiques mais des affectations successives : la nouvelle valeur remplace l'ancienne.

10.9 Méthode 4 : Maîtriser compteurs et accumulateurs**MÉTHODE** Maîtriser compteurs et accumulateurs

1. Initialisation : donner une valeur de départ avant la boucle. Un compteur démarre à 0 (`compteur <- 0`), un accumulateur de somme à 0 (`somme <- 0`), un accumulateur de produit à 1 (`produit <- 1`).
2. Incrémentation : augmenter une variable d'une valeur fixe : `compteur <- compteur + 1` (pas de 1) ou `compteur <- compteur + pas` (pas quelconque).
3. Décrémentation : diminuer une variable : `n <- n - 1`.
4. Comprendre l'écriture : `compteur <- compteur + 1` se lit « la nouvelle valeur de compteur est l'ancienne plus 1 ». Ce n'est pas une équation mathématique.
5. Placer l'incrémentation au bon endroit : dans une boucle, elle doit être exécutée à chaque passage, sinon la boucle ne progresse pas (risque de boucle infinie).

EXERCICE RÉSOLU 11 Comptage des élèves admis d'une classe

Énoncé. On saisit les moyennes des 30 élèves d'une classe de Première TI. Compléter le fragment d'algorithme suivant pour qu'il compte et affiche le nombre d'élèves admis (moyenne ≥ 10) :

```
Variables
  i, nbAdmis : Entier
  moyenne : Reel
Debut
  ..... // instruction 1
  Pour i Allant De 1 A 30 Faire
    Lire(moyenne)
    Si moyenne >= 10 Alors
      ..... // instruction 2
    FinSi
  FinPour
  Ecrire("Nombre d'admis : ", nbAdmis)
Fin
```

Justifier chaque instruction ajoutée.

SOLUTION

Solution. Instruction 1 (initialisation). Avant la boucle, aucun élève n'a encore été traité : le compteur d'admis doit partir de zéro :

```
nbAdmis <- 0
```

Sans cette initialisation, `nbAdmis` contiendrait une valeur indéfinie et le calcul `nbAdmis + 1` serait faux dès le premier admis. Instruction 2 (incrémentation). Chaque fois qu'une moyenne est supérieure ou égale à 10, on a trouvé un admis de plus :

```
nbAdmis <- nbAdmis + 1
```

Cette affectation prend l'ancienne valeur de `nbAdmis`, lui ajoute 1, et range le résultat dans `nbAdmis`. Remarque. La variable `i` est initialisée et incrémentée automatiquement par la boucle `Pour` : il ne faut ni l'initialiser ni écrire `i <- i + 1` dans le corps de la boucle. Conclusion. Le fragment complété est : `nbAdmis <- 0` avant la boucle (initialisation) et `nbAdmis <- nbAdmis + 1` dans la branche `Alors` (incrémentation conditionnelle). Si 18 élèves ont une moyenne ≥ 10 , l'algorithme affiche « Nombre d'admis : 18 ».

10.10 Structures itératives : Pour, Tant que, Répéter

PROPRIÉTÉ**Comparatif des trois boucles**

Critère	Pour ... Faire	TantQue ... Faire	Repeter ... Jusqu'à
Nombre de tours	Connu à l'avance (n fixé)	Inconnu (dépend condition)	Inconnu (dépend condition)
Moment du test	Implicite (avant chaque tour)	Avant chaque tour (pré-test)	Après chaque tour (post-test)
Exécution minimale	0 tour possible (si $\text{debut} > \text{fin}$)	0 tour possible (si condition fausse au départ)	1 tour garanti (corps exécuté avant test)
Gestion compteur	Automatique (initialisation, test, incrémentation)	Manuelle (programmeur gère tout)	Manuelle (programmeur gère tout)
Condition d'arrêt	Atteinte de <code>fin</code>	Condition FAUSSE (on continue tant que VRAI)	Condition VRAIE (on s'arrête quand VRAI)

10.11 Méthode 5 : Employer la boucle Pour à bon escient

MÉTHODE Employer la boucle Pour à bon escient

1. Condition d'emploi : on utilise `Pour` quand le nombre de répétitions est connu à l'avance (30 élèves, 12 mois, n valeurs lues au préalable).
2. Syntaxe : `Pour compteur Allant De debut A fin [Pas p] Faire ... FinPour`. Le compteur est automatiquement initialisé à `debut` puis incrémenté de `p` (1 par défaut) à chaque tour.
3. Nombre de tours : de `debut` à `fin` avec un pas de 1, la boucle s'exécute exactement $\text{fin} - \text{debut} + 1$ fois.
4. Ne jamais modifier le compteur à l'intérieur du corps de la boucle : c'est la boucle qui le gère.
5. Initialiser les accumulateurs avant la boucle, jamais dedans (sinon ils sont remis à zéro à chaque tour).

EXERCICE RÉSOLU 12 Somme des 12 mensualités d'une tontine

Énoncé. Dans une tontine (association d'épargne rotative très répandue au Cameroun), chaque membre cotise chaque mois. Écrire un algorithme qui lit les 12 cotisations mensuelles d'un membre et affiche le total annuel épargné. Combien de fois la boucle s'exécute-t-elle ? Justifier.

SOLUTION

Solution. Le nombre de répétitions est connu : 12 mois. On utilise donc une boucle `Pour`. L'accumulateur `total` est initialisé à 0 avant la boucle.

```

Algorithme TontineAnnuelle
Variables
    mois : Entier
    cotisation, total : Reel
Debut
    total <- 0
    Pour mois Allant De 1 A 12 Faire
        Ecrire("Cotisation du mois ", mois, " : ")
        Lire(cotisation)
        total <- total + cotisation
    FinPour
    Ecrire("Total annuel epargne : ", total, " FCFA")
Fin
  
```

Nombre de tours. Le compteur `mois` va de 1 à 12 avec un pas de 1 : la boucle s'exécute $12 - 1 + 1 = 12$ fois, une fois par mois. Vérification sur un exemple. Si chaque cotisation vaut

5000 FCFA, après le tour k on a $\text{total} = 5000 \times k$; après le 12^{ème} tour, $\text{total} = 12 \times 5000 = 60000$ FCFA. Conclusion. L'algorithme affiche le total des 12 cotisations, par exemple « Total annuel épargne : 60000 FCFA »; la boucle `Pour` est le bon choix car le nombre d'itérations (12) est connu à l'avance.

10.12 Méthode 6 : Choisir entre Tant que et Répéter

MÉTHODE Choisir entre Tant que et Répéter

1. Condition d'emploi : on utilise ces boucles quand le nombre de répétitions est inconnu : on répète tant qu'une condition est vraie (ou jusqu'à ce qu'elle le devienne).
2. Tant que condition Faire ... FinTantQue : le test est fait avant chaque tour. Le corps peut ne jamais s'exécuter (0 tour) si la condition est fausse dès le départ. Nécessite un amorçage (initialisation des variables de la condition) avant la boucle.
3. Répéter ... Jusqu'à condition : le test est fait après chaque tour. Le corps s'exécute donc au moins une fois. Pas d'amorçage nécessaire pour la première entrée. Attention : la condition écrite est la **condition d'arrêt** (on s'arrête quand elle est vraie), c'est la négation logique de celle du `Tant que`.
4. Garantir la terminaison : au moins une instruction du corps doit modifier une variable de la condition, sinon la boucle est infinie.
5. Équivalence : `Tant que C Faire ... FinTantQue` $\equiv$ `Répéter ... Jusqu'à NON C`, à ceci près que `Répéter` exécute toujours le corps une première fois.

EXERCICE RÉSOLU 13 Saisie contrôlée d'une note

Énoncé. (a) Écrire un algorithme qui demande une note sur 20 et la redemande tant qu'elle n'est pas comprise entre 0 et 20 (bornes incluses), d'abord avec un `Tant que`, puis avec un `Répéter ... Jusqu'à`. (b) Expliquer pourquoi le `Répéter` est ici plus naturel.

SOLUTION

Solution. (a) Version `Tant que`. Le test étant fait avant le corps, il faut une première lecture avant la boucle (amorçage) :

```
Algorithme SaisieNoteTantQue
Variables
    note : Reel
Debut
    Ecrire("Entrez une note entre 0 et 20 : ")
```

```

Lire(note)
Tant Que (note < 0) OU (note > 20) Faire
    Ecrire("Erreur ! Recommencez : ")
    Lire(note)
FinTantQue
Ecrire("Note validee : ", note)
Fin

```

Version Répéter. Le test étant fait après le corps, une seule lecture suffit ; la condition d'arrêt est la négation de la condition de continuation :

```

Algorithme SaisieNoteRepeter
Variables
    note : Reel
Debut
    Repeter
        Ecrire("Entrez une note entre 0 et 20 : ")
        Lire(note)
    Jusqu'à (note >= 0) ET (note <= 20)
        Ecrire("Note validee : ", note)
Fin

```

Vérifions la négation : la continuation du `Tant que` est $(note < 0) \text{ OU } (note > 20)$; sa négation, par les lois de De Morgan, est $(note \geq 0) \text{ ET } (note \leq 20)$: c'est exactement la condition d'arrêt du `Répéter`. (b) Justification. Ici, on doit toujours saisir la note au moins une fois : le `Répéter` est plus naturel car il évite de dupliquer les instructions `Ecrire/Lire` avant la boucle. Le code est plus court et moins sujet aux oublis. Conclusion. Les deux algorithmes sont équivalents ; pour une saisie avec validation, la boucle `Répéter ... Jusqu'à` est la structure la plus adaptée.

10.13 Synthèse : écrire un algorithme complet et structuré

10.14 Méthode 7 : Conduire la résolution d'un problème type Bac

MÉTHODE Conduire la résolution d'un problème type Bac

1. Analyser l'énoncé : souligner les données d'entrée, les résultats attendus, les traitements (calculs, choix, répétitions).
2. Déclarer toutes les constantes et variables avec leurs types (zone `Constantes` puis `Variables`).
3. Structurer le corps en trois blocs logiques (séparés par des lignes de commentaires) : Entrées (lectures, éventuellement contrôlées par `Répéter`), Traitements (calculs, alternatives `Si`, boucles `Pour/TantQue`), Sorties (écritures finales).
4. Choisir la bonne boucle : nombre de tours connu $\Rightarrow$ `Pour` ; inconnu avec test avant $\Rightarrow$

Tant que ; inconnu avec au moins un passage $\Rightarrow$ Répéter.

5. Tester mentalement avec un jeu de valeurs simple (trace d'exécution) avant de rédiger la version finale.
6. Soigner la présentation : indentation rigoureuse (2 ou 4 espaces), mots-clés de fin (FinSi, FinPour, FinTantQue) alignés sur le début de structure, commentaires brefs et utiles.

EXERCICE RÉSOLU 14 Sujet de synthèse : bulletin d'une classe

Énoncé. Un enseignant de Première TI veut un algorithme qui : lit le nombre n d'élèves de la classe ($n \geq 1$) ; lit ensuite les n moyennes (chacune entre 0 et 20, saisie contrôlée) ; calcule et affiche la moyenne de la classe ainsi que le nombre d'élèves admis (moyenne ≥ 10). Écrire cet algorithme complet.

SOLUTION

Solution. Analyse. Entrées : n puis n moyennes. Traitements : boucle de n tours (nombre connu une fois n lu $\Rightarrow$ Pour), avec à l'intérieur une saisie contrôlée ($\Rightarrow$ Répéter) et un comptage conditionnel ($\Rightarrow$ Si). Sorties : moyenne de classe et nombre d'admis.

```

Algorithme BulletinClasse
Variables
    n, i, nbAdmis : Entier
    moy, somme, moyClasse : Reel
Debut
    // Entree du nombre d'eleves (controle)
    Repeter
        Ecrire("Nombre d'eleves (>= 1) : ")
        Lire(n)
    Jusqu'à n >= 1
    // Initialisations
    somme <- 0
    nbAdmis <- 0
    // Lecture des moyennes et traitements
    Pour i Allant De 1 A n Faire
        Repeter
            Ecrire("Moyenne de l'eleve ", i, " : ")
            Lire(moy)
            Jusqu'à (moy >= 0) ET (moy <= 20)
            somme <- somme + moy
            Si moy >= 10 Alors
                nbAdmis <- nbAdmis + 1
            FinSi
        FinPour
    // Calcul et affichage des resultats
    moyClasse <- somme / n
    Ecrire("Moyenne de la classe : ", moyClasse)
    Ecrire("Nombre d'admis : ", nbAdmis)
Fin

```

LESAVAIS-TU ? Al-Khwarizmi et l'origine du mot "algorithme"

Le mot **algorithme** vient du nom du mathématicien perse Muhammad ibn Musa al-Khwarizmi (vers 780–850), qui travailla à la Maison de la Sagesse de Bagdad. Son ouvrage *Al-Kitab al-mukhtasar fi hisab al-jabr wal-muqabala* (« Le livre abrégé sur le calcul par la restauration et la comparaison ») a donné le mot « algèbre » (al-jabr) et son nom, latinisé en *Algorismi*, a donné « algorithme ». Au Cameroun, la logique algorithmique est au cœur des concours d'entrée aux grandes écoles (ENSET, Polytechnique) et du Probatoire TI : la maîtriser, c'est s'ouvrir les portes de l'ingénierie logicielle et de la data science.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Variable non déclarée ou mal typée. Utiliser `moyenne` sans l'avoir déclarée, ou la déclarer en `Entier` alors qu'une moyenne vaut 11,33 : déclarez toutes les variables, avec le type `Réel` dès qu'une division intervient.
2. Inverser le sens de l'affectation. Écrire `prixHT + prixHT * TAUX <- prixTTC` est faux : la variable qui reçoit le résultat est toujours à gauche du symbole `<-`.
3. Oublier l'initialisation d'un compteur ou d'une somme. Sans `somme <- 0` avant la boucle, le premier calcul `somme <- somme + moy` part d'une valeur indéfinie : le résultat est faux. De même, un produit s'initialise à 1, pas à 0.
4. Confondre les conditions du Tant que et du Répéter. La condition du `Jusqu'à` est la négation de celle du `Tant que` : pour une note valide, on écrit `Tant que (note < 0) OU (note > 20)` mais `Jusqu'à (note >= 0) ET (note <= 20)`. Inverser `ET` et `OU` ici est l'erreur la plus fréquente (lois de De Morgan).
5. Créer une boucle infinie. Oublier l'incrémentation dans un `Tant que` (ex. ne jamais faire évoluer `i` dans `Tant que i <= n`), ou modifier le compteur d'un `Pour` dans le corps de la boucle : la condition d'arrêt n'est jamais atteinte. Vérifiez toujours qu'au moins une instruction du corps fait progresser la condition vers l'arrêt.

EXERCICE 9 Entraînement : Factorielle et recherche

Énoncé. (a) Écrire un algorithme qui lit un entier $n \geq 0$ (saisie contrôlée) et affiche $n!$ (factorielle). Rappel : $0! = 1$, $n! = n \times (n - 1) \times \dots \times 1$.
 (b) Modifier l'algorithme pour qu'il affiche aussi la plus grande factorielle $\leq 10^6$ (un million) et la valeur de n correspondante.

CORRIGÉ Exercice : Factorielle et recherche

(a) Factorielle simple.

```

Algorithme Factorielle
Variables
  n, i : Entier
  fact : Reel // Reel car factorielle grandit vite (dépasse Entier 32 bits des 13!)
Debut
  Repeter
    Ecrire("Entrez n (entier >= 0) : ")
    Lire(n)
  Jusqu'à n >= 0
  fact <- 1
  Pour i Allant De 2 A n Faire
    fact <- fact * i
  FinPour
  Ecrire(n, " ! = ", fact)
Fin
  
```

Justification : Boucle Pour car n connu. Initialisation $fact \leftarrow 1$ (élément neutre du produit). La boucle commence à 2 car multiplier par 1 ne change rien. Type Reel pour $fact$ pour éviter débordement rapide (en C on utiliserait long long ou double).

(b) Recherche du seuil.

```

Algorithme FactSeuil
Variables
  n, i : Entier
  fact, factPrec : Reel
Debut
  fact <- 1
  n <- 0
  Repeter
    n <- n + 1
    factPrec <- fact
    fact <- fact * n
  Jusqu'à fact > 1000000
  // A la sortie : fact > 1 000 000, factPrec <= 1 000 000 et correspond a (n-1)!
  Ecrire("Plus grande factorielle <= 1 000 000 : ", (n-1), " ! = ", factPrec)
Fin
  
```

Analyse : On utilise Répéter car le nombre de tours est inconnu. On mémorise la factorielle précédente ($factPrec$) avant de la dépasser. La condition d'arrêt $fact > 1000000$ garantit qu'à la sortie, $factPrec$ est la réponse cherchée pour $n - 1$. Résultat : $9! = 362\,880 \leq 10^6$ et

$10! = 3\,628\,800 > 10^6$. L'algorithme affiche : « Plus grande factorielle $\leq 1\,000\,000$: $9! = 362880$ ».

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 10 QCM : notions de base

Pour chaque question, une seule réponse est correcte. Entoure la lettre correspondante.

1. Une **variable** est :

- a) une valeur qui ne change jamais pendant l'exécution d'un algorithme ;
- b) un emplacement mémoire nommé dont le contenu peut changer pendant l'exécution ;
- c) une instruction qui affiche un résultat à l'écran.

2. L'instruction `Lire(n)` a pour effet :

- a) d'afficher la valeur de `n` à l'écran ;
- b) d'affecter à `n` une valeur saisie au clavier ;
- c) de déclarer la variable `n`.

3. Après les instructions `a ← 5` puis `a ← a + 3`, la variable `a` contient :

- a) 5 ; b) 3 ; c) 8.

4. La structure `Répéter ... Jusqu'à` exécute son corps de boucle :

- a) au moins une fois ;
- b) zéro ou plusieurs fois ;
- c) exactement une fois.

EXERCICE 11 Vrai ou faux — justifier

Réponds par Vrai ou Faux, puis justifie ta réponse en une ou deux phrases.

- 1. Une constante peut recevoir une nouvelle valeur par affectation en cours d'exécution.
- 2. Dans une boucle `Tant que`, si la condition est fausse dès le départ, le corps de la boucle n'est jamais exécuté.
- 3. L'instruction `i ← i + 1` est une décrémentation.
- 4. Une boucle dont la condition d'arrêt n'est jamais atteinte est appelée boucle infinie.
- 5. La structure `Si ... Sinon ... FinSi` permet d'exécuter deux traitements différents selon qu'une condition est vraie ou fausse.

EXERCICE 12 Définitions

Définis clairement, en deux phrases maximum chacune, les notions suivantes : **constante**, **instruction**, **algorithme séquentiel**, **structure alternative**, **structure itérative**. Pour chaque notion, donne un exemple d'écriture algorithmique.

EXERCICE 13 Calculs directs : valeurs des variables

On considère la suite d'instructions suivante :

```
a ← 4
b ← 10
a ← a + b
b ← a - b
a ← a - b
```

1. Donne la valeur de a et celle de b après chaque ligne (présente un tableau de suivi).
2. Que remarques-tu sur les valeurs finales de a et b ? Quel traitement cet algorithme réalise-t-il donc ?

11.2 Je m'entraîne**EXERCICE 14 Le plus grand de deux nombres**

1. Écris un algorithme qui lit deux nombres réels a et b , puis affiche le plus grand des deux.
2. Trace l'exécution de ton algorithme avec $a = 7$ et $b = 12$: indique la valeur de chaque variable et chaque message affiché.
3. Modifie ton algorithme pour qu'il affiche « Les deux nombres sont égaux » lorsque $a = b$.

EXERCICE 15 Table de multiplication de 7

1. Écris un algorithme qui calcule et affiche la table de multiplication de 7 (de 7×1 jusqu'à 7×10) à l'aide d'une boucle `Pour ... Faire`.
2. Identifie dans ton algorithme : l'instruction d'initialisation, l'instruction d'incrémenta-tion et la condition d'arrêt.
3. Transforme ton algorithme pour qu'il demande à l'utilisateur le nombre dont il veut la table, au lieu d'utiliser toujours 7.

EXERCICE 16 Admis ou ajourné, avec mentions

Au lycée de Nkongsamba, le professeur d'informatique veut automatiser l'affichage des décisions du conseil de classe.

1. Écris un algorithme qui lit une note sur 20 et affiche « Admis » si la note est supérieure ou égale à 10, « Ajourné » sinon.
2. Transforme cet algorithme pour qu'il affiche en plus la mention : « Passable » si $10 \leq \text{note} < 12$, « Assez bien » si $12 \leq \text{note} < 14$, « Bien » si $14 \leq \text{note} < 16$, « Très bien » si la note est supérieure ou égale à 16.
3. Ajoute un contrôle qui affiche « Note invalide » si la valeur saisie est inférieure à 0 ou supérieure à 20.

EXERCICE 17 Somme des entiers de 1 à 100

1. Écris un algorithme qui calcule et affiche la somme $S = 1 + 2 + 3 + \dots + 100$ à l'aide d'une boucle `Tant que ... Faire`.
2. Réécris ce même algorithme en utilisant une boucle `Répéter ... Jusqu'à`.
3. Quelle valeur affiche ton algorithme? Vérifie avec la formule $S = \frac{n(n+1)}{2}$ pour $n = 100$.
4. Explique ce qui se passerait si l'on oubliait l'instruction d'incrémentation dans la boucle `Tant que`.

11.3 Je me prépare au Probatoire**EXERCICE 18** Type Probatoire : chasse aux erreurs

L'élève Moussa a écrit l'algorithme ci-dessous, censé lire un entier n strictement positif et afficher la somme des entiers pairs compris entre 2 et n (inclus). Son algorithme contient cinq erreurs.

```

Algorithme SommePairs
Variables n, i, S : Entier
Debut
  Lire(n)
  i ← 2
  Tant que i < n Faire
    Si i mod 2 = 0 Alors
      S ← S + i
      i ← i + 1
    FinTantQue
  Ecrire("La somme vaut : ", S)

```

Fin

1. Recopie l'algorithme et souligne les cinq erreurs (pense à : initialisation des variables, condition d'arrêt de la boucle, structure du test, cas de la valeur saisie).
2. Écris l'algorithme corrigé et complet.
3. Trace l'exécution de l'algorithme corrigé pour $n = 10$: présente un tableau donnant les valeurs de i et S à chaque tour de boucle, puis le message affiché.
4. Explique pourquoi l'absence d'initialisation de S est une erreur grave, même si l'algorithme semble parfois « fonctionner ».

EXERCICE 19 Type Probatoire : trace d'exécution

On donne l'algorithme suivant :

```

Algorithme Mystere
Variables n, c, r : Entier
Debut
  Lire(n)
  c ← 0
  Tant que n > 0 Faire
    r ← n mod 10
    Si r mod 2 = 0 Alors
      c ← c + 1
    FinSi
    n ← n div 10
  FinTantQue
  Ecrire(c)
Fin

```

(`mod` donne le reste de la division entière et `div` le quotient.)

1. Trace l'exécution de cet algorithme pour $n = 5\,382$: présente un tableau donnant les valeurs de n , r et c à chaque tour de boucle.
2. Quelle valeur est affichée à la fin ?
3. Refais la trace pour $n = 7\,135$. Quelle valeur est affichée ?
4. Dédus des questions précédentes ce que calcule cet algorithme, et exprime-le par une phrase claire.
5. Que se passe-t-il si l'utilisateur saisit $n = 0$? Justifie.

EXERCICE 20 Type Probatoire : recensement dans un quartier de Yaoundé

Dans le cadre d'un recensement, un agent de la mairie de Yaoundé saisit les âges des ha-

bitants d'un quartier. La saisie s'arrête dès que l'agent entre une valeur négative (cette valeur négative n'est pas un âge et ne doit pas être comptée).

1. Écris un algorithme qui lit des âges jusqu'à la saisie d'une valeur négative, puis affiche le nombre de mineurs (âge strictement inférieur à 18 ans) et le nombre de majeurs (âge supérieur ou égal à 18 ans).
2. Précise pourquoi la boucle `Répéter ... Jusqu'à` est bien adaptée à ce problème, et donne la condition d'arrêt exacte.
3. Modifie ton algorithme pour qu'il affiche également l'âge le plus élevé saisi.
4. Trace l'exécution de l'algorithme de la question 1 avec la série de saisies suivante : 12, 45, 17, 18, 8, -1. Donne les valeurs des compteurs après chaque saisie et le résultat final affiché.
5. L'agent se trompe et saisit l'âge 253. Propose une amélioration de l'algorithme qui refuse les âges supérieurs à 130 en affichant « Âge invalide, ressaisissez » sans arrêter la saisie.

12 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : notions de base

1. Réponse b). Une variable est un emplacement mémoire nommé dont le contenu peut changer pendant l'exécution. La réponse a) décrit une **constante** ; la réponse c) décrit l'instruction `Ecrire`.
2. Réponse b). `Lire(n)` affecte à la variable `n` une valeur saisie au clavier par l'utilisateur. Afficher une valeur est le rôle de `Ecrire` ; la déclaration se fait dans la partie `Variables`.
3. Réponse c). Après `a <- 5`, `a` vaut 5. L'instruction `a <- a + 3` calcule d'abord $5 + 3 = 8$, puis range 8 dans `a`. La nouvelle valeur écrase l'ancienne : `a` contient donc 8.
4. Réponse a). Dans la structure `Répéter ... Jusqu'à`, le test est placé après le corps de boucle : celui-ci est donc exécuté au moins une fois, puis répété tant que la condition d'arrêt n'est pas vérifiée.

CORRIGÉ Exercice 2 — Vrai ou faux — justifier

1. Faux. Une constante reçoit sa valeur une seule fois, à la déclaration ; toute tentative d'affectation ultérieure est interdite. C'est précisément ce qui la distingue d'une variable.
2. Vrai. Dans `Tant que`, la condition est testée avant chaque exécution du corps. Si elle est fausse dès le premier test, le corps est exécuté zéro fois : c'est la différence essentielle avec `Répéter ... Jusqu'à`.
3. Faux. `i <- i + 1` augmente `i` de 1 : c'est une **incrément**. Une décrémentation serait `i <- i - 1`.
4. Vrai. Si la condition d'arrêt n'est jamais atteinte (par exemple parce qu'on a oublié l'incrément), la boucle ne se termine pas : c'est une **boucle infinie**, erreur grave qui bloque le programme.
5. Vrai. Si `... Sinon ... FinSi` offre exactement deux branches : le traitement du `Si` s'exécute si la condition est vraie, celui du `Sinon` si elle est fausse.

CORRIGÉ Exercice 3 — Définitions

Constante : emplacement mémoire nommé dont la valeur est fixée une fois pour toutes et ne peut pas être modifiée pendant l'exécution. Exemple : `Constante TVA = 0,1925`.
Instruction : ordre élémentaire donné à la machine, réalisant une action précise (lecture, écriture, affectation, calcul). Exemple : `a <- 5`.

Algorithme séquentiel : algorithme dont les instructions s'exécutent les unes après les autres, dans l'ordre d'écriture, sans test ni répétition. Exemple : Lire(prix) ; total <- prix * 2 ; Ecrire(total).

Structure alternative : structure de contrôle qui choisit le traitement à exécuter selon qu'une condition est vraie ou fausse. Exemple : Si note >= 10 Alors Ecrire("Admis") Sinon Ecrire("Ajourné") FinSi.

Structure itérative : structure de contrôle qui répète un bloc d'instructions tant qu'une condition est vérifiée (ou jusqu'à ce qu'une condition d'arrêt soit atteinte). Exemple : Pour i de 1 à 10 Faire Ecrire(i) FinPour.

CORRIGÉ Exercice 4 — Calculs directs : valeurs des variables

1. Tableau de suivi :

Instruction	a	b	Commentaire
a <- 4	4	—	initialisation de a
b <- 10	4	10	initialisation de b
a <- a + b	14	10	4 + 10 = 14
b <- a - b	14	4	14 - 10 = 4 (ancienne valeur de a)
a <- a - b	10	4	14 - 4 = 10

2. À la fin, a vaut 10 et b vaut 4 : les valeurs initiales ont été échangées. Cet algorithme réalise donc la **permutation** (l'échange) du contenu de deux variables, sans utiliser de variable temporaire.

CORRIGÉ Exercice 5 — Le plus grand de deux nombres

1. Algorithme :

```

Algorithme PlusGrand
Variables a, b, max : Reel
Debut
    Ecrire("Entrez deux nombres : ")
    Lire(a, b)
    Si a >= b Alors
        max <- a
    Sinon
        max <- b
    FinSi
    Ecrire("Le plus grand est : ", max)
Fin
  
```

2. Trace pour a = 7 et b = 12 : le test a >= b donne 7 ≥ 12, qui est faux ; on exécute donc la

branche Sinon : max $\leftarrow$ 12. Message affiché : « Le plus grand est : 12 ».

3. Prise en compte de l'égalité :

```
Algorithme PlusGrand2
Variables a, b : Reel
Debut
  Ecrire("Entrez deux nombres : ")
  Lire(a, b)
  Si a = b Alors
    Ecrire("Les deux nombres sont égaux")
  Sinon
    Si a > b Alors
      Ecrire("Le plus grand est : ", a)
    Sinon
      Ecrire("Le plus grand est : ", b)
    FinSi
  FinSi
Fin
```

CORRIGÉ Exercice 6 — Table de multiplication de 7

1. Algorithme :

```
Algorithme TableDe7
Variables i, p : Entier
Debut
  Pour i de 1 à 10 Faire
    p  $\leftarrow$  7 * i
    Ecrire("7 x ", i, " = ", p)
  FinPour
Fin
```

2. Identification :

- Initialisation : $i \leftarrow 1$ (effectuée automatiquement au début du Pour);
- Incrémentation : $i \leftarrow i + 1$ (automatique à chaque tour);
- Condition d'arrêt : la boucle s'arrête quand i dépasse 10 (dernier tour : $i = 10$).

3. Version avec nombre demandé :

```
Algorithme TableQuelconque
Variables n, i, p : Entier
Debut
  Ecrire("De quel nombre voulez-vous la table ? ")
  Lire(n)
  Pour i de 1 à 10 Faire
    p  $\leftarrow$  n * i
    Ecrire(n, " x ", i, " = ", p)
  FinPour
Fin
```

CORRIGÉ Exercice 7 — Admis ou ajourné, avec mentions

1. Décision simple :

```
Algorithme Decision
Variable note : Reel
Debut
    Ecrire("Entrez la note sur 20 : ")
    Lire(note)
    Si note >= 10 Alors
        Ecrire("Admis")
    Sinon
        Ecrire("Ajourné")
    FinSi
Fin
```

2. Avec mentions (tests imbriqués) :

```
Algorithme Mentions
Variable note : Reel
Debut
    Ecrire("Entrez la note sur 20 : ")
    Lire(note)
    Si note < 10 Alors
        Ecrire("Ajourné")
    Sinon
        Ecrire("Admis")
        Si note < 12 Alors
            Ecrire("Mention Passable")
        Sinon Si note < 14 Alors
            Ecrire("Mention Assez bien")
        Sinon Si note < 16 Alors
            Ecrire("Mention Bien")
        Sinon
            Ecrire("Mention Tres bien")
        FinSi
    FinSi
Fin
```

3. Contrôle de validité : on encadre tout le traitement par un premier test :

```
Lire(note)
Si (note < 0) ou (note > 20) Alors
    Ecrire("Note invalide")
Sinon
    ... traitement de la question 2 ...
FinSi
```

Le contrôle de saisie doit toujours être placé avant tout traitement sur la donnée.

CORRIGÉ Exercice 8 — Somme des entiers de 1 à 100

1. Avec Tant que :

```

Algorithme Somme100
Variables i, S : Entier
Debut
    S <- 0
    i <- 1
    Tant que i <= 100 Faire
        S <- S + i
        i <- i + 1
    FinTantQue
    Ecrire("La somme vaut : ", S)
Fin

```

2. Avec Répéter ... Jusqu'à :

```

Algorithme Somme100bis
Variables i, S : Entier
Debut
    S <- 0
    i <- 1
    Repeter
        S <- S + i
        i <- i + 1
    Jusqu'à i > 100
    Ecrire("La somme vaut : ", S)
Fin

```

Remarque : la condition du Jusqu'à est la négation de celle du Tant que ($i > 100$ au lieu de $i \leq 100$).

3. Vérification : avec la formule $S = \frac{n(n+1)}{2}$ pour $n = 100$:

$$S = \frac{100 \times 101}{2} = \frac{10\,100}{2} = 5\,050.$$

L'algorithme affiche bien 5 050.

4. Oubli de l'incréméntation : la variable i resterait bloquée à 1 ; la condition $i \leq 100$ serait toujours vraie : on obtiendrait une **boucle infinie** et le programme ne s'arrêterait jamais.

CORRIGÉ

Exercice 9 — Type Probatoire : chasse aux erreurs

1. Les cinq erreurs :

1. S n'est pas initialisée ($S \leftarrow 0$ manquant) ;
2. la condition $i < n$ exclut n lui-même : il faut $i \leq n$ (n « inclus ») ;
3. le Si n'est pas fermé : il manque FinSi avant $i \leftarrow i + 1$;
4. l'incréméntation $i \leftarrow i + 1$ est placée à l'intérieur du Si (elle ne s'exécuterait que pour les pairs) : elle doit être dans la boucle mais après le FinSi ;
5. aucun contrôle de saisie : l'énoncé exige n strictement positif, il faut vérifier $n > 0$.

2. Algorithme corrigé :

```

Algorithme SommePairs
Variables n, i, S : Entier
Debut
  Repeter
    Ecrire("Entrez un entier strictement positif : ")
    Lire(n)
  Jusqu'à n > 0
  S ← 0
  i ← 2
  Tant que i ≤ n Faire
    Si i mod 2 = 0 Alors
      S ← S + i
    FinSi
    i ← i + 1
  FinTantQue
  Ecrire("La somme vaut : ", S)
Fin

```

3. Trace pour $n = 10$:

Tour	i	S	Action
départ	2	0	initialisations
1	2	2	pair : $S \leftarrow 0 + 2$
2	3	2	impair : rien
3	4	6	pair : $S \leftarrow 2 + 4$
4	5	6	impair : rien
5	6	12	pair : $S \leftarrow 6 + 6$
6	7	12	impair : rien
7	8	20	pair : $S \leftarrow 12 + 8$
8	9	20	impair : rien
9	10	30	pair : $S \leftarrow 20 + 10$

Quand i devient 11, la condition $i \leq 10$ est fausse : sortie de boucle. Message affiché : « La somme vaut : 30 ». Vérification : $2 + 4 + 6 + 8 + 10 = 30$. ✓

4. Gravité de l'oubli d'initialisation : une variable non initialisée contient une valeur imprévisible (résidu mémoire). Le calcul $S \leftarrow S + i$ part alors d'une valeur fautive : le résultat est faux de façon aléatoire, parfois juste « par chance ». C'est un bug d'autant plus dangereux qu'il ne se manifeste pas toujours.

Barème indicatif (10 pts) : identification des 5 erreurs (2,5 pts) ; algorithme corrigé (3 pts) ; trace correcte (2,5 pts) ; justification de la question 4 (2 pts).

Points de vigilance : citer explicitement `FinSi` manquant ; bien distinguer condition de continuation ($i \leq n$) et condition d'arrêt ; la trace doit montrer la sortie de boucle.

CORRIGÉ Exercice 10 — Type Probatoire : trace d'exécution

1. Trace pour $n = 5\,382$:

Tour	n	$r = n \bmod 10$	c	n devient
1	5 382	2	1	538
2	538	8	2	53
3	53	3	2	5
4	5	5	2	0

La condition $n > 0$ devient fausse : sortie de boucle.

2. Valeur affichée : 2.

3. Trace pour $n = 7\,135$: r vaut successivement 5, 3, 1, 7 — tous impairs — donc c reste à 0.
Valeur affichée : 0.

4. Conclusion : cet algorithme compte le nombre de chiffres pairs contenus dans l'entier n saisi. ($n \bmod 10$ isole le dernier chiffre, $n \div 10$ le supprime.)

5. Cas $n = 0$: la condition $n > 0$ est fausse dès le départ, le corps de boucle n'est jamais exécuté ; l'algorithme affiche $c = 0$. C'est cohérent : 0 ne contient aucun chiffre pair « utile » dans cette logique.

Barème indicatif (10 pts) : trace 5 382 (3 pts) ; valeur affichée (1 pt) ; trace 7 135 (2 pts) ; rôle de l'algorithme exprimé clairement (2 pts) ; cas $n = 0$ justifié (2 pts).

Points de vigilance : exiger la justification « condition fausse dès le départ $\Rightarrow$ zéro passage dans la boucle » ; le rôle doit être formulé en une phrase complète, pas seulement « il compte ».

CORRIGÉ Exercice 11 — Type Probatoire : recensement à Yaoundé

1. Algorithme :

```

Algorithme Recensement
Variables age, nbMineurs, nbMajeurs : Entier
Debut
  nbMineurs <- 0
  nbMajeurs <- 0
  Repeter
    Ecrire("Entrez un age (negatif pour terminer) : ")
    Lire(age)
    Si age >= 0 Alors
      Si age < 18 Alors
        nbMineurs <- nbMineurs + 1
      Sinon
        nbMajeurs <- nbMajeurs + 1
    FinSi
  FinSi
  Jusqu'a age < 0
  Ecrire("Mineurs : ", nbMineurs)
  Ecrire("Majeurs : ", nbMajeurs)

```

Fin

2. Répéter ... Jusqu'à est adaptée car on doit saisir au moins une valeur avant de pouvoir décider d'arrêter : le test porte sur la donnée saisie, qui n'existe qu'après la première lecture. Condition d'arrêt exacte : $\text{age} < 0$ (la valeur négative est une sentinelle, non comptée grâce au test $\text{age} \geq 0$).

3. Ajout de l'âge maximum :

```

ageMax <- 0
...
Si age >= 0 Alors
  Si age > ageMax Alors
    ageMax <- age
  FinSi
... compteurs ...
FinSi
...
Ecrire("Age le plus eleve : ", ageMax)

```

4. Trace avec 12, 45, 17, 18, 8, -1 :

Saisie	nbMineurs	nbMajeurs	Remarque
12	1	0	mineur
45	1	1	majeur
17	2	1	mineur
18	2	2	majeur ($18 \geq 18$)
8	3	2	mineur
-1	3	2	sentinelle : arrêt, non comptée

Affichage final : « Mineurs : 3 » et « Majeurs : 2 ».

5. Refus des âges > 130 : on ajoute un test intermédiaire qui ignore la saisie invalide sans arrêter la boucle :

```

Si age > 130 Alors
  Ecrire("Age invalide, ressaisissez")
Sinon Si age >= 0 Alors
  ... compteurs et maximum ...
FinSi

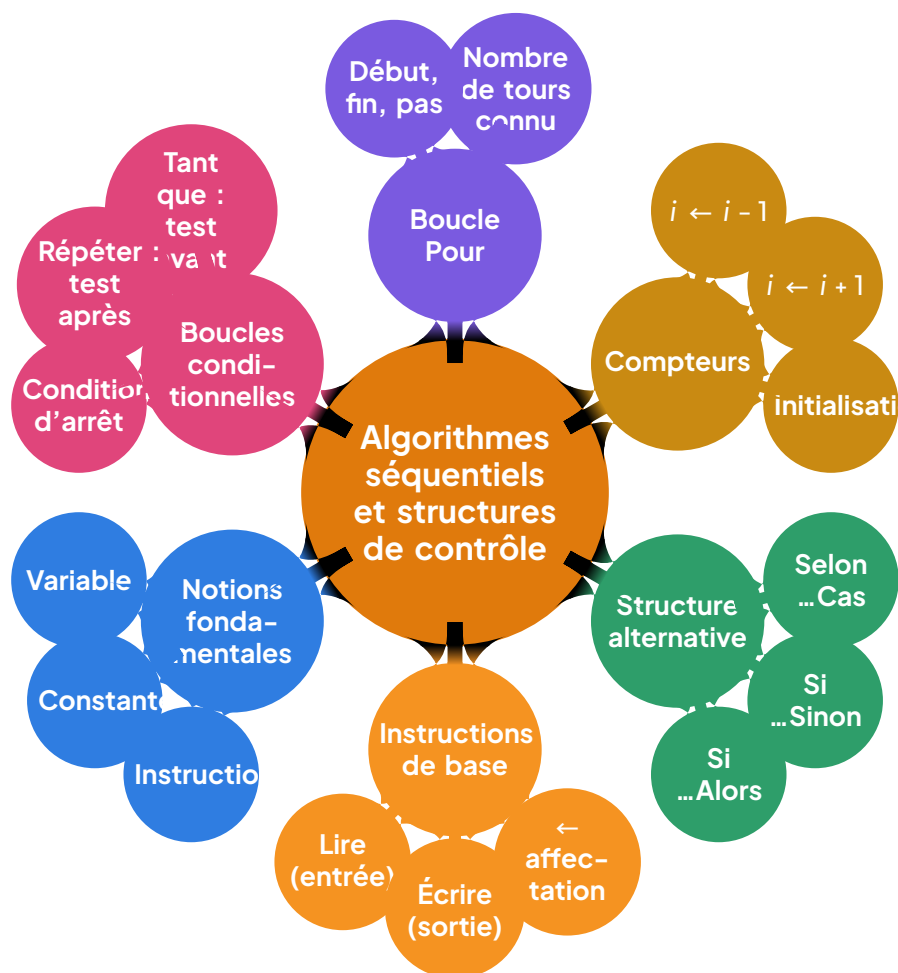
```

La boucle continue car la condition d'arrêt reste uniquement $\text{age} < 0$.

Barème indicatif (10 pts) : algorithme correct avec sentinelle (3 pts) ; justification du choix de boucle et condition d'arrêt (2 pts) ; âge maximum (2 pts) ; trace exacte (2 pts) ; gestion des âges invalides (1 pt).

Points de vigilance : la sentinelle ne doit pas être comptée (test $\text{age} \geq 0$ obligatoire) ; attention à la borne 18 : « mineur » signifie $\text{age} < 18$, donc 18 est majeur ; initialiser les compteurs à 0 avant la boucle.

13 FICHE BILAN



Carte mentale de la leçon : les six grandes familles de savoirs autour des algorithmes séquentiels et des structures de contrôle.

FICHE BILAN L'essentiel à connaître pour l'examen

1. Définitions clés

- **Algorithme** : suite finie et ordonnée d'**instructions** permettant de résoudre un problème. Un algorithme est **séquentiel** lorsque ses instructions s'exécutent dans l'ordre, de la première à la dernière, sans saut ni répétition.
- **Variable** : zone mémoire nommée dont le contenu peut changer au cours de l'exécution. Elle possède un nom (identifiant), un type (Entier, Réel, Caractère, Chaîne, Booléen) et une valeur.

- **Constante** : zone mémoire nommée dont la valeur est fixée à la déclaration et ne peut jamais être modifiée (ex. $\pi = 3,14$).
- **Instruction** : ordre élémentaire exécuté par la machine (lecture, écriture, affectation).
- **Structure de contrôle** : instruction qui modifie l'ordre d'exécution : alternative (choix) ou itérative (répétition).

2. Syntaxes à connaître par cœur

Notion	Syntaxe algorithmique	Rôle / condition d'emploi
Déclaration	Variable age : Entier	Toute variable est déclarée avant usage.
Constante	Constante TVA = 19,25	Valeur fixe, jamais affectée ensuite.
Lecture	Lire(age)	Entrée d'une donnée au clavier.
Écriture	Écrire("Bonjour", nom)	Affichage d'un résultat à l'écran.
Affectation	total $\leftarrow$ total + prix	Le membre de droite est évalué, puis rangé dans la variable de gauche.
Alternative	Si cond Alors ... Sinon ... FinSi	Le Sinon est facultatif.
Choix multiple	Selon x Cas 1 : ... FinSelon	Plusieurs cas selon la valeur d'une variable.
Pour	Pour i de 1 à n Faire ... FinPour	Nombre d'itérations connu à l'avance.
Tant que	TantQue cond Faire ... FinTantQue	Test avant : 0 tour possible.
Répéter	Répéter ... Jusqu'à cond	Test après : au moins 1 tour.

3. Formules et schémas types

- **Initialisation** : donner une valeur de départ avant la boucle : $s \leftarrow 0$ (somme), $cpt \leftarrow 0$ (compteur).
- **Incrémentation** : $i \leftarrow i + 1$; **décrément** : $i \leftarrow i - 1$.
- **Accumulateur** : $s \leftarrow s + x$; **produit** : $p \leftarrow p * x$ (initialisé à 1).
- **Nombre de tours d'un Pour** : i de a à b (pas de 1) : $b - a + 1$.
- **Échange de deux variables** : il faut une variable temporaire : $tmp \leftarrow a$; $a \leftarrow b$; $b \leftarrow tmp$.

4. Méthodes express

- **Écrire un algorithme** : (1) comprendre le problème et lister entrées/sorties ; (2) déclarer variables et constantes ; (3) lire les données ; (4) traiter ; (5) écrire les résultats.

- Choisir la boucle : nombre de tours connu $\Rightarrow$ Pour ; inconnu avec test en entrée $\Rightarrow$ TantQue ; inconnu avec au moins un passage $\Rightarrow$ Répéter.
- Tracer (débuguer) un algorithme : construire un tableau d'évolution des variables ligne par ligne.
- Éviter la boucle infinie : vérifier que le corps de la boucle fait progresser la variable de la condition d'arrêt.

À RETENIR Checklist avant le Bac

- ☐ Je sais définir et distinguer **variable**, **constante**, **instruction** et **algorithme séquentiel**.
- ☐ Je déclare toujours mes variables avec un type correct avant de les utiliser.
- ☐ Je maîtrise les trois instructions de base : Lire, Écrire, affectation ($\leftarrow$).
- ☐ Je sais que dans $a \leftarrow a + 1$, le membre de droite est évalué avec l'ancienne valeur de a .
- ☐ Je sais écrire un Si ...Alors ...Sinon ...FinSi et un Selon ...Cas.
- ☐ Je sais initialiser un compteur ou une somme avant la boucle, jamais dedans.
- ☐ Je distingue incrémentation ($i \leftarrow i + 1$) et décrémentation ($i \leftarrow i - 1$).
- ☐ Je choisis la bonne boucle : Pour si le nombre de tours est connu, sinon TantQue ou Répéter.
- ☐ Je sais que TantQue peut ne jamais s'exécuter, alors que Répéter s'exécute au moins une fois.
- ☐ Je sais identifier la condition d'arrêt d'une boucle et garantir qu'elle sera atteinte (pas de boucle infinie).
- ☐ Je sais tracer un algorithme avec un tableau d'évolution des variables.
- ☐ Je ferme toujours mes structures : FinSi, FinPour, FinTantQue, Jusqu'à.

**Signé • OnBuch+**Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**